

Royaume du Maroc
Universite Sultan Moulay Slimane
Ecole Superieure de Technologie - Beni Mellal
Departement Informatique et Techniques de Gestion

Filiere : Cybersecurite
Memoire de Projet de Fin d'Etudes

Ernevil

Plateforme de supervision et de gestion centralisee des postes
d'un reseau local

Realise par : Prenom NOM
Etablissement d'origine : Ecole Superieure de Technologie de Beni Mellal
Organisme d'accueil : ISTA NTIC Beni Mellal
Service : Administration
Encadrant pedagogique : Mme.
Encadrant professionnel : M.
Periode : Du 30/03/2026 au 30/06/2026
Annee universitaire : 2025–2026

Remerciements

Je remercie les responsables, les formateurs et le personnel administratif de l'ISTA NTIC de Beni Mellal pour leur accueil et pour la confiance accordee pendant la periode du stage. Les interventions realisees dans les salles, les echanges autour des besoins de supervision et l'observation du fonctionnement quotidien d'un etablissement de formation ont donne un cadre concret au projet Ernevil.

Je remercie egalement mon encadrant professionnel pour les remarques pratiques concernant la maintenance des postes, la gestion des laboratoires et les contraintes de securite dans un environnement partage par plusieurs groupes. Ces retours ont oriente plusieurs choix du projet : automatiser l'inventaire, reduire les manipulations manuelles, centraliser les informations et distinguer les postes disponibles des postes indisponibles.

Je remercie enfin mon encadrant pedagogique a l'EST de Beni Mellal pour le suivi academique, les orientations methodologiques et les remarques qui ont aide a transformer un prototype logiciel en un memoire structure, argumente et soutenable.

Resume

Ce memoire presente la conception et la realisation d'Ernevil, une plateforme de supervision et de gestion centralisee des postes informatiques d'un reseau local. Le projet a ete developpe dans le cadre d'un stage effectue a l'ISTA NTIC de Beni Mellal, au sein du service administration, pendant la periode allant du 30 mars au 30 juin 2026.

Ernevil repond a un besoin observe dans les salles informatiques : disposer d'une vision claire sur les machines connectees, leur etat, leur configuration systeme, leurs interfaces reseau, leur memoire, leurs partitions, leurs processus, leurs utilisateurs et certains evenements materiels comme les connexions USB. La solution s'appuie sur un agent Python installe sur les postes, un serveur FastAPI, une couche de persistance SQLAlchemy et une interface Streamlit pour l'administration.

Le travail presente dans ce memoire couvre l'analyse du besoin, l'etude de l'existant, la conception de l'architecture, les choix techniques, l'implementation des modules backend et agent, les mecanismes de supervision, les alertes, les commandes distantes, la base de donnees, les limites de securite et les scenarios de validation. La partie relative a l'intelligence artificielle presente un module de detection d'intrusion par apprentissage supervise. Un modele HistGradientBoostingClassifier a ete entraine sur le jeu de donnees CIC-IDS-2017 avec 18 caracteristiques de flux reseau, atteignant environ 95% de precision apres optimisation par GridSearchCV. Le modele est integre dans l'agent sous forme de pipeline temps reel : capture reseau par scapy, construction des flux, inference et emission d'alertes par WebSocket vers le serveur.

Mots-cles : supervision, cybersecurite, FastAPI, Streamlit, SQLAlchemy, agent Python, WebSocket, heartbeat, inventaire, reseau local, USB, Ernevil.

Abstract

This final-year project report presents Ernevil, a centralized monitoring and management platform for computers connected to a local area network. The project was developed during an internship at ISTA NTIC Beni Mellal, within the administration department, from March 30 to June 30, 2026.

Ernevil addresses a practical need observed in computer laboratories : administrators need a reliable view of connected devices, their availability, system information, memory usage, network interfaces, disks, processes, active users and selected hardware events such as USB connections. The implemented solution is based on a Python agent, a FastAPI backend, SQLAlchemy persistence and a Streamlit administration interface.

The report details the internship context, requirements analysis, architectural design, implementation, security considerations, monitoring mechanisms, alert logic, validation scenarios and limitations. The current repository does not include a trained machine learning model ; therefore, the artificial intelligence section focuses only on implemented detection mechanisms and future integration perspectives.

Table des matières

Table des figures

Liste des tableaux

Chapitre 1

Introduction generale

1.1 Contexte general

Les salles informatiques d'un etablissement de formation sont des environnements dynamiques. Les postes sont utilises par plusieurs groupes, les systemes sont modifies pendant les travaux pratiques, les peripheriques sont branches et retires, et les incidents simples peuvent rapidement perturber une seance entiere. Dans ce contexte, l'administration ne se limite pas a reparer une panne visible. Elle consiste aussi a connaitre l'etat du parc, anticiper les indisponibilites, identifier les changements significatifs et disposer d'informations exploitables sans devoir passer physiquement devant chaque machine.

Le projet Ernevil est ne de cette observation. Il vise a fournir une plateforme locale de supervision capable de recenser automatiquement des postes, de stocker leurs caracteristiques, de suivre leur disponibilite, de remonter des donnees systeme et d'offrir une interface de consultation et d'action. Le projet ne cherche pas a remplacer une solution professionnelle complete de type EDR, SIEM ou outil de gestion de parc mature. Il cherche plutot a construire une base technique maitrisee, adaptee a un environnement de formation, ouverte a l'evolution et suffisamment concrete pour etre testee sur un reseau local.

1.2 Problematique

La problematique traitee peut etre formulee ainsi : comment concevoir une plateforme legere, extensible et deployable localement permettant a un administrateur de superviser plusieurs postes, d'observer leurs informations techniques, de detecter leur indisponibilite et de declencher certaines actions a distance, tout en gardant une architecture comprehensible et ameliorable ?

Cette question contient plusieurs sous-problemes. Le premier concerne l'identification des machines : un poste ne doit pas etre confondu avec un autre lorsqu'il redemarre,

change d'adresse IP ou modifie son nom. Le deuxieme concerne la fraicheur des donnees : une information systeme n'a de valeur que si elle correspond a l'etat actuel ou a un etat recent. Le troisieme concerne la securite : la collecte et l'execution de commandes distantes sont sensibles et doivent etre analysees avec rigueur. Le quatrieme concerne l'ergonomie : une plateforme de supervision doit aider a comprendre rapidement la situation sans noyer l'utilisateur dans des donnees brutes.

1.3 Objectifs du memoire

Ce memoire poursuit trois objectifs. Le premier est de presenter le contexte de stage et l'organisme d'accueil de maniere fidele au rapport existant. Le second est d'analyser et de documenter Ernevil comme un projet logiciel complet : besoin, conception, architecture, implementation, validation et limites. Le troisieme est d'adopter une posture critique : expliquer ce qui fonctionne, ce qui reste incomplet, ce qui doit etre securise et ce qui pourrait etre transforme en version plus robuste.

1.4 Organisation du document

Le document est structure en huit chapitres. Le premier chapitre introduit le contexte et la problematique. Le deuxieme presente le stage et l'organisme d'accueil. Le troisieme developpe l'analyse du besoin et l'etude de l'existant. Le quatrieme traite la conception. Le cinquieme constitue le coeur technique du memoire et detaille la realisation. Le sixieme analyse la partie detection et les perspectives d'intelligence artificielle. Le septieme presente les tests et la validation. Le dernier chapitre conclut le projet et ouvre les perspectives.

Chapitre 2

Stage et organisme d'accueil

2.1 Présentation de l'OFPPT

L'Office de la Formation Professionnelle et de la Promotion du Travail occupe une place centrale dans la formation professionnelle au Maroc. Sa mission est de former des profils opérationnels dans des secteurs variés, notamment l'informatique, les réseaux, le développement, l'administration système et les technologies numériques. Dans un tel cadre, les équipements informatiques ne sont pas uniquement des outils administratifs ; ils constituent le support principal des apprentissages pratiques.

Le stage s'est déroulé à l'ISTA NTIC de Beni Mellal. Cet organisme accueille des apprenants qui manipulent quotidiennement des postes, des logiciels, des environnements réseau et des supports pédagogiques numériques. Cette intensité d'utilisation rend la maintenance informatique visible et directement liée à la qualité de la formation. Une panne de réseau, un poste indisponible ou une configuration incohérente peut bloquer un exercice, retarder un groupe ou mobiliser un formateur sur une tâche qui n'est pas pédagogique.

2.2 Environnement professionnel

Le service administration constitue un point de contact entre les besoins pédagogiques et les contraintes techniques. Les travaux observés ou réalisés pendant le stage concernaient principalement la disponibilité des salles, la connectivité, l'accompagnement ponctuel des utilisateurs et la recherche de solutions simples pour améliorer le suivi des postes.

L'environnement se caractérise par plusieurs contraintes concrètes. Les salles sont partagées entre plusieurs groupes. Les postes peuvent être utilisés pour des travaux pratiques différents selon les modules. Certains changements ne sont pas toujours documentés. Les demandes arrivent souvent sous forme de symptômes : un poste

qui ne se connecte plus, une salle qui ne repond pas correctement, un besoin de verifier rapidement les machines disponibles. Ces observations ont donne du sens au developpement d'une plateforme comme Ernevil.

2.3 Missions realisees

Les missions realisees pendant le stage peuvent etre regroupees en quatre categories. La premiere concerne la maintenance et l'administration reseau : verification de liaisons, diagnostic de connectivite, remise en ordre de configurations et suivi des postes. La deuxieme concerne l'assistance aux utilisateurs et aux formateurs : aide lors d'incidents, accompagnement sur certains outils et participation a la continuite des travaux pratiques. La troisieme concerne l'observation des besoins recurrents : inventaire, disponibilite, supervision et centralisation. La quatrieme concerne la conception et le developpement du projet Ernevil.

TABLE 2.1 – Synthese des missions et lien avec Ernevil

Mission	Observation terrain	Lien avec le projet
Diagnostic reseau	Les incidents de connectivite demandent une verification poste par poste.	Collecte des interfaces reseau et affichage des adresses IP.
Suivi des postes	La disponibilite d'une machine n'est pas toujours connue avant une seance.	Heartbeat et indicateur <code>is_alive</code> .
Maintenance systeme	Les processus et l'etat memoire influencent l'utilisation des postes.	Collecte via <code>psutil</code> et consultation dans Streamlit.
Gestion des salles	Les emplacements physiques doivent etre associes aux machines.	Preparation d'un module de localisation.
Securite	Les peripheriques USB et les commandes distantes sont sensibles.	Detection USB et analyse des risques d'execution distante.

2.4 Difficultes rencontrees

La premiere difficulte a ete de passer d'un besoin general, "mieux superviser les postes", a un perimetre implementable. Une plateforme de supervision peut vite devenir trop large : inventaire, alertes, gestion d'utilisateurs, historiques, graphiques, commandes,

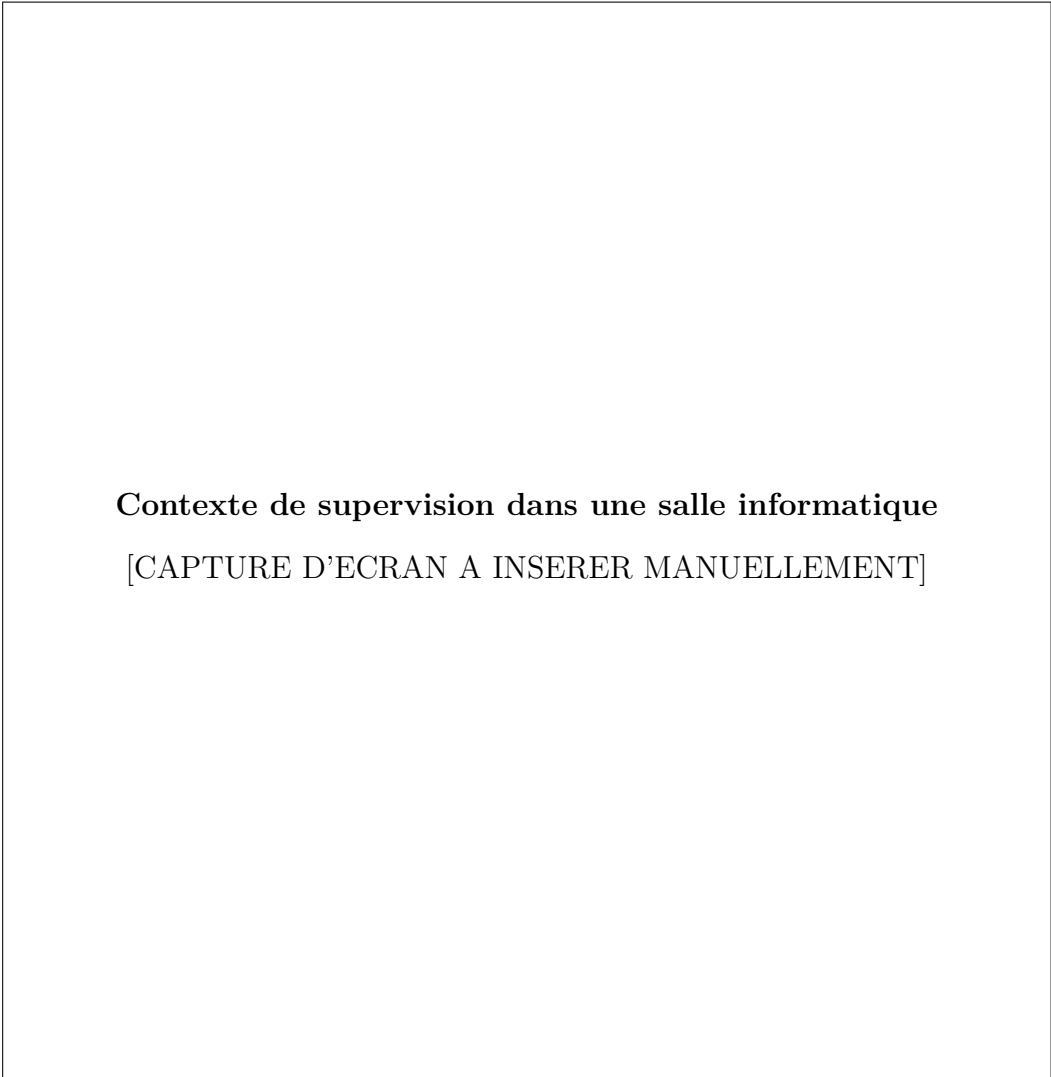
securite, reporting. Le choix a donc ete de construire d'abord un socle : agent, serveur, base de donnees, interface et mecanisme de presence.

La deuxieme difficulte a concerne l'heterogeneite des systemes. Le code de l'agent distingue les environnements Unix et Windows, notamment pour la recuperation des utilisateurs et l'execution des commandes. Cette distinction montre que la portabilite n'est pas automatique. Une fonction simple sur Linux peut demander une approche differente sur Windows, par exemple pour les commandes d'arret ou de redemarrage.

La troisieme difficulte a ete liee a la securite. Ernevil manipule des fonctions puissantes : identification des machines, collecte de processus, surveillance USB et execution de commandes via WebSocket. Ces fonctions sont utiles pour l'administration, mais elles deviennent dangereuses si elles sont exposees sans controle strict. Le memoire documente donc les limites actuelles au lieu de les masquer.

2.5 Competences acquises

Le stage a permis de consolider des competences techniques et methodologiques. Sur le plan technique, le projet a mobilise Python, FastAPI, SQLAlchemy, Streamlit, WebSockets, `psutil`, la collecte USB et les principes d'une architecture agent-serveur. Sur le plan methodologique, il a impose une demarche progressive : observer le besoin, definir un perimetre, prototyper, tester, corriger et documenter les limites.



Contexte de supervision dans une salle informatique
[CAPTURE D'ECRAN A INSERER MANUELLEMENT]

FIGURE 2.1 – Capture d'écran a inserer : Contexte de supervision dans une salle informatique

Chapitre 3

Analyse et etude du projet Ernevil

3.1 Contexte du projet

Ernevil est une plateforme open source de supervision locale. Le depot contient trois ensembles principaux : un agent Python installe sur les machines supervisees, un serveur FastAPI qui recoit et expose les donnees, et une interface Streamlit qui permet a l'administrateur de consulter les informations et de declencher certaines actions.

Le contexte n'est pas celui d'une grande entreprise disposant deja d'un SOC mature. Il s'agit plutot d'un reseau local de formation dans lequel l'objectif prioritaire est la visibilite. Avant de parler de detection avancee, il faut savoir quels postes existent, lesquels sont actifs, quelles interfaces reseau sont configurees, quels utilisateurs sont visibles, quelles partitions sont montees et quels processus tournent.

3.2 Enjeux de cybersecurite

La supervision des postes a un lien direct avec la cybersecurite. Un administrateur ne peut pas proteger efficacement un parc qu'il ne connait pas. L'absence d'inventaire rend difficile la detection d'une machine inconnue. L'absence de suivi de disponibilite complique l'identification d'un poste arrete ou isole. L'absence de vision sur les processus limite la capacite a reperer une activite inhabituelle. L'absence de suivi USB peut masquer des comportements a risque dans un environnement ou les peripheriques amovibles circulent facilement.

Dans Ernevil, ces enjeux sont abordes par des mecanismes simples mais concrets. L'agent envoie une empreinte derivee de l'identifiant machine. Le serveur conserve les postes dans une base de donnees. Un heartbeat actualise la date de derniere presence. Les informations systeme sont stockees dans des tables separees. Les WebSockets permettent une communication serveur-agent pour les commandes et les alertes.

3.3 Etude de l'existant

Plusieurs types de solutions existent pour superviser un parc informatique. Les outils de monitoring classiques comme Zabbix ou Nagios sont puissants pour la disponibilité et les métriques. Les solutions d'inventaire comme GLPI avec FusionInventory ou OCS Inventory sont adaptées à la gestion de parc. Les EDR visent la détection et la réponse à un incident. Les SIEM centralisent les journaux et les événements.

Ernevil ne prétend pas concurrencer ces solutions. Son intérêt pédagogique et technique réside dans la construction d'une chaîne complète et maîtrisable. Le projet permet de comprendre les briques fondamentales : agent, API, base de données, interface, présence, collecte, commandes et alertes. Cette maîtrise est importante dans un mémoire de cybersécurité, car elle montre comment une plateforme de supervision fonctionne sous le capot.

TABLE 3.1 – Comparaison entre Ernevil et quelques familles de solutions

Famille	Points forts	Limites pour le contexte du projet	Position d'Ernevil
Monitoring classique	Supervision mature, alertes, historiques, tableaux de bord.	Configuration parfois lourde pour un prototype pédagogique.	Comprendre et reproduire un socle léger.
Inventaire de parc	Fiches machines, matériel, logiciels, tickets.	Moins centré sur les commandes temps réel et l'expérimentation agent.	Inventaire technique simplifié et extensible.
EDR	Détection avancée, réponse, télémétrie de sécurité.	Coût, complexité, exigences fortes de sécurité.	Base pédagogique de détection et de réaction.

SIEM	Correlation d'évenements, journalisation, regles.	Necessite beaucoup de sources et de normalisation.	Perspective future de centralisation d'évenements.
------	---	--	--

3.4 Besoins fonctionnels

Les besoins fonctionnels identifiés sont directement liés au dépôt. La plateforme doit permettre l'enregistrement automatique d'un poste par l'agent. Elle doit stocker les informations principales : nom de machine, système d'exploitation, état de vie, heure de démarrage, empreinte, mémoire, interfaces réseau, partitions, processus, utilisateurs et périphériques USB. Elle doit exposer ces informations via une API REST. Elle doit afficher les postes disponibles dans une interface. Elle doit permettre la sélection d'une machine en ligne et la consultation de ses données. Elle doit offrir un canal de communication temps réel pour l'exécution de commandes et les alertes.

3.5 Besoins non fonctionnels

Les besoins non fonctionnels sont aussi importants que les fonctionnalités visibles. La solution doit rester légère afin de fonctionner dans un environnement local. Elle doit être compréhensible par un étudiant ou un administrateur qui veut l'adapter. Elle doit séparer les responsabilités : collecte côté agent, persistance côté serveur, consultation côté interface. Elle doit être extensible, car le module de localisation, les rapports, la blacklist et les alertes réseau sont présents dans l'interface mais encore incomplets dans le code.

3.6 Contraintes techniques

Le projet est écrit en Python et cible Python 3.10 ou plus. Les dépendances déclarées incluent FastAPI, SQLAlchemy, Streamlit, `psutil`, `requests`, `websockets`, `pyusb`, `usb-monitor`, `ifaddr` et `py-machineid`. La base configurée dans le dépôt est SQLite via `sqlite:///./database/data.db`, même si le cahier d'objectif mentionne une intégration PostgreSQL. Il faut donc distinguer l'architecture actuelle et l'évolution visée : l'ORM SQLAlchemy facilite une migration vers PostgreSQL, mais le code fourni n'utilise pas encore PostgreSQL directement.

3.7 Contraintes de securite

La securite est le point le plus sensible du projet. L'authentification des agents repose actuellement sur le couple `computer_id` et `fingerprint`. Ce mecanisme limite certaines mises a jour non autorisees si l'empreinte n'est pas connue, mais il ne constitue pas une authentification forte. Les routes de consultation utilisent souvent des requetes SQL textuelles avec interpolation de parametres. Cette pratique doit etre remplacee par des requetes parametrees ou par l'API ORM de SQLAlchemy. Les WebSockets de commande permettent d'executer des instructions sur un poste agent, ce qui impose une authentification stricte, une autorisation par role, une journalisation et idealement une liste blanche de commandes.

3.8 Analyse des risques

TABLE 3.2 – Analyse des risques adaptee a Ernevil

Risque	Cause possible	Impact	Mesure recommandee
Usurpation d'agent	Empreinte recuperee ou absence de secret fort.	Envoi de fausses donnees.	Jetons d'enrolement, rotation de secrets, TLS.
Injection SQL	Requetes textuelles interpolées.	Lecture ou modification non autorisee.	Requetes parametrees et ORM.
Commande distante abusive	WebSocket non protege.	Prise de controle d'un poste.	RBAC, authentification forte, audit, liste blanche.
Donnees obsoletes	Agent arrete ou heartbeat absent.	Mauvaise interpretation de l'etat.	Seuils explicites, historique, alerte d'indisponibilite.
Fuite d'informations	Exposition de processus, utilisateurs, IP.	Atteinte a la confidentialite.	Controle d'accès et chiffrement.
Faux positifs USB	Peripheriques legitimes non distingues.	Alertes peu utiles.	Inventaire de confiance et niveaux de severite.

3.9 Objectifs retenus

Les objectifs retenus pour la version actuelle sont volontairement progressifs : enregistrer un poste, le maintenir identifiable, collecter ses données principales, afficher les machines, suivre leur présence, permettre des actions distantes de base et préparer le terrain pour des alertes plus avancées. Les objectifs non finalisés, comme l'authentification utilisateur complète, PostgreSQL en production, la blacklist opérationnelle ou un moteur d'IA, sont traités comme des perspectives et non comme des fonctionnalités achevées.

Chapitre 4

Conception

4.1 Choix technologiques

Le choix de Python s'explique par la disponibilité de bibliothèques système, la facilité de prototypage et la cohérence entre agent, serveur et interface. `psutil` fournit une API stable pour la mémoire, les processus, les utilisateurs, les partitions et le temps de démarrage. `ifaddr` facilite la récupération des interfaces réseau. `pyusb` et `usb-monitor` permettent de travailler sur les périphériques USB. FastAPI offre une structure claire pour les routes REST et WebSocket. SQLAlchemy donne une couche d'abstraction pour la base. Streamlit permet de produire rapidement une interface administrative exploitable.

TABLE 4.1 – Justification des principaux choix techniques

Technologie	Role	Justification
Python	Langage principal	Même langage pour agent, backend et scripts, bibliothèques système disponibles.
FastAPI	API et WebSocket	Typage, validation Pydantic, routes claires, support asynchrone.
SQLAlchemy	Persistance	Modélisation relationnelle, migration future possible vers PostgreSQL.
SQLite	Base actuelle	Simplicité de développement local, zéro serveur externe.
Streamlit	Interface	Rapidité de création de tableaux de bord et pages administratives.

psutil	Collecte systeme	Acces portable aux metriques et processus.
websockets	Communication agent	Canal bidirectionnel pour commandes et alertes.

4.2 Architecture logique

L'architecture logique est organisee autour de trois couches. La premiere couche est l'agent, responsable de la collecte locale et de la communication avec le serveur. La deuxieme couche est le serveur, responsable de l'API, de l'authentification agent minimale, de la persistance et des WebSockets. La troisieme couche est l'interface, responsable de la presentation et des actions administratives.

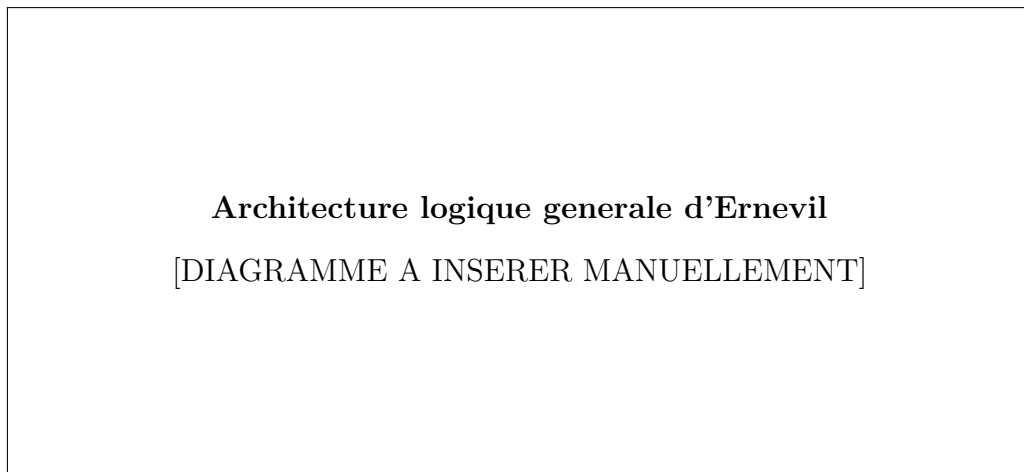


FIGURE 4.1 – Emplacement reserve : Architecture logique generale d'Ernevil

4.3 Strategie de communication

Deux modes de communication coexistent. Le premier est REST : l'agent envoie les donnees initiales avec une requete `POST`, puis les mises a jour avec `PUT`, `PATCH` ou `DELETE`. L'interface Streamlit lit les donnees avec des requetes `GET`. Le deuxieme mode est WebSocket : le serveur garde une connexion active avec l'agent pour transmettre une commande et recevoir le resultat.

Cette separation est pertinente. Les donnees periodiques n'ont pas besoin d'un canal permanent : REST suffit. Les commandes distantes, en revanche, ont besoin d'un canal capable de pousser une instruction vers l'agent. Le WebSocket repond a ce besoin, mais il augmente fortement les exigences de securite.

4.4 Conception de la base de donnees

La base est modelisee dans `server/database/dbschema.py`. La table principale est `computerInfo`. Elle est reliee aux tables `memoryinfo`, `netinfo`, `processesinfo`, `disksinfo`, `computerusers` et `usb_devices`. Une table `locations` existe pour rattacher un poste a un emplacement, et une table `users` prepare la gestion des utilisateurs administrateurs.

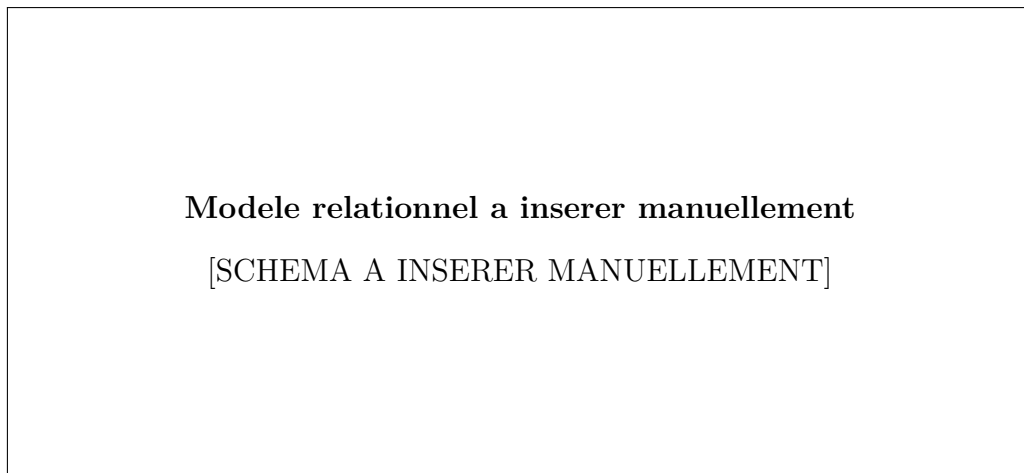


FIGURE 4.2 – Emplacement reserve : Modele relationnel a inserer manuellement

TABLE 4.2 – Tables principales du modele SQLAlchemy

Table	Responsabilite	Commentaire de conception
<code>computerInfo</code>	Identite et etat des postes.	Table pivot contenant l'identifiant, le nom, l'OS, l'empreinte, le heartbeat et l'etat.
<code>memoryinfo</code>	Etat memoire.	Une entree par poste dans l'etat actuel du code.
<code>netinfo</code>	Interfaces reseau.	Plusieurs lignes possibles par poste, une par interface.
<code>processesinfo</code>	Processus.	Remplace l'ensemble des processus a chaque mise a jour.
<code>disksinfo</code>	Partitions.	Stocke nom de partition, point de montage et type de systeme de fichiers.
<code>computerusers</code>	Utilisateurs visibles.	Liste les utilisateurs actifs ou connus selon le systeme.
<code>usb_devices</code>	Peripheriques USB.	Stocke fabricant, produit, vendor id et product id.

<code>locations</code>	Emplacements.	Prepare le rattachement des postes aux salles.
------------------------	---------------	--

4.5 Strategie de supervision

La supervision repose sur deux temporalites. La premiere est l'initialisation : l'agent construit une image de la machine et l'envoie au serveur. La deuxieme est la mise a jour : l'agent compare certaines donnees courantes avec les donnees conservees localement et envoie les changements. Le heartbeat est independant des mises a jour detaillees afin de distinguer la presence de la machine de la fraicheur de toutes ses metriques.

4.6 Conception du systeme d'alertes

Le code contient une base de systeme d'alertes USB. Le module `usb_monitor.py` ecoute les connexions et deconnexions de peripheriques, puis place les evenements dans une file. Le module `alert_handler.py` prepare l'envoi de ces evenements au serveur via WebSocket. Le serveur expose `/api/ws/alert`. Dans l'etat actuel, l'alerte est recue et affichee cote serveur, mais la persistance, la classification et l'affichage dans la page `netalerts.py` restent a finaliser.

4.7 Integration de l'intelligence artificielle

La conception prevoit une integration future de mecanismes de detection plus evolues. Toutefois, le depot actuel ne contient pas de modele entraine, pas de pipeline d'apprentissage, pas de fichier de modele et pas d'inference statistique. Il est donc plus juste de parler de preparation a la detection : collecte de donnees, structuration, evenements, seuils possibles et points d'extension.

4.8 Gestion des utilisateurs

Une table `users` est presente avec `username`, `email` et `password`. Elle montre l'intention de gerer des comptes administrateurs. Cependant, les routes d'inscription, de connexion, de hachage de mot de passe, de session et de role ne sont pas presentes dans l'etat actuel. La conception cible devrait inclure une authentication administrateur, une separation des roles et une journalisation des actions sensibles.

4.9 Conception de la securite

La securite cible doit couvrir quatre axes. Le premier est l'enrolement des agents : chaque agent devrait recevoir un secret unique lors de son ajout. Le deuxieme est le transport : les communications devraient utiliser HTTPS/WSS. Le troisieme est l'autorisation : toutes les commandes distantes doivent etre controlees. Le quatrieme est l'audit : chaque action doit etre journalisee avec l'identite de l'administrateur, la machine cible, l'horodatage et le resultat.

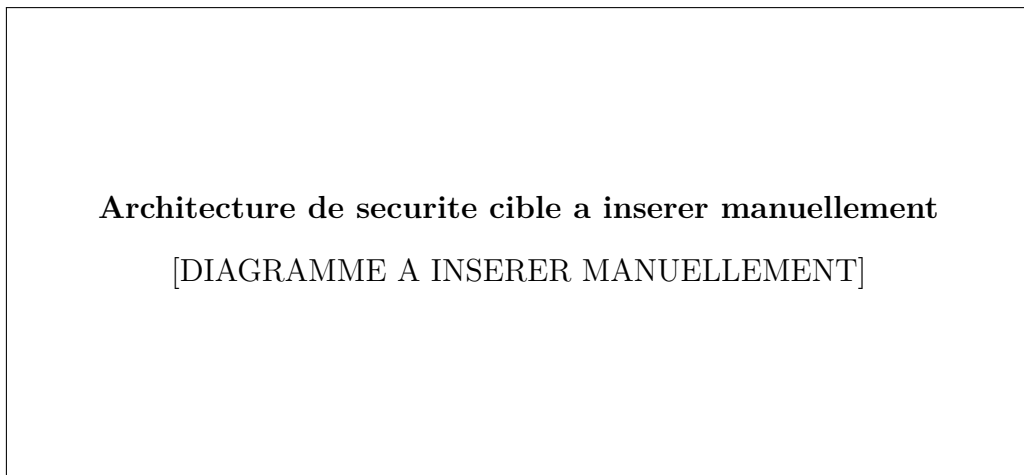


FIGURE 4.3 – Emplacement reserve : Architecture de securite cible a inserer manuellement

Chapitre 5

Realisation et implementation

5.1 Organisation du depot

Le depot est organise de maniere lisible. Le dossier **agent** contient l'agent et ses modules de collecte. Le dossier **server** contient le backend, la base, les routeurs, les schemas et l'interface. Le fichier `pyproject.toml` declare les dependances. Le fichier `README.md` presente Ernevil comme une plateforme open source de monitoring local avec une perspective cloud future.

```
1 Project-Ernevil/  
2   agent/  
3     agent.py  
4     collectors/  
5     utils/  
6     agentdata.json  
7     serverdata.json  
8   server/  
9     main.py  
10    routers/  
11    schemas/  
12    database/  
13    frontend/  
14  pyproject.toml  
15  README.md  
16  Rapport.tex
```

Listing 5.1 – Structure simplifiee du depot

5.2 Backend FastAPI

Le serveur est initialise dans `server/main.py`. Il cree les tables via SQLAlchemy puis inclut plusieurs routeurs : ajout de ressources, lecture, mise a jour, suppression, heartbeat, locations et WebSocket. Cette organisation par routeur evite de concentrer toute la logique dans un seul fichier.

```
1 app.include_router(add_resources.router ,
    prefix="/api/computers")
2 app.include_router(get_resources.router ,
    prefix="/api/computers")
3 app.include_router(update_resources.router ,
    prefix="/api/computers")
4 app.include_router(delete_resources.router ,
    prefix="/api/computers")
5 app.include_router(heartbeats.router , prefix="/api/computers")
6 app.include_router(locations.router , prefix="/api/locations")
7 app.include_router(websocket.router , prefix="/api")
```

Listing 5.2 – Initialisation des routeurs dans FastAPI

Au demarrage, le serveur lance une tache asynchrone `expireHeartBeat`. Cette tache parcourt periodiquement les postes et marque comme indisponibles ceux dont le dernier heartbeat depasse un seuil. Cette approche est simple et efficace pour une premiere version : elle evite d'attendre une action utilisateur pour recalculer l'etat d'une machine.

5.3 Persistance avec SQLAlchemy

La couche de persistance repose sur SQLAlchemy. Le fichier `db.py` definit le moteur, la session et la classe de base. Le choix actuel est SQLite, ce qui simplifie le developpement et les tests locaux. La presence de SQLAlchemy permet toutefois de migrer vers PostgreSQL en modifiant l'URL de connexion, puis en adaptant les migrations et les contraintes.

La modelisation montre une separation correcte entre l'identite du poste et ses donnees detaillees. Cette separation evite d'ajouter trop de colonnes dans une table unique. Elle facilite aussi l'evolution : il est possible d'ajouter une table d'historique memoire ou d'evenements sans casser la table principale.

5.4 Enregistrement automatique d'un agent

L'enregistrement est implemente dans `add_resources.py`. L'agent envoie un objet contenant les informations de la machine. Le serveur verifie si une machine possede deja

la meme empreinte. Si elle existe, il renvoie une erreur. Sinon, il cree l'entree principale, puis ajoute les informations memoire, reseau, processus, utilisateurs, disques et USB.

Ce mecanisme est important car il reduit l'administration manuelle. Dans un reseau local, il suffit de configurer l'adresse du serveur dans `serverdata.json`, puis de lancer l'agent. La machine s'inscrit d'elle-meme et devient visible dans l'interface.

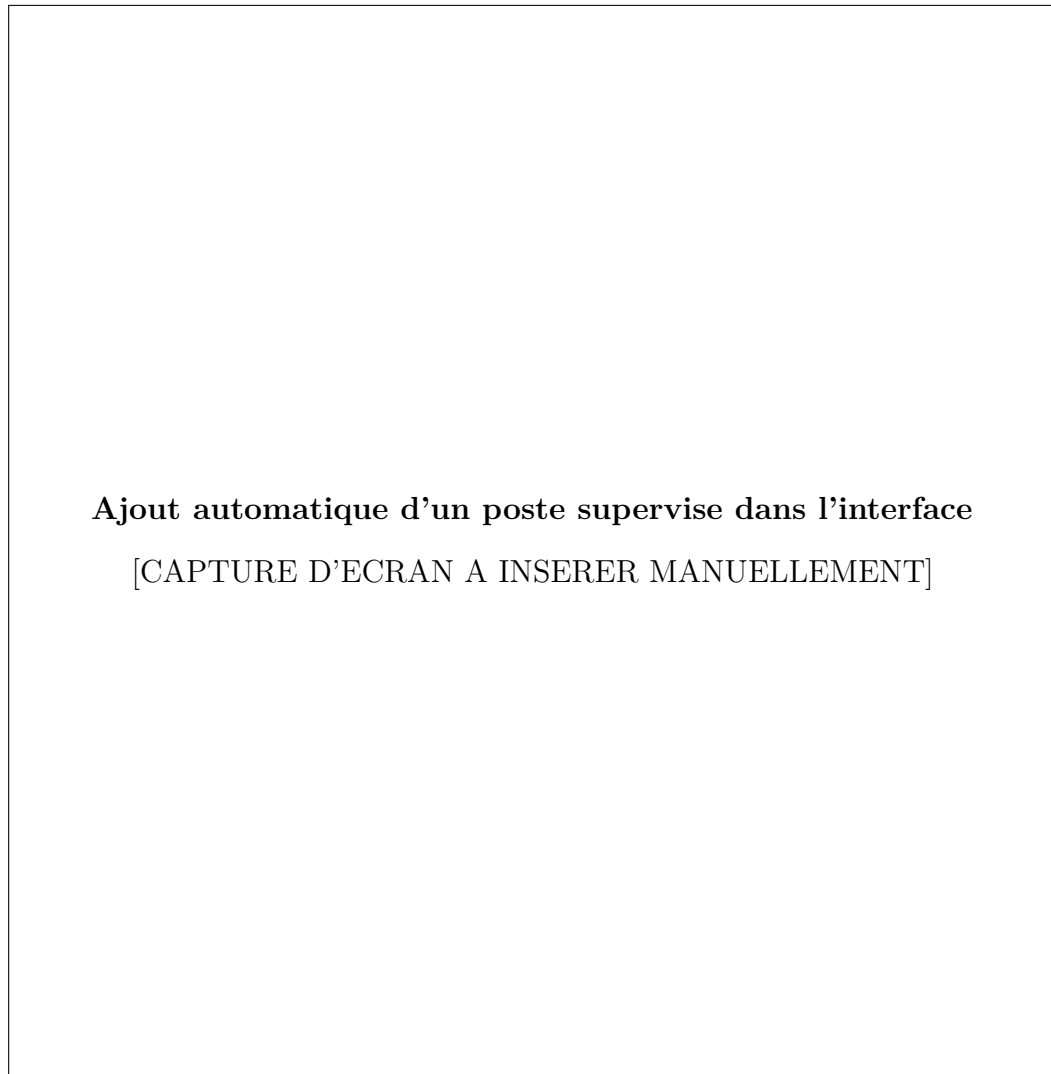


FIGURE 5.1 – Capture d'ecran a inserer : Ajout automatique d'un poste supervise dans l'interface

5.5 Identification par empreinte

L'agent calcule une empreinte SHA-256 a partir de l'identifiant machine fourni par `py-machineid`. Cette empreinte est envoyee au serveur et conservee en base. Elle est ensuite utilisee avec le `computer_id` pour authentifier les mises a jour.

Ce choix est meilleur qu'une identification uniquement par adresse IP, car une adresse IP peut changer. Il est aussi meilleur qu'une identification uniquement par nom

de machine, car plusieurs postes peuvent être renommés ou clones. Il reste cependant insuffisant comme mécanisme de sécurité fort : une empreinte transmise et stockée comme secret peut être récupérée. La prochaine version devrait introduire un jeton d'agent généré par le serveur.

5.6 Collecte des informations système

Le module `collectors/computer_info.py` regroupe les fonctions de collecte. Il récupère le nom de la machine, le système d'exploitation, les utilisateurs, le nombre de processeurs, l'usage CPU, la mémoire, les partitions, les interfaces réseau, les processus, le temps de démarrage, l'identifiant machine, l'adresse MAC et les périphériques USB.

TABLE 5.1 – Données collectées par l'agent

Donnée	Bibliothèque ou source	Utilité pour la supervision
Nom machine	<code>socket.gethostname()</code>	Identifier lisiblement le poste.
Système	<code>platform.system()</code>	Adapter l'affichage et les commandes.
Utilisateurs	<code>pwd</code> ou <code>psutil.users()</code>	Comprendre l'activité locale.
Mémoire	<code>psutil.virtual_memory()</code>	Suivre la pression mémoire.
Partitions	<code>psutil.disk_partitions()</code>	Inventorier les disques et montages.
Interfaces	<code>ifaddr</code>	Connaitre les adresses IPv4.
Processus	<code>psutil.process_iter()</code>	Examiner l'activité logicielle.
USB	<code>pyusb</code>	Inventorier les périphériques visibles.

5.7 Agent principal

Le fichier `agent.py` orchestre l'agent. Il charge la configuration serveur depuis `serverdata.json`, construit l'objet `Computer`, prépare le dictionnaire `currentAgentInfo`, lit `agentdata.json`, vérifie si l'agent possède déjà un identifiant, puis initialise la machine si nécessaire.

L'agent sépare ensuite les responsabilités dans plusieurs processus : heartbeat, mises à jour, WebSocket de commandes et alertes. Cette idée est pertinente car elle évite qu'une boucle de mise à jour lente bloque l'envoi du heartbeat. Le code actuel contient toutefois une erreur à corriger : la variable lancée est `websocket.start()` alors que le processus défini s'appelle `commands_websocket`. Cette limite doit être mentionnée car elle influence les tests réels.

5.8 Mises a jour periodiques

Le module `updater.py` contient les fonctions de mise a jour. La memoire est envoyee si elle change. Les interfaces reseau sont comparees entre l'etat stocke et l'etat courant afin de detecter les ajouts, suppressions et modifications. Les partitions sont comparees de maniere similaire. Les processus sont renvoyes sous forme de liste complete, puis remplaces cote serveur.

Cette approche mixte est pragmatique. Pour les processus, la comparaison fine serait plus couteuse et moins utile dans une premiere version. Pour les interfaces reseau, la comparaison par nom d'interface permet de ne transmettre que les changements. A terme, il faudrait ajouter un historique, car la version actuelle conserve principalement l'etat courant.

5.9 Heartbeat et disponibilite

Le heartbeat est implemente dans `heart.py` cote agent et `heartbeats.py` cote serveur. L'agent envoie periodiquement `computer_id` et `fingerprint`. Le serveur verifie l'authenticite minimale, met a jour `last_heartbeat` et force `is_alive` a vrai si la machine etait marquee hors ligne.

Le serveur execute en parallele `expireHeartBeat`. Le seuil actuel est fixe a 150 secondes, avec une verification toutes les 10 secondes. Si `last_heartbeat` est trop ancien, la machine est marquee hors ligne. Ce mecanisme est simple, mais il devrait etre rendu configurable selon le contexte : un reseau instable ou un agent configure avec un intervalle long peut produire des indisponibilites temporaires.

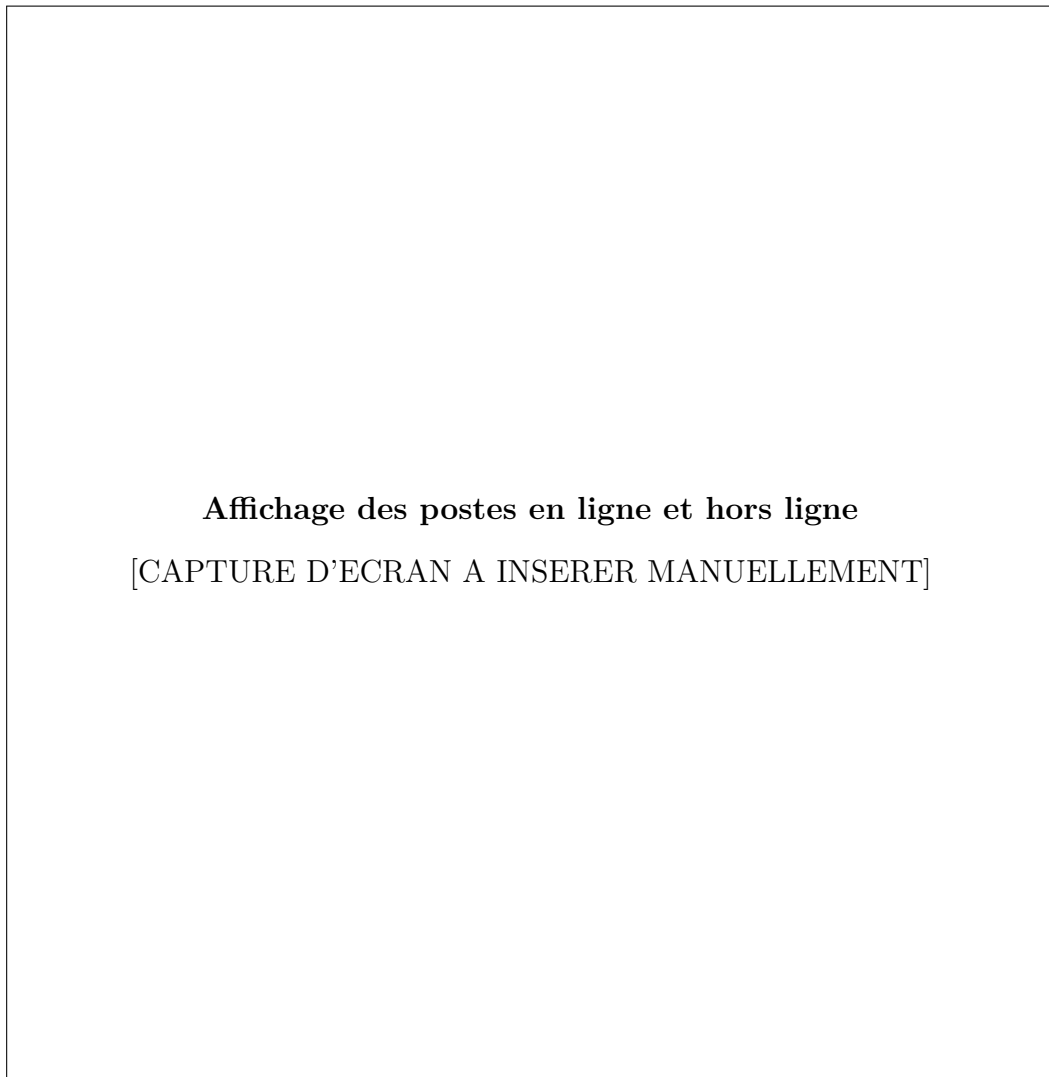


FIGURE 5.2 – Capture d'écran a inserer : Affichage des postes en ligne et hors ligne

5.10 API de consultation

Le routeur `get_resources.py` expose les informations collectees. Les endpoints principaux permettent de lister tous les postes, de lister les postes en ligne, de lire une machine, de recuperer la memoire, le reseau, les processus, les disques et les utilisateurs. Ces endpoints sont consommes par l'interface Streamlit.

Une limite importante apparait dans ce fichier : plusieurs requetes utilisent `text(f"...")` avec interpolation directe. Meme si certains parametres sont declares comme entiers par FastAPI, cette pratique reste fragile. Le code devrait utiliser `select()` avec conditions SQLAlchemy ou des parametres lies.

5.11 API de mise a jour

Le routeur `update_resources.py` recoit les changements envoyes par l'agent. Chaque endpoint reconstruit une structure `auth_data` et appelle `authenticateComputer`. Cette fonction verifie l'existence d'un couple `computer_id/fingerprint`. Si l'authentification echoue, une erreur HTTP 400 est renvoyee.

Les mises a jour couvrent le nom, la memoire, le reseau, les processus, les disques, les utilisateurs et le temps de demarrage. Le endpoint des processus supprime les anciens processus avant d'ajouter la nouvelle liste. Cette decision evite les doublons et garde une image recente, mais elle ne permet pas d'analyser l'historique d'apparition ou disparition d'un processus.

5.12 WebSockets et commandes distantes

Le routeur `websocket.py` maintient les connexions actives dans un `ConnectionManager`. Lorsqu'une requete `/api/commands` arrive, le serveur retrouve le WebSocket associe au `computer_id`, envoie la commande, attend la reponse et la renvoie a l'interface.

Cote agent, `websockets_init.py` recoit la commande. Sur Unix, elle est executee avec `subprocess.run(msg.split())`. Sur Windows, certaines commandes speciales sont traduites en PowerShell pour l'arret ou le redemarrage. Le resultat standard, l'erreur ou un message de statut est renvoye au serveur.

Cette fonctionnalite est puissante mais dangereuse. Dans un memoire de cybersécurité, elle doit etre analysee comme une capacite d'administration distante qui exige des protections fortes. Sans authentification administrateur, sans chiffrement et sans restrictions de commande, elle ne doit pas etre exposee hors environnement de test.

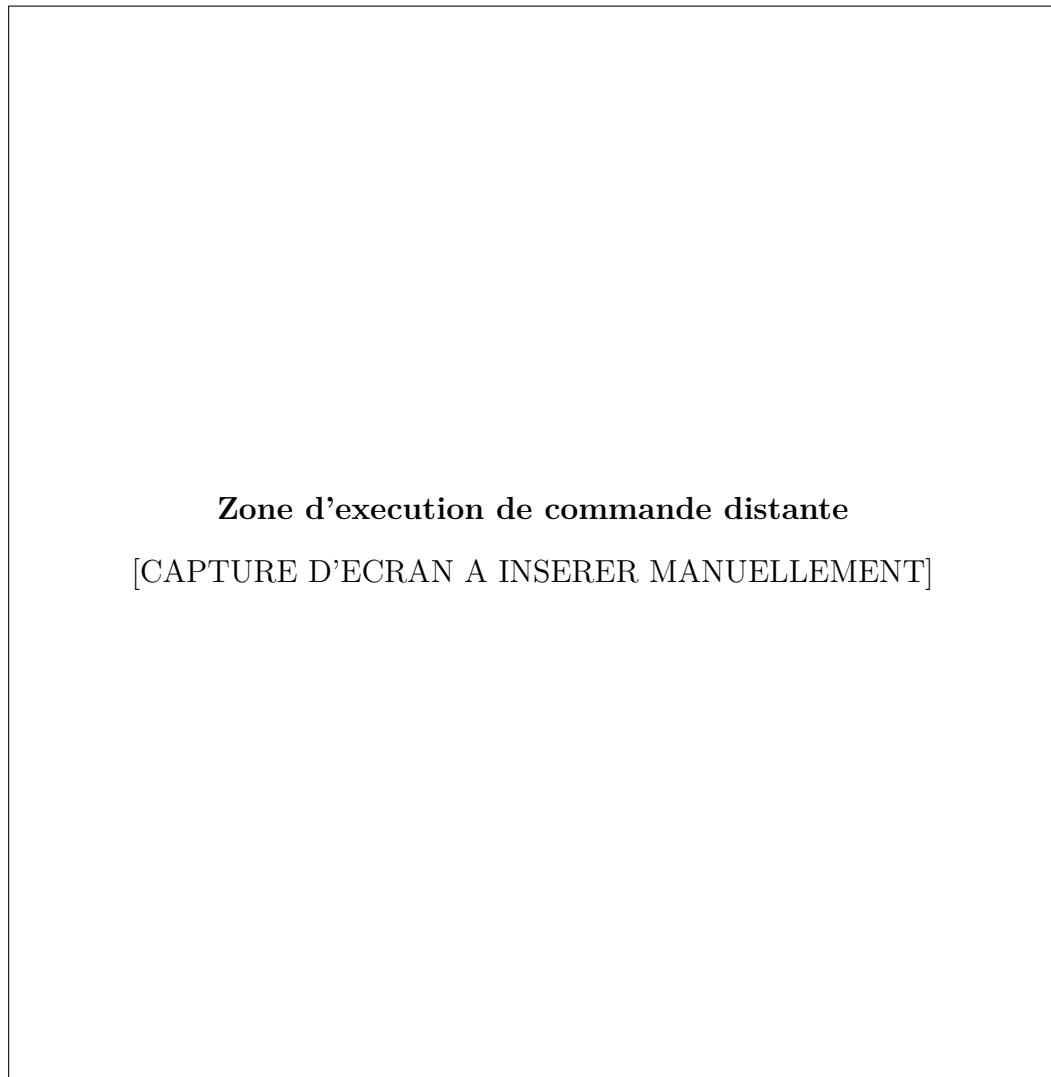


FIGURE 5.3 – Capture d'ecran a inserer : Zone d'execution de commande distante

5.13 Interface Streamlit

L'interface est organisee dans `server/frontend`. Le fichier `app.py` definit la navigation : Overview, Available Devices, Monitor Devices, Blacklist Devices, Locations, Add Locations, Computer Report et Network Alerts. Certaines pages sont encore vides, ce qui montre que le projet est en cours d'evolution.

La page `overviewpage.py` presente le projet, un guide d'utilisation et des metriques generales : nombre de machines, nombre d'emplacements et dernier poste ajoute. La page `listdev.py` liste les machines detectees sous forme de tableau avec nom, emplacement, OS et statut. La page `monitor.py` constitue la page la plus riche : selection d'une machine en ligne, affichage memoire, temps de demarrage, OS, utilisateurs, disques, processus, reseau et commandes distantes.

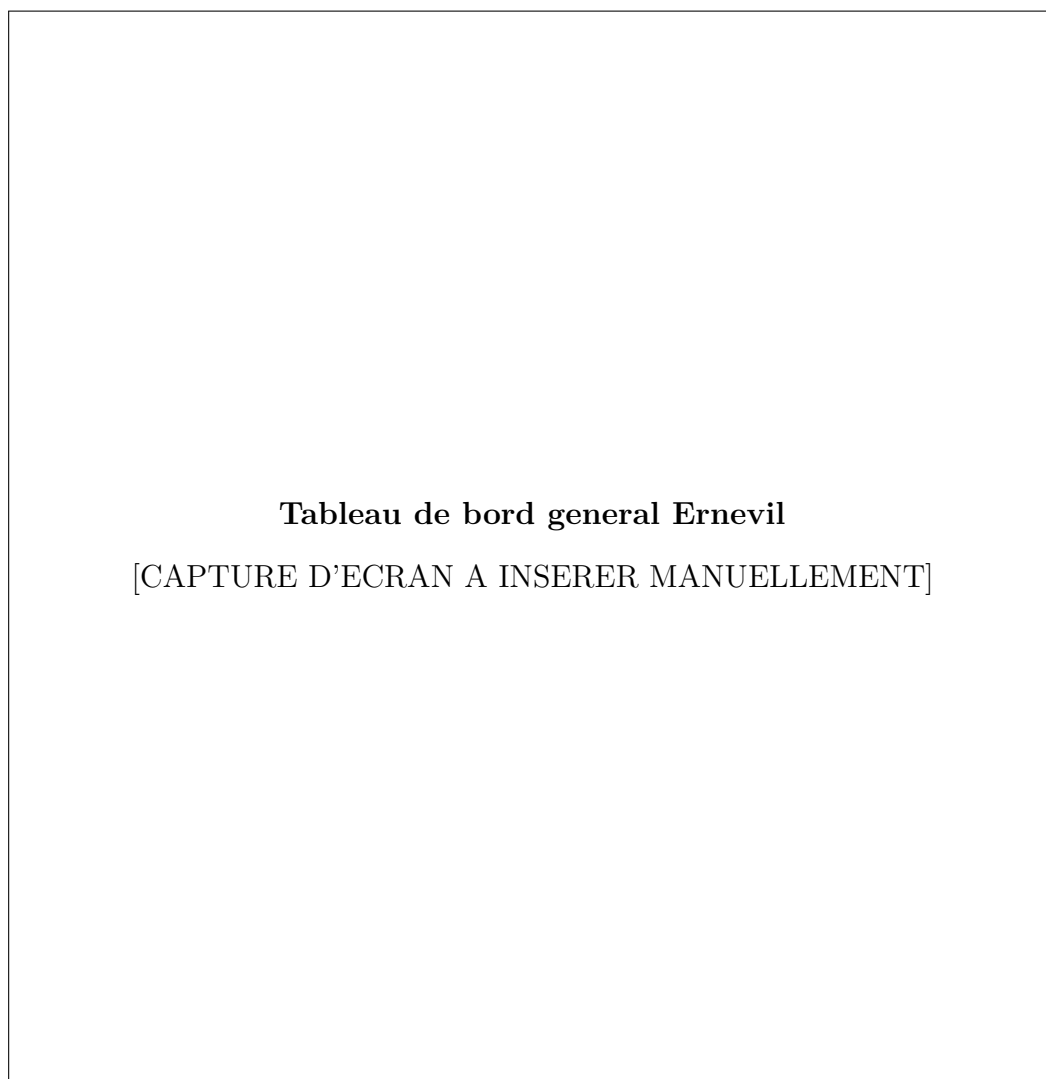


FIGURE 5.4 – Capture d'écran a inserer : Tableau de bord general Ernevil



FIGURE 5.5 – Capture d'écran a inserer : Liste des postes disponibles

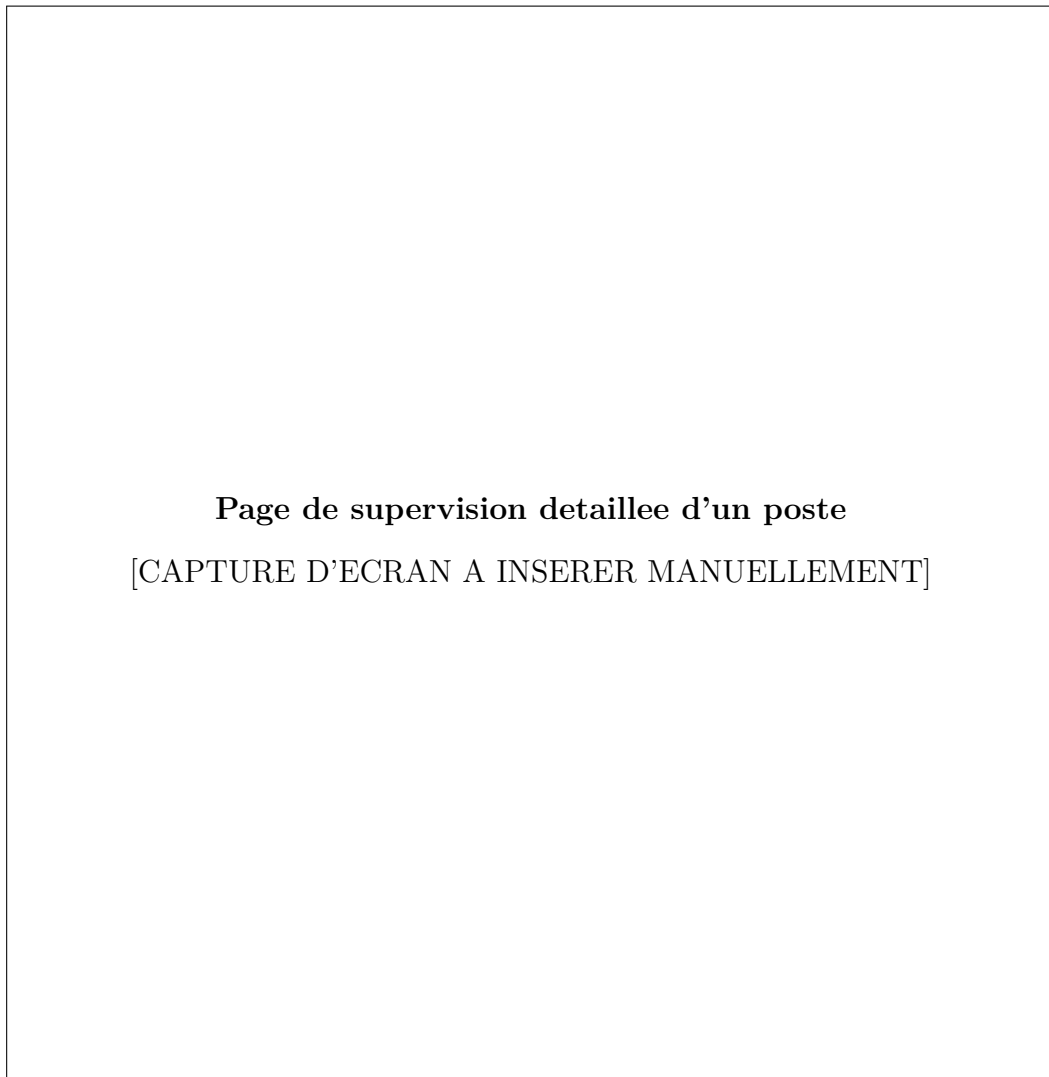


FIGURE 5.6 – Capture d'écran a inserer : Page de supervision detaillee d'un poste

5.14 Gestion des postes supervises

La gestion des postes repose sur la table `computerInfo`. Chaque poste a un identifiant interne, un nom, un indicateur Unix, un systeme d'exploitation, un temps de demarrage, un identifiant machine, une empreinte, une date d'ajout, un dernier heartbeat et un etat `is_alive`. Cette structure est suffisante pour une premiere version d'inventaire actif.

La relation avec `locations` prepare la gestion par salle. Cependant, la page de localisation et le routeur correspondant retournent encore des donnees de test. Dans le memoire, cette partie doit donc etre presentee comme une fonctionnalite partiellement preparee, non comme une gestion complete des emplacements.

5.15 Gestion des processus

La collecte des processus utilise `psutil.process_iter(['pid', 'name', 'username'])`. Le choix des champs est volontairement limite : PID, nom et utilisateur. Cela reduit le volume transmis par rapport a une collecte complete. C'est suffisant pour une premiere lecture administrative. Pour une detection avancee, il faudrait ajouter le chemin de l'executable, les arguments, le hash du fichier, la date de creation du processus et l'arbre parent-enfant.

5.16 Gestion de la memoire

La memoire est collectee avec `psutil.virtual_memory()`. L'agent conserve le total, la memoire disponible et le pourcentage d'utilisation. La page de supervision affiche le pourcentage sous forme de metrique. Cette information permet d'identifier rapidement un poste surcharge, mais elle ne suffit pas a expliquer la cause. L'interet apparait lorsqu'elle est lue avec la liste des processus.

5.17 Gestion reseau

La collecte reseau s'appuie sur `ifaddr`. Le code ignore les adresses non IPv4 et conserve une adresse par nom d'interface. Cette approche est simple et adaptee au besoin de base : connaitre l'adresse d'une machine et ses interfaces actives. Elle peut toutefois perdre de l'information si une interface possede plusieurs adresses IPv4 ou si IPv6 devient necessaire.

5.18 Gestion des disques

Les partitions sont collectees avec `psutil.disk_partitions(all=False)`. Le systeme conserve le nom de partition, le point de montage et le type de systeme de fichiers. Cette collecte donne une vue d'inventaire, mais ne mesure pas encore l'espace utilise. Une evolution naturelle consisterait a ajouter `psutil.disk_usage()` pour surveiller la saturation.

5.19 Detection USB

Deux niveaux existent. Le premier est l'inventaire USB au moment de l'initialisation, avec `pyusb`. Le second est la detection d'evenements avec `usb-monitor`. Lorsqu'un peripherique est connecte ou deconnecte, le module construit un dictionnaire contenant fabricant, produit, vendor id et product id.

Cette base est interessante pour la cybersécurité. Dans un laboratoire, les clés USB peuvent servir à copier des fichiers, installer des outils ou introduire des exécutables. La détection d'un branchement ne prouve pas une attaque, mais elle donne un signal utile. La valeur de ce signal dépendra d'une future politique : périphériques autorisés, périphériques inconnus, sévérité, historique et corrélation avec l'utilisateur connecté.

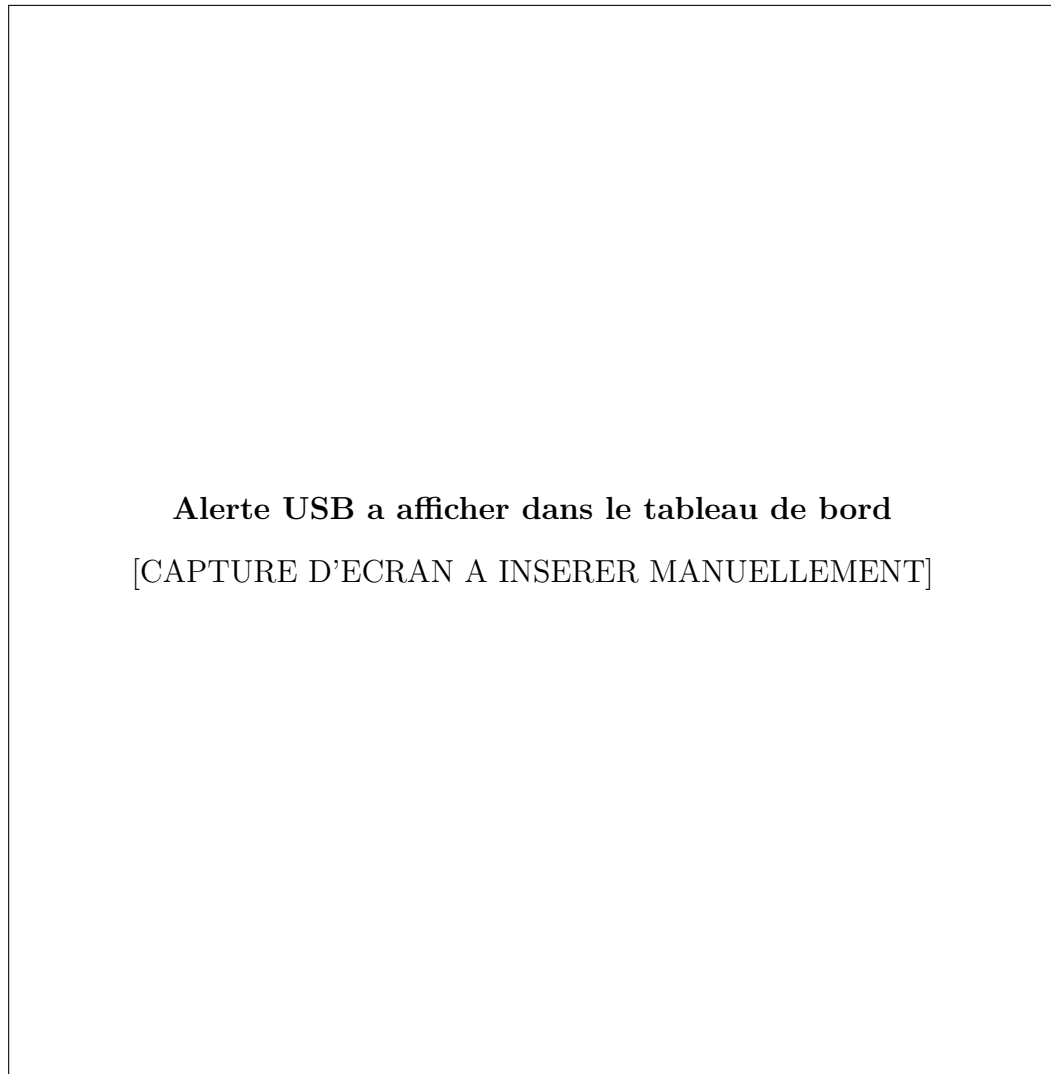


FIGURE 5.7 – Capture d'écran à insérer : Alerte USB à afficher dans le tableau de bord

5.20 Rapports

La page `reports.py` existe mais est vide. Le besoin de reporting reste donc une perspective proche. Un rapport utile devrait regrouper l'identité du poste, l'état de disponibilité, les interfaces réseau, les partitions, les utilisateurs, les derniers événements USB et les observations de sécurité. Il devrait pouvoir être généré en PDF ou exporté en CSV selon le besoin administratif.

5.21 Blacklist

La page `blacklist.py` existe mais ne contient pas encore de logique. Dans une version future, la blacklist pourrait servir à marquer des machines interdites, des périphériques USB refusés, des processus suspects ou des commandes bloquées. La priorité serait de définir clairement l'objet de la blacklist afin d'éviter une fonctionnalité ambiguë.

5.22 Locations

Le module de localisation est amorcé. Le schéma contient une table `Locations`, et les pages `listlocations.py` et `addlocations.py` existent. Le routeur retourne actuellement des données de test. Une implémentation complète devrait permettre de créer des salles, associer des postes à une salle, filtrer l'interface par emplacement et produire des statistiques par laboratoire.

5.23 Integration PostgreSQL

Le code actuel utilise SQLite. Pour une version de production ou de démonstration multi-utilisateur, PostgreSQL serait plus adapté : meilleure gestion de la concurrence, robustesse, rôles, sauvegardes, extensions et supervision. L'utilisation de SQLAlchemy rend cette migration réaliste, mais elle ne sera pas automatique. Il faudra modifier la chaîne de connexion, créer une stratégie de migration, tester les types de dates, vérifier les contraintes d'unicité et adapter le déploiement.

5.24 Limites d'implémentation

Plusieurs limites ressortent du code. Certaines pages sont vides. Des imports inutilisés existent. Le module `agent` contient une incohérence de nom de processus pour le WebSocket. Les requêtes SQL doivent être sécurisées. La gestion des utilisateurs administrateurs est inachevée. La collecte CPU est préparée côté agent mais non stockée côté base. Les alertes USB sont envoyées mais pas encore historisées. Ces limites ne diminuent pas l'intérêt du projet ; elles montrent plutôt les étapes restantes pour passer d'un prototype fonctionnel à une plateforme plus robuste.

5.25 Analyse critique des choix

Le principal point fort d'Ernevil est sa cohérence pédagogique. Toutes les briques sont visibles et modifiables. Le projet montre comment un agent s'enregistre, comment

une API recoit les donnees, comment une base relationnelle les organise et comment une interface les consomme. Son principal point faible est la securite des actions sensibles. Cette faiblesse est frequente dans les prototypes de supervision, car la priorite initiale est souvent la fonctionnalite. Dans une approche cybersecurite, elle doit devenir prioritaire avant tout deploiement reel.

Chapitre 6

Approfondissements techniques et dossiers d'implementation

6.1 Demarche iterative de developpement

Le developpement d'Ernevil n'a pas ete pense comme une seule livraison monolithique. Le projet a plutot progresse par couches. La premiere couche consistait a prouver qu'un agent pouvait collecter des informations locales fiables. La deuxieme consistait a envoyer ces donnees vers une API. La troisieme a ajoute la persistance. La quatrieme a introduit l'interface d'administration. La cinquieme a traite la presence en temps quasi reel avec le heartbeat. La sixieme a experimente les WebSockets pour les commandes et les alertes.

Cette progression est importante car elle explique plusieurs choix visibles dans le code. Certaines fonctionnalites sont completes, d'autres sont amorcees, et quelques fichiers servent de points d'extension. Par exemple, les pages `blacklist.py`, `reports.py` et `netalerts.py` existent mais ne contiennent pas encore de logique. Ce n'est pas incoherent : elles materialisent des axes prevus, mais le socle de collecte et de supervision a ete prioritaire.

TABLE 6.1 – Iterations de developpement du projet

Iteration	Objectif	Travail realise	Decision issue de l'iteration
I1	Collecte locale	Classe <code>Computer</code> , usage de <code>psutil</code> , <code>ifaddr</code> , <code>pyusb</code> .	Centraliser les fonctions systeme dans un collecteur.
I2	Enregistrement	Route <code>POST /api/computers</code> .	Utiliser une empreinte pour eviter les doublons.

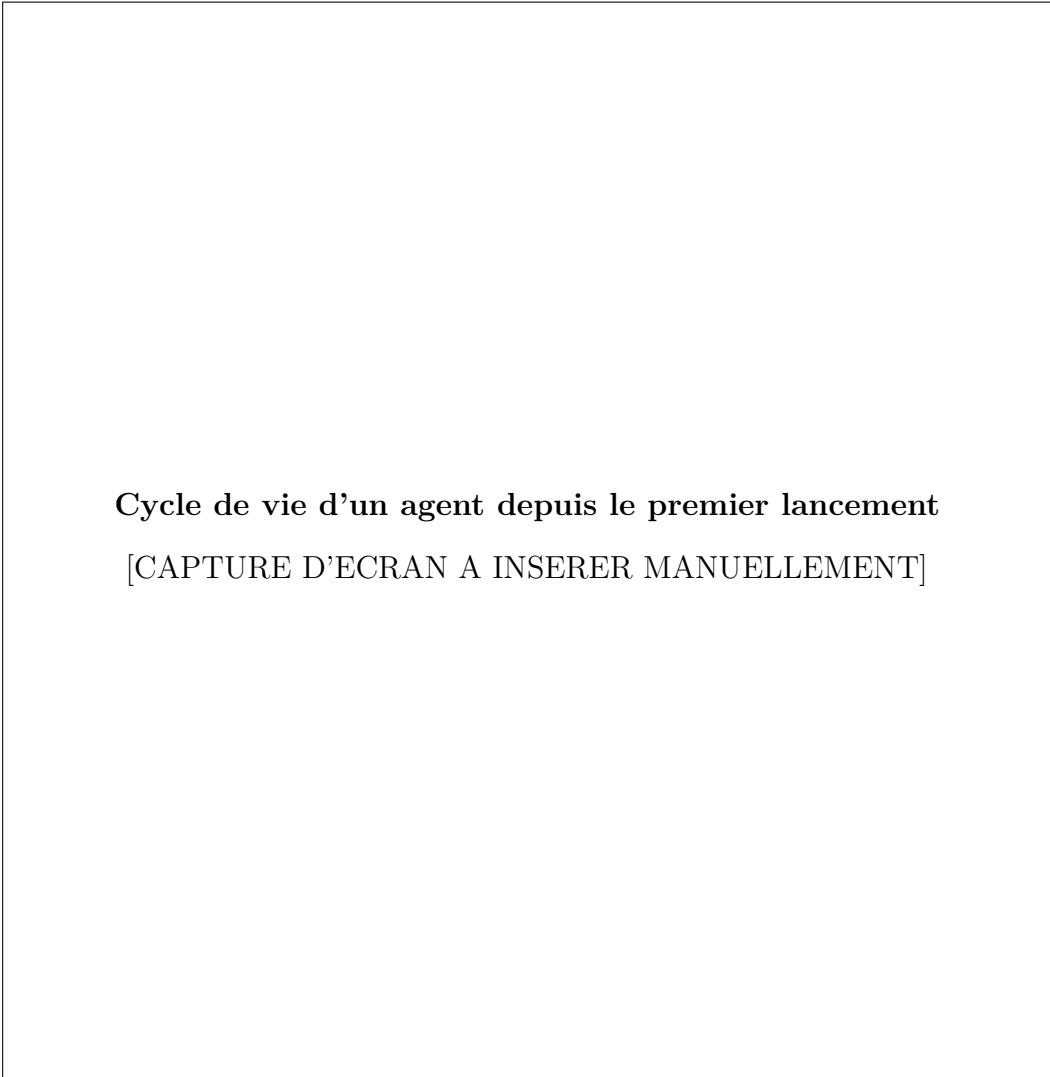
I3	Base de donnees	Modeles SQLAlchemy	Distinguer poste, memoire, reseau, processus, disques, utilisateurs.
I4	Interface	Pages Streamlit principales.	Donner une vue administrateur rapidement exploitable.
I5	Disponibilite	Heartbeat agent et tache serveur.	Separer presence et donnees detaillees.
I6	Commandes	WebSocket et execution distante.	Traiter cette fonction comme sensible et a securiser.
I7	Alertes USB	Ecoute USB et canal d'alerte.	Construire une base evenementielle avant l'IA.
I8	Analyse critique	Relecture du code et identification des limites.	Documenter les limites au lieu de les masquer.

6.2 Dossier technique : cycle de vie d'un agent

Le cycle de vie d'un agent commence avant meme la premiere requete HTTP. L'agent lit `serverdata.json` afin de connaitre l'adresse du serveur. Il construit ensuite l'objet `Computer`, ce qui initialise le nom de machine et le type de systeme. Les collectes sont executees pour produire une photographie initiale du poste. Cette photographie contient des donnees stables, comme l'identifiant machine, et des donnees variables, comme la memoire ou les processus.

Lorsque `agentdata.json` contient deja un identifiant, l'agent verifie que ce poste existe encore cote serveur. Si la verification reussit, l'agent evite un nouvel enregistrement. Si l'identifiant est absent ou invalide, il envoie les donnees initiales au serveur et conserve le `computer_id` retourne. Cette logique reduit les doublons et rend le redemarrage de l'agent plus propre.

La suite du cycle de vie est continue. Le heartbeat maintient la presence. Les mises a jour actualisent les donnees techniques. Le WebSocket garde la possibilite de recevoir une commande. Le canal d'alerte prepare l'envoi d'evenements. Ces boucles doivent etre independantes, car une panne temporaire sur une collecte ne doit pas forcement empecher le heartbeat d'indiquer que la machine est encore presente.



Cycle de vie d'un agent depuis le premier lancement
[CAPTURE D'ECRAN A INSERER MANUELLEMENT]

FIGURE 6.1 – Capture d'écran a inserer : Cycle de vie d'un agent depuis le premier lancement

6.3 Dossier technique : interpretation de l'etat d'un poste

Dans Ernevil, un poste peut etre inconnu, connu hors ligne, connu en ligne ou connu mais partiellement renseigne. Cette distinction est plus precise qu'un simple "connecte/deconnecte". Un poste inconnu n'existe pas en base. Un poste connu hors ligne a deja ete enregistre mais ne donne plus de heartbeat recent. Un poste connu en ligne actualise son heartbeat. Un poste partiellement renseigne peut etre en ligne mais avoir une erreur sur une collecte specifique, par exemple USB ou processus.

Cette analyse permet d'eviter une interpretation incorrecte. Une machine hors ligne n'est pas necessairement en panne : l'agent peut etre arrete, le reseau peut etre coupe ou le serveur peut etre inaccessible. Une machine en ligne n'est pas necessairement

saine : elle peut avoir une memoire saturee, un processus suspect ou un peripherique non autorise. Le tableau de bord doit donc afficher l'etat de presence comme un indicateur parmi d'autres.

TABLE 6.2 – Etats possibles d'un poste dans Ernevil

Etat	Condition technique	Interpretation administrative
Inconnu	Aucune ligne dans <code>computerInfo</code> .	Le poste n'a jamais ete enrole ou la base a ete reinitialisee.
Enregistre	Ligne presente avec empreinte unique.	Le poste fait partie du parc connu.
En ligne	Heartbeat recent et <code>is_alive=True</code> .	L'agent communique avec le serveur.
Hors ligne	Heartbeat depasse le seuil.	L'agent ne communique plus ou la machine est eteinte.
Partiel	Certaines tables associees sont vides.	Une collecte a echoue ou une fonctionnalite n'est pas active.
Suspect	Evenement ou metrique incohérente.	Une verification humaine est necessaire.

6.4 Dossier technique : qualite des donnees collectees

La qualite des donnees depend de trois facteurs : la bibliotheque utilisee, les permissions du processus agent et la stabilite du systeme supervise. `psutil` donne des informations riches, mais certaines donnees peuvent etre limitees par les droits de l'utilisateur. Sur Windows, les utilisateurs visibles via `psutil.users()` ne representent pas exactement la meme realite que la liste retournee par `pwd` sur Unix. De meme, l'inventaire USB peut varier selon les pilotes disponibles et les droits d'accès au bus USB.

Pour cette raison, le memoire ne presente pas les donnees comme absolument completes. Elles sont suffisantes pour une supervision initiale, mais elles doivent etre qualifiees. Une bonne interface devrait afficher "donnee indisponible" lorsqu'une collecte echoue, au lieu de laisser croire que l'absence de donnees signifie absence de probleme. Cette nuance est importante dans une plateforme de cybersecurite.

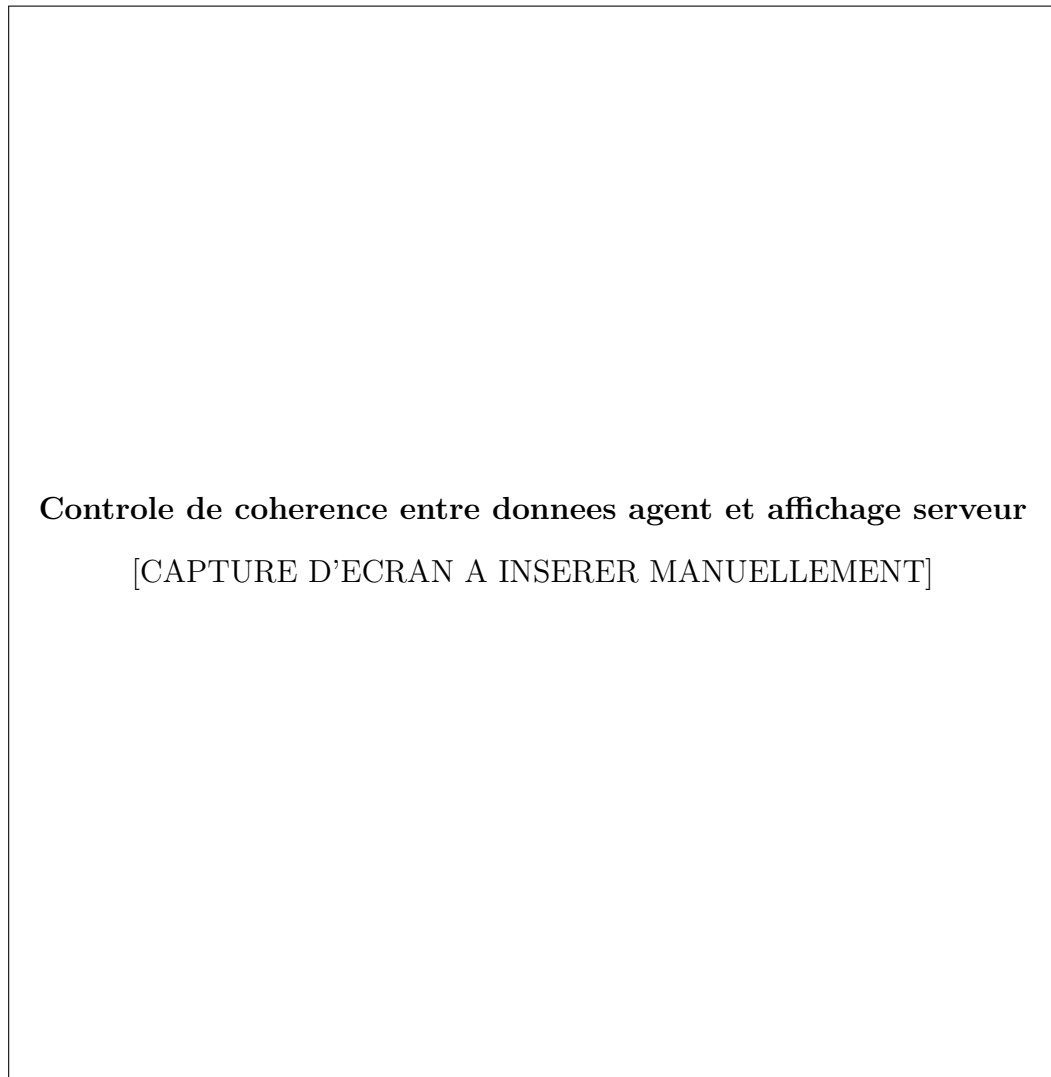


FIGURE 6.2 – Capture d'écran a inserer : Controle de coherence entre donnees agent et affichage serveur

6.5 Dossier technique : conception des schemas Pydantic

Les schemas Pydantic presents dans le dossier `server/schemas` servent de contrat entre l'agent, l'API et l'interface. Ils definissent les champs attendus lors de la creation d'un poste, des mises a jour et de l'authentification agent. Ce role de contrat est essentiel : sans schema, le serveur accepterait des structures trop libres, ce qui rendrait les erreurs plus difficiles a localiser.

La validation automatique de FastAPI apporte un avantage immediat. Si l'agent oublie un champ obligatoire ou envoie un type incompatible, l'erreur est detectee a l'entree de l'API. Le code metier peut donc se concentrer sur la creation ou la mise a jour des objets SQLAlchemy. Une amelioration future consisterait a renforcer les

schemas avec des contraintes plus precises : longueur maximale, formats d'adresse IP, listes non vides et validation des dates.

6.6 Dossier technique : separation REST et WebSocket

REST et WebSocket ne repondent pas au meme besoin. REST est adapte aux operations ponctuelles, idempotentes ou facilement journalisables : ajouter un poste, lire la memoire, mettre a jour une interface. WebSocket est adapte aux interactions ou le serveur doit envoyer une instruction vers un agent deja connecte.

Dans Ernevil, cette separation a une consequence de securite. Les routes REST peuvent etre protegees par des jetons classiques, des roles et des permissions. Les WebSockets doivent aussi etre authentifies au moment de la connexion, puis associes a une identite agent et une session administrateur. Le code actuel stocke les connexions dans un dictionnaire indexe par `computer_id`. C'est simple et lisible, mais il faut ajouter la gestion des deconnexions, des collisions d'identifiants et des connexions multiples.

Connexion WebSocket d'un agent dans un environnement de test
[CAPTURE D'ECRAN A INSERER MANUELLEMENT]

FIGURE 6.3 – Capture d'écran a inserer : Connexion WebSocket d'un agent dans un environnement de test

6.7 Dossier technique : execution distante et garde-fous

L'exécution distante est la fonctionnalité la plus critique. Elle donne à l'administrateur la capacité de lancer une commande sur une machine supervisée. Dans un laboratoire, cette fonction peut aider à vérifier un service, redémarrer un poste, consulter une configuration ou exécuter un diagnostic. Mais la même fonction peut devenir un vecteur de compromission si elle est mal protégée.

La version actuelle exécute la commande transmise. Sur Unix, le message est séparé avec `split()`, ce qui limite certains cas complexes mais ne constitue pas une protection. Sur Windows, le message est passé à PowerShell. Les commandes d'arrêt et de redémarrage sont traduites en commandes natives. La prochaine version devrait

introduire au minimum cinq garde-fous : authentification administrateur, rôle autorisé, confirmation visuelle, journalisation, et liste blanche de commandes.

TABLE 6.3 – Garde-fous recommandés pour les commandes distantes

Garde-fou	Raison	Implementation cible
Authentification admin	Identifier la personne qui agit.	Session sécurisée ou JWT.
Autorisation par rôle	Limiter les commandes dangereuses.	Rôle administrateur, technicien, lecteur.
Liste blanche	Éviter les commandes arbitraires.	Catalogue de commandes paramétrées.
Journalisation	Permettre l'audit.	Table <code>command_logs</code> .
Confirmation	Éviter les actions accidentelles.	Dialogue pour arrêt/redémarrage.
Expiration	Éviter les sessions persistantes.	TTL pour les WebSockets et jetons.

Resultat d'une commande distante et trace attendue
[CAPTURE D'ECRAN A INSERER MANUELLEMENT]

FIGURE 6.4 – Capture d'écran a inserer : Resultat d'une commande distante et trace attendue

6.8 Dossier technique : supervision des processus

La supervision des processus est une source d'information riche. Un processus permet de comprendre ce qui s'exécute sur le poste. Dans Ernevil, chaque processus est représenté par son PID, son nom et son utilisateur. Cette représentation est volontairement simple. Elle permet de remplir un tableau, de filtrer par nom et d'observer rapidement une activité inhabituelle.

Pour aller plus loin, il faudrait enrichir la collecte. Le chemin de l'exécutable permettrait de distinguer un processus légitime d'un binaire placé dans un répertoire temporaire. Les arguments de lancement aideraient à comprendre le comportement. Le processus parent permettrait de reconstituer une chaîne d'exécution. Le hash de l'exécutable permettrait une comparaison avec une base de confiance. Ces enrichissements augmenteraient cependant le volume de données et les risques de confidentialité.

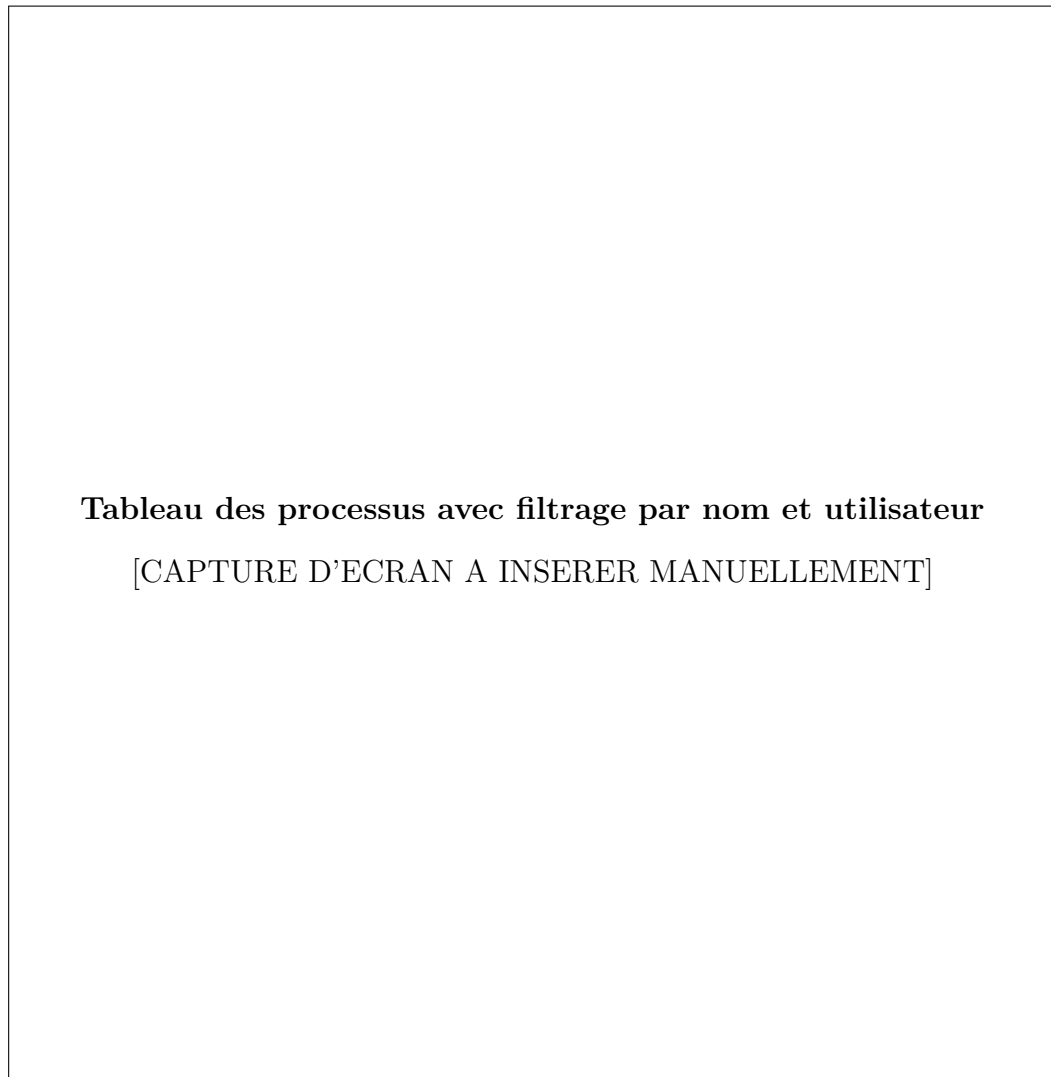
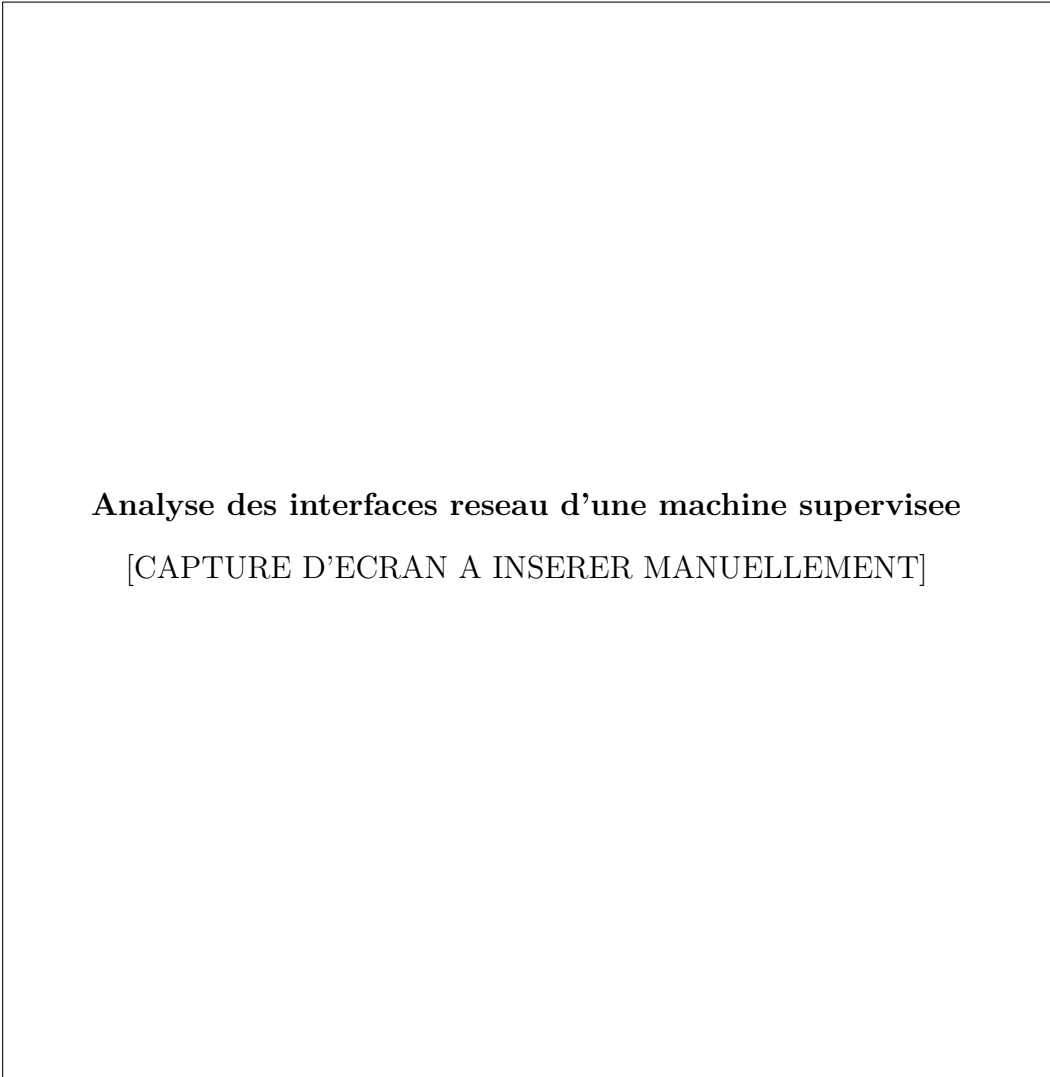


FIGURE 6.5 – Capture d'écran a inserer : Tableau des processus avec filtrage par nom et utilisateur

6.9 Dossier technique : supervision reseau

La supervision reseau actuelle affiche les interfaces et les adresses IPv4. Cette information aide a identifier rapidement un poste, verifier son segment reseau et detecter un changement. Dans un laboratoire, une adresse inattendue peut signaler une mauvaise configuration, une interface virtuelle activee ou une connexion a un autre reseau.

La prochaine etape serait de distinguer les interfaces physiques, virtuelles et de boucle locale. Le code filtre deja les interfaces contenant "virtual" dans certaines mises a jour, mais ce filtrage reste textuel et incomplet. Une approche plus robuste consisterait a utiliser les statistiques de `psutil.net_if_stats()`, l'adresse MAC et une classification explicite.



Analyse des interfaces reseau d'une machine supervisee
[CAPTURE D'ECRAN A INSERER MANUELLEMENT]

FIGURE 6.6 – Capture d'écran a inserer : Analyse des interfaces reseau d'une machine supervisee

6.10 Dossier technique : supervision memoire et CPU

La memoire est implementee dans la base et l'interface. Le CPU est collecte dans l'agent, mais il n'est pas encore persiste dans le modele SQLAlchemy. Cette situation montre une difference entre collecte disponible et fonctionnalite complete. Dans le tableau de bord, la metrique CPU est actuellement affichee avec une valeur fixe dans la page `monitor.py`. Le memoire doit donc signaler que l'integration CPU reste a finaliser.

Une implementation complete du CPU devrait ajouter une table ou une colonne dediee, une route de mise a jour, une route de lecture et un affichage dynamique. Il faudrait aussi choisir entre une valeur instantanee et une moyenne. Une valeur instantanee peut fluctuer fortement. Une moyenne sur plusieurs secondes est plus stable

mais moins reactive.

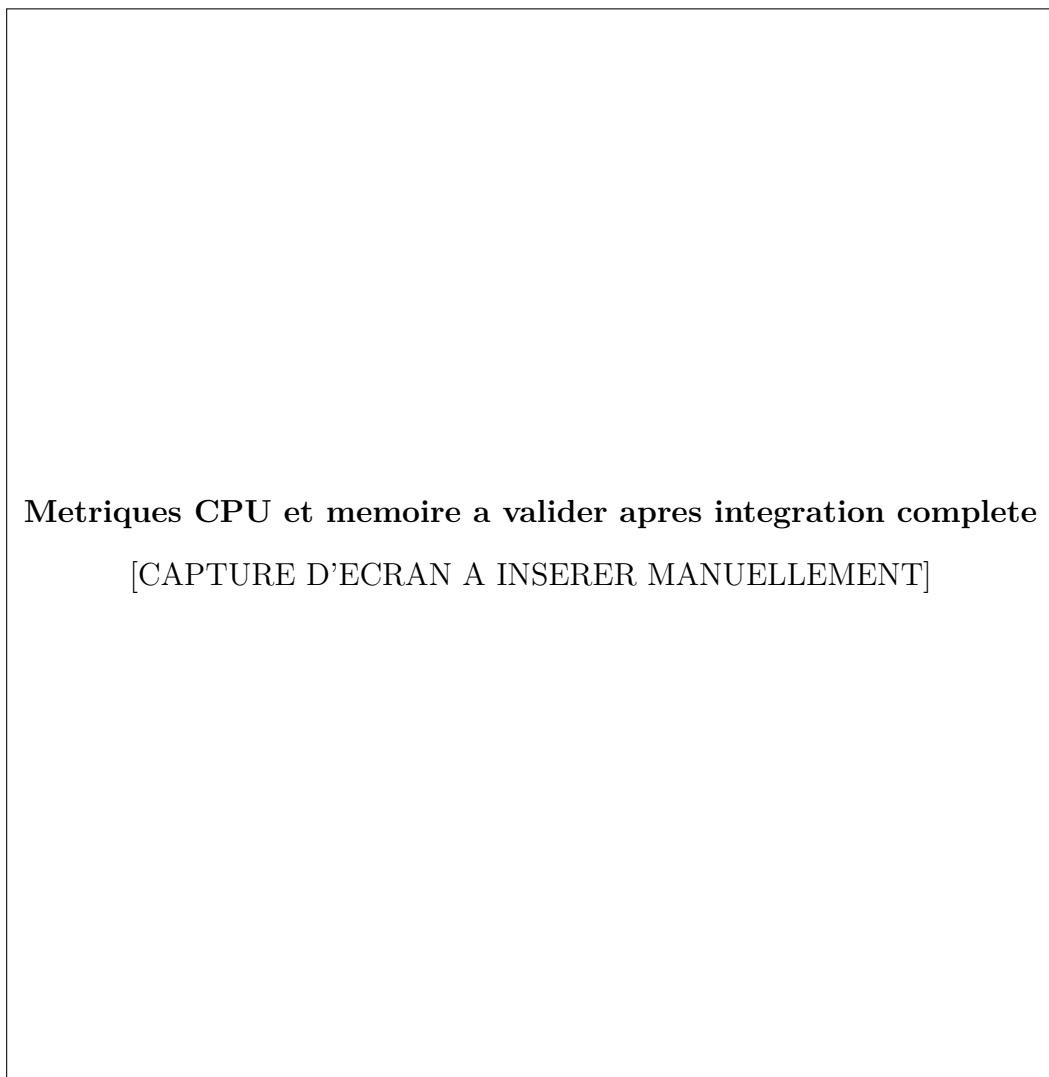


FIGURE 6.7 – Capture d'écran a inserer : Metriques CPU et memoire a valider apres integration complete

6.11 Dossier technique : USB et politique de confiance

La detection USB devient utile lorsqu'elle est associee a une politique de confiance. Sans politique, chaque connexion est seulement un evenement. Avec une politique, l'evenement peut etre classe : peripherique connu, peripherique nouveau, peripherique interdit, peripherique sensible ou peripherique a verifier. Les champs `vendor_id` et `product_id` constituent une base pour cette classification.

Une table future pourrait stocker les peripheriques autorises avec leur fabricant, produit, identifiants et salle autorisee. Lorsqu'un evenement arrive, le serveur compare l'identifiant avec cette liste. Si le peripherique est inconnu, il cree une alerte de severite moyenne. Si le peripherique est explicitement interdit, il cree une alerte critique. Cette

logique resterait deterministe et explicable.

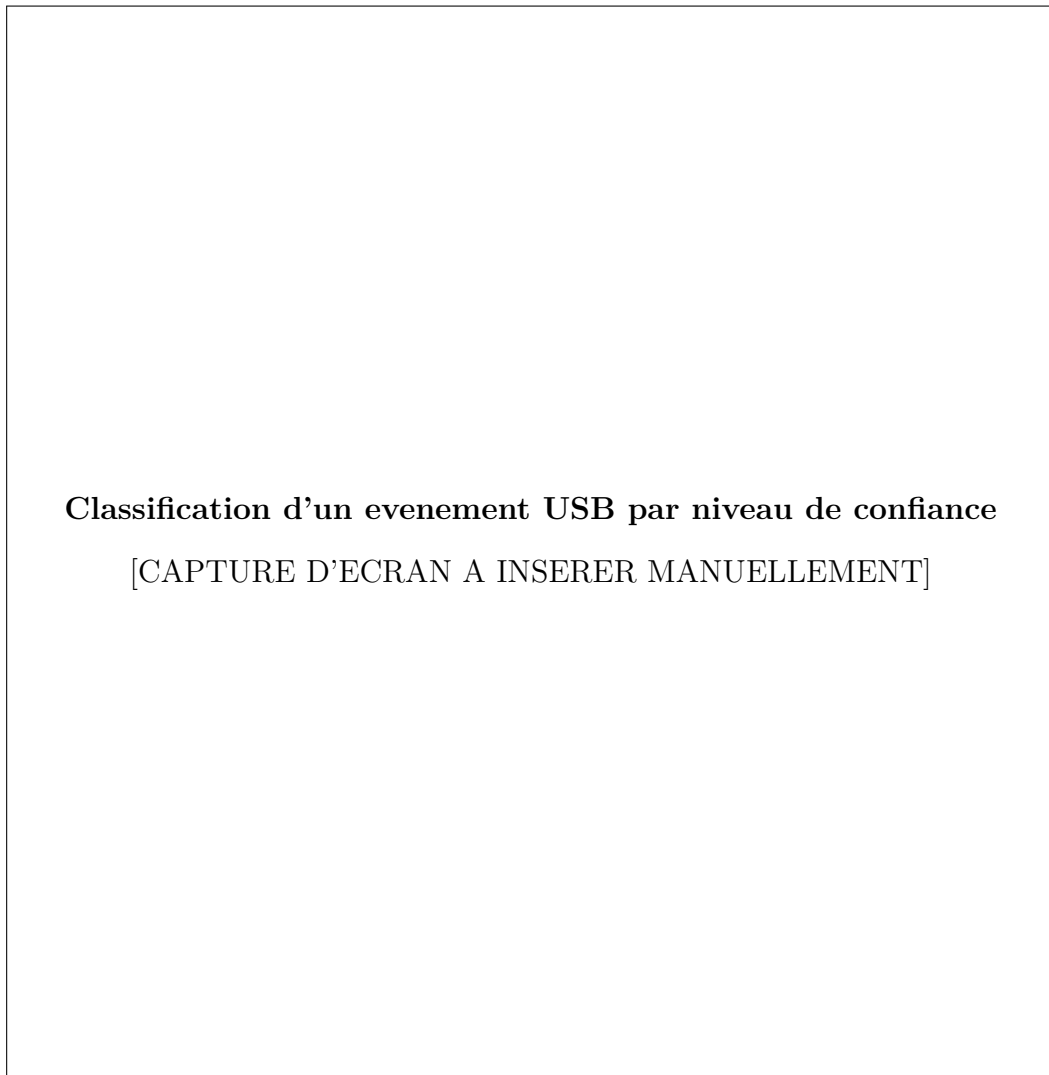


FIGURE 6.8 – Capture d'écran a inserer : Classification d'un evenement USB par niveau de confiance

6.12 Dossier technique : tableau de bord et priorisation visuelle

Un tableau de bord de supervision doit reduire le temps de comprehension. Il ne suffit pas d'afficher beaucoup de donnees. Il faut aider l'administrateur a voir ce qui demande une action. La page Overview pose les bases avec le nombre de machines, les emplacements et le dernier poste ajoute. Pour une version plus mature, elle devrait afficher les postes hors ligne, les alertes recentes, les salles les plus touchees et les machines dont les donnees sont incompletes.

La priorisation visuelle doit rester sobre. Les alertes critiques doivent etre visibles sans rendre l'interface agressive. Les tableaux doivent permettre le tri et le filtrage. Les

metriques doivent distinguer les valeurs reelles, les valeurs indisponibles et les valeurs encore non implementees. Cette distinction evite de tromper l'utilisateur.



FIGURE 6.9 – Capture d'écran a inserer : Tableau de bord cible avec priorisation des postes a traiter

6.13 Dossier technique : gestion des erreurs

La gestion des erreurs est un chantier transversal. Cote agent, certaines exceptions sont interceptees, par exemple lors de l'envoi du heartbeat. D'autres chemins peuvent encore provoquer un arret ou une incoherence. Cote serveur, les erreurs HTTP existent mais utilisent parfois le code 400 pour des situations differentes : authentication, machine indisponible ou ressource inexistante.

Une version plus propre devrait distinguer les codes : 401 pour une authentication manquante, 403 pour une action interdite, 404 pour une machine inexistante, 409 pour un conflit d'enregistrement et 422 pour une donnees invalide. Cette precision facilite le

debogage et rend l'API plus professionnelle.

TABLE 6.4 – Amelioration proposee des codes d'erreur

Situation	Etat actuel probable	Code recommande
Agent inconnu	HTTP 400.	401 ou 404 selon le cas.
Empreinte invalide	HTTP 400.	401 Unauthorized.
Machine inexistante	HTTP 400.	404 Not Found.
Machine deja enregistree	HTTP 400.	409 Conflict.
Schema invalide	Validation FastAPI.	422 Unprocessable Entity.
WebSocket absent	HTTP 400.	404 ou 409 selon contexte.

6.14 Dossier technique : journalisation et tracabilite

La journalisation est indispensable pour transformer Ernevil en outil d'administration fiable. Un administrateur doit pouvoir savoir qui a lance une commande, sur quel poste, a quelle heure, avec quel resultat. Il doit aussi pouvoir consulter les evenements importants : agent ajoute, agent hors ligne, heartbeat retabli, USB connecte, erreur de mise a jour, tentative de commande refusee.

Le depot actuel contient peu de journalisation persistante. Une evolution naturelle serait d'ajouter une table **events** avec un type, une severite, une source, un message, un horodatage et une reference optionnelle au poste. Cette table pourrait alimenter la page Network Alerts et les rapports.

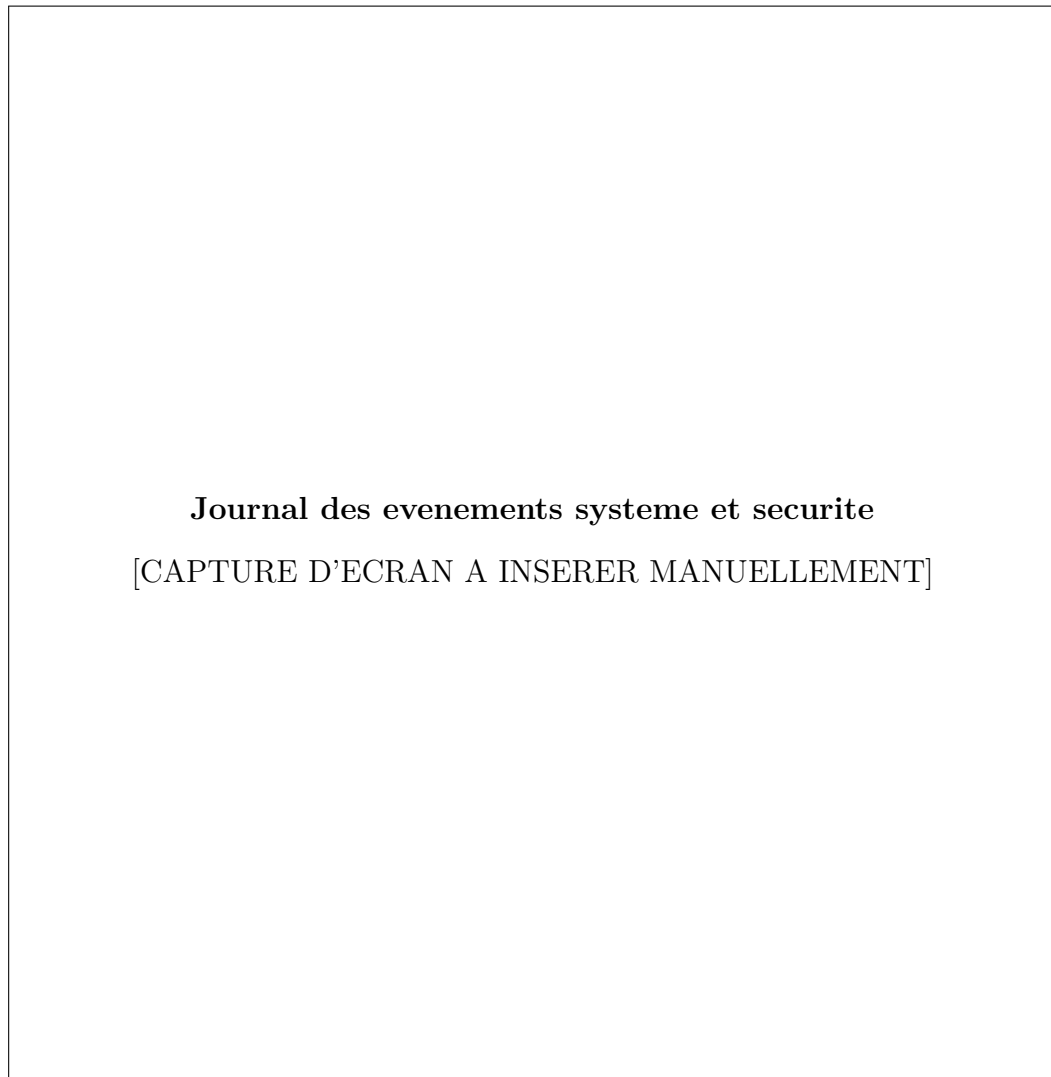


FIGURE 6.10 – Capture d'écran a inserer : Journal des evenements systeme et securite

6.15 Dossier technique : migration vers PostgreSQL

La migration vers PostgreSQL ne consiste pas seulement a changer une URL. SQLite est permissif et pratique pour un prototype local. PostgreSQL impose une gestion plus stricte des types, de la concurrence et des connexions. Il faudrait definir les variables d'environnement, creer l'utilisateur de base, initialiser le schema, tester les contraintes et prevoir les sauvegardes.

Dans Ernevil, la couche SQLAlchemy simplifie cette migration, car les modeles sont deja declares. Cependant, une migration professionnelle devrait utiliser Alembic. Sans outil de migration, chaque changement de modele risque de demander une reinitialisation de la base, ce qui n'est pas acceptable lorsque des historiques et des alertes sont conserves.

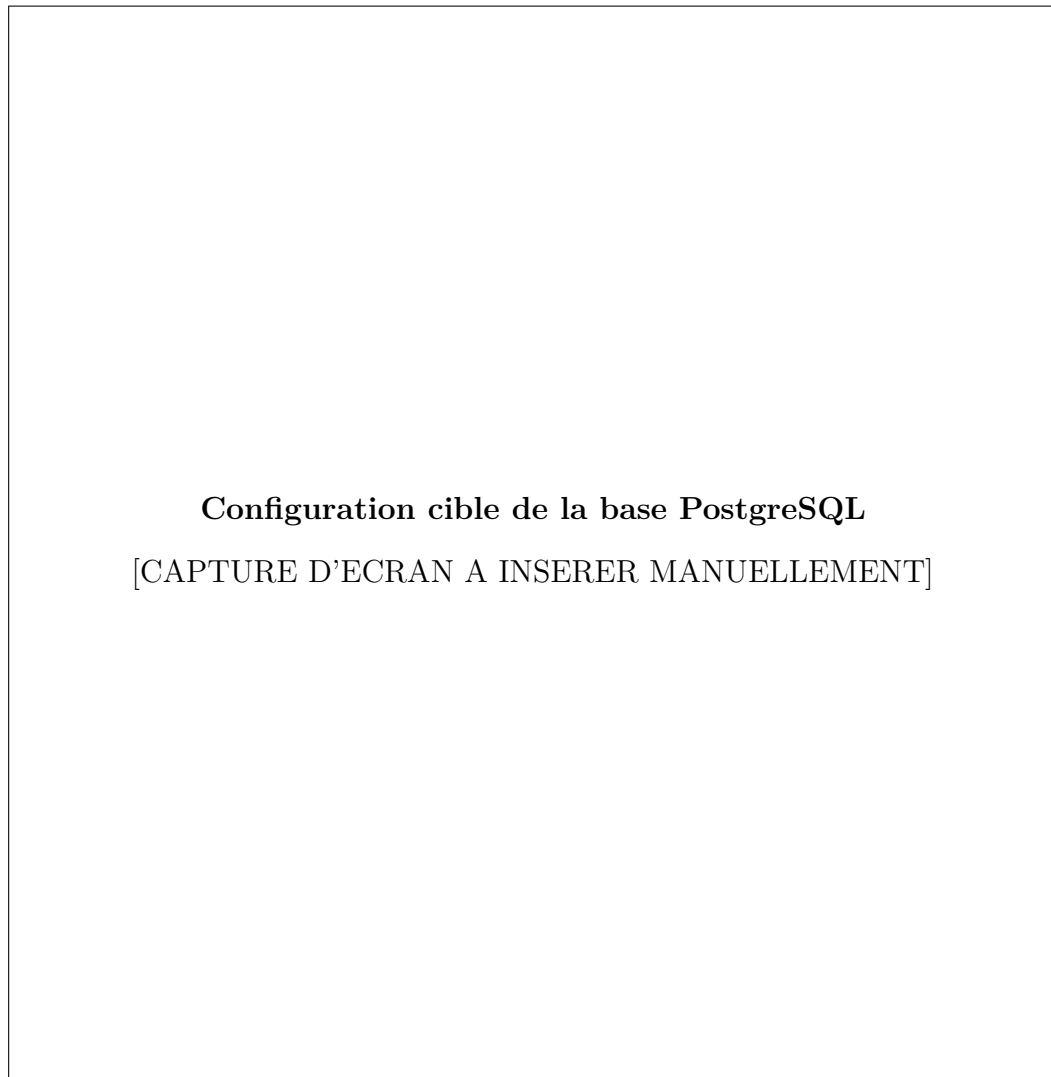
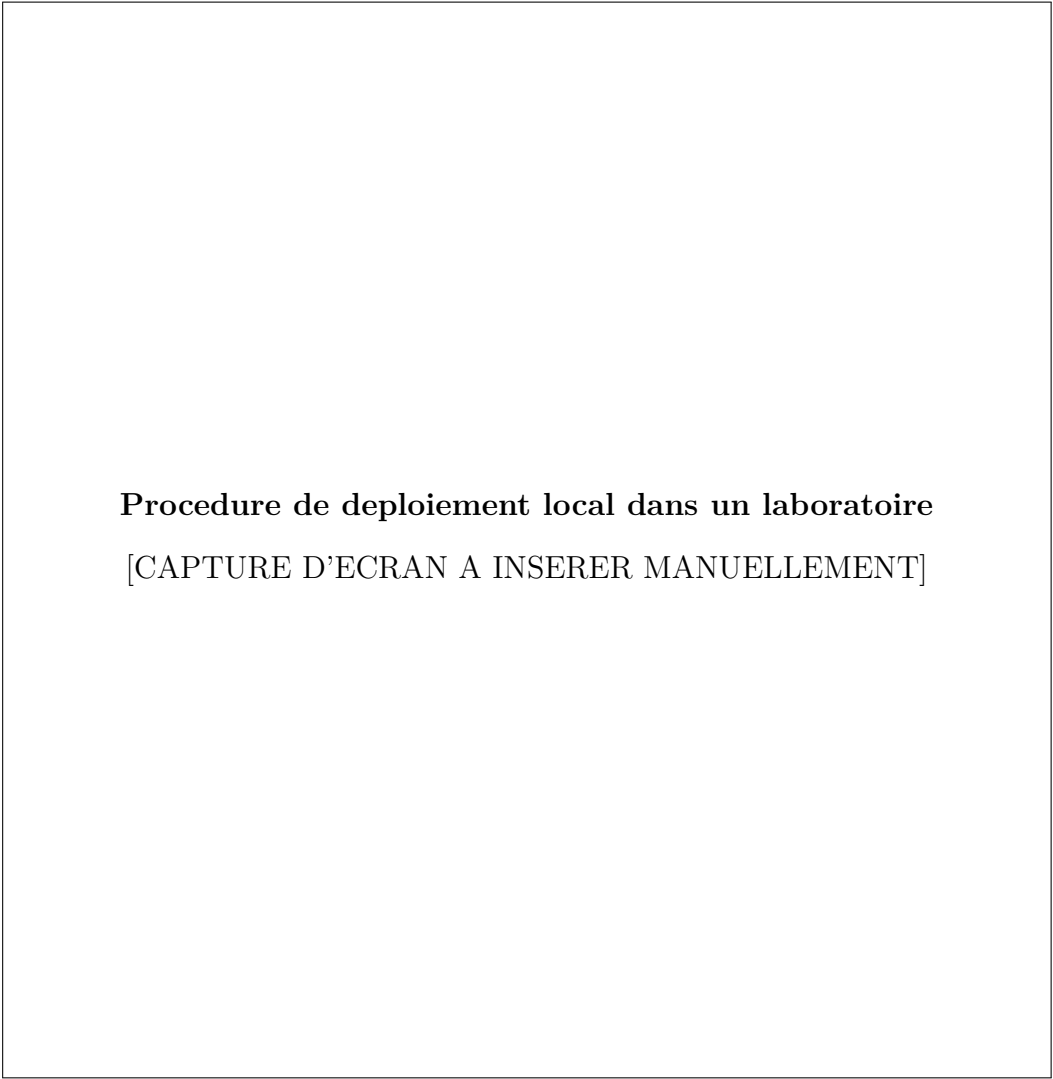


FIGURE 6.11 – Capture d'écran a inserer : Configuration cible de la base PostgreSQL

6.16 Dossier technique : deployment local

Un deployment local d'Ernevil comprend au moins trois processus : le serveur FastAPI, l'interface Streamlit et un ou plusieurs agents. Pour une demonstration, ces processus peuvent etre lances manuellement. Pour un usage regulier, il faut les transformer en services. Sur Linux, **systemd** peut gerer le demarrage automatique. Sur Windows, un service ou une tache planifiee peut lancer l'agent.

Le deployment doit aussi prendre en compte la configuration. L'agent lit **serverdata.json**. Cette approche est simple mais fragile si le fichier est modifie manuellement sur beaucoup de machines. Une evolution serait de fournir un script d'installation qui ecrit la configuration, enregistre l'agent et verifie la connectivite.



Procédure de déploiement local dans un laboratoire
[CAPTURE D'ECRAN A INSERER MANUELLEMENT]

FIGURE 6.12 – Capture d'écran à insérer : Procédure de déploiement local dans un laboratoire

6.17 Dossier technique : ergonomie d'administration

L'ergonomie d'une plateforme de supervision se mesure dans des situations de pression : avant un cours, pendant un incident, lors d'une vérification rapide. L'administrateur ne doit pas perdre du temps à chercher une information simple. Les pages actuelles séparent déjà Overview, Available Devices et Monitor Devices. Cette séparation est claire, mais elle peut être améliorée.

La page de liste devrait permettre de filtrer par statut, OS, salle et nom. La page Monitor devrait conserver le dernier poste sélectionné. Les actions dangereuses comme Shutdown et Restart devraient être séparées visuellement des actions de lecture. Les messages d'erreur devraient indiquer si le problème vient du serveur, de l'agent ou de la machine cible.

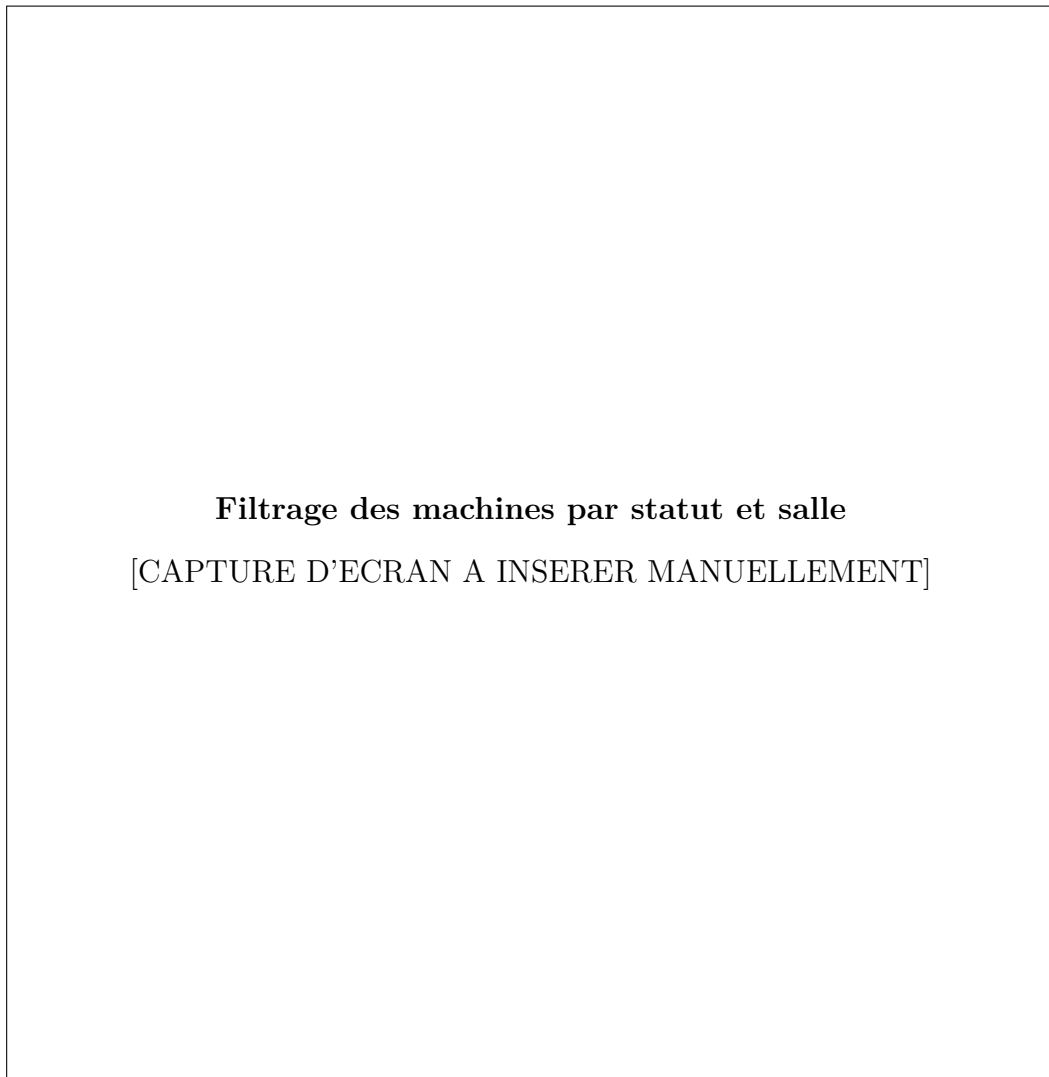


FIGURE 6.13 – Capture d'écran a inserer : Filtrage des machines par statut et salle

6.18 Dossier technique : coherence avec le contexte de stage

Le lien entre Ernevil et le stage repose sur la gestion des postes d'un environnement pedagogique. Les salles de l'ISTA NTIC sont utilisees par plusieurs groupes et necessitent un suivi regulier. Le projet repond a une situation observee : la supervision manuelle est couteuse, surtout lorsqu'il faut verifier plusieurs postes avant ou pendant une seance.

Ernevil apporte une reponse proportionnee. Il ne demande pas une infrastructure lourde. Il peut fonctionner dans un reseau local. Il centralise des informations utiles a la maintenance. Il donne une base de securite pour surveiller les peripheriques et les processus. Il reste ouvert a des ameliorations qui correspondent aux besoins d'un centre de formation : rapports, salles, alertes, historique et comptes administrateurs.

6.19 Matrice detaillee des fonctionnalites

TABLE 6.5 – Matrice de maturite des fonctionnalites Ernevil

Fonctionnalite	Etat	Fichier principal	Commentaire
Collecte me- moire	Implementee	computer_info.py	Affichee dans Monitor.
Collecte CPU	Partielle	agent.py	Valeur collectee mais non persistee completement.
Collecte reseau	Implementee	updater.py	IPv4 seulement, filtrage virtuel a renforcer.
Collecte proces- sus	Implementee	updater.py	Image courante sans historique.
Collecte disques	Implementee	updater.py	Pas encore d'espace utilise.
Collecte utiliza- teurs	Partielle	computer_info.py	Difference Unix/Windows a clarifier.
Inventaire USB	Implementee	computer_info.py	Depend des droits et pilotes USB.
Alerte USB	Partielle	alert_handler.py	Envoi prepare, stockage absent.
Heartbeat	Implementee	heartbeats.py	Seuil fixe de 150 secondes.
Liste des postes	Implementee	listdev.py	Emplacement affiche comme NULL.
Supervision de- taillee	Implementee	monitor.py	Page principale fonctionnelle.
Commandes distantes	Experimentale	websocket.py	Fonction sensible a securiser.
Arret/redemarrage	Experimentale	websocket.py	A proteger par confirmation et role.
Rapports	Non finalisee	reports.py	Page vide.
Blacklist	Non finalisee	blacklist.py	Page vide.
Locations	Partielle	locations.py	Donnees de test.
Comptes admin	Non finalisee	dbschema.py	Table presente, routes absentes.
PostgreSQL	Perspective	db.py	SQLite utilise actuellement.

IA partielle	Entraîne	<code>bestmodel.pkl</code>	HistGradientBoostingClassifier entraîne, alerte temps reel.
--------------	----------	----------------------------	---

Chapitre 7

Catalogue de captures commentees et validation visuelle

7.1 Role des captures dans le memoire

Les captures d’ecran ne doivent pas etre utilisees comme decoration. Dans ce memoire, elles servent a prouver visuellement le fonctionnement des modules, a montrer les resultats attendus et a guider la soutenance. Chaque emplacement ci-dessous correspond a une capture a produire manuellement apres lancement du serveur, de l’interface Streamlit et d’un ou plusieurs agents. Le commentaire associe indique ce que le jury doit observer et quelles limites doivent etre mentionnees.

7.2 Lancement du serveur FastAPI

La capture doit montrer le serveur FastAPI demarre localement, idealement avec les routes chargees et l’absence d’erreur au demarrage. Elle permet de confirmer que la couche backend est operationnelle avant le lancement de l’agent. Le commentaire doit signaler que la tache de heartbeat est creee au demarrage du serveur.

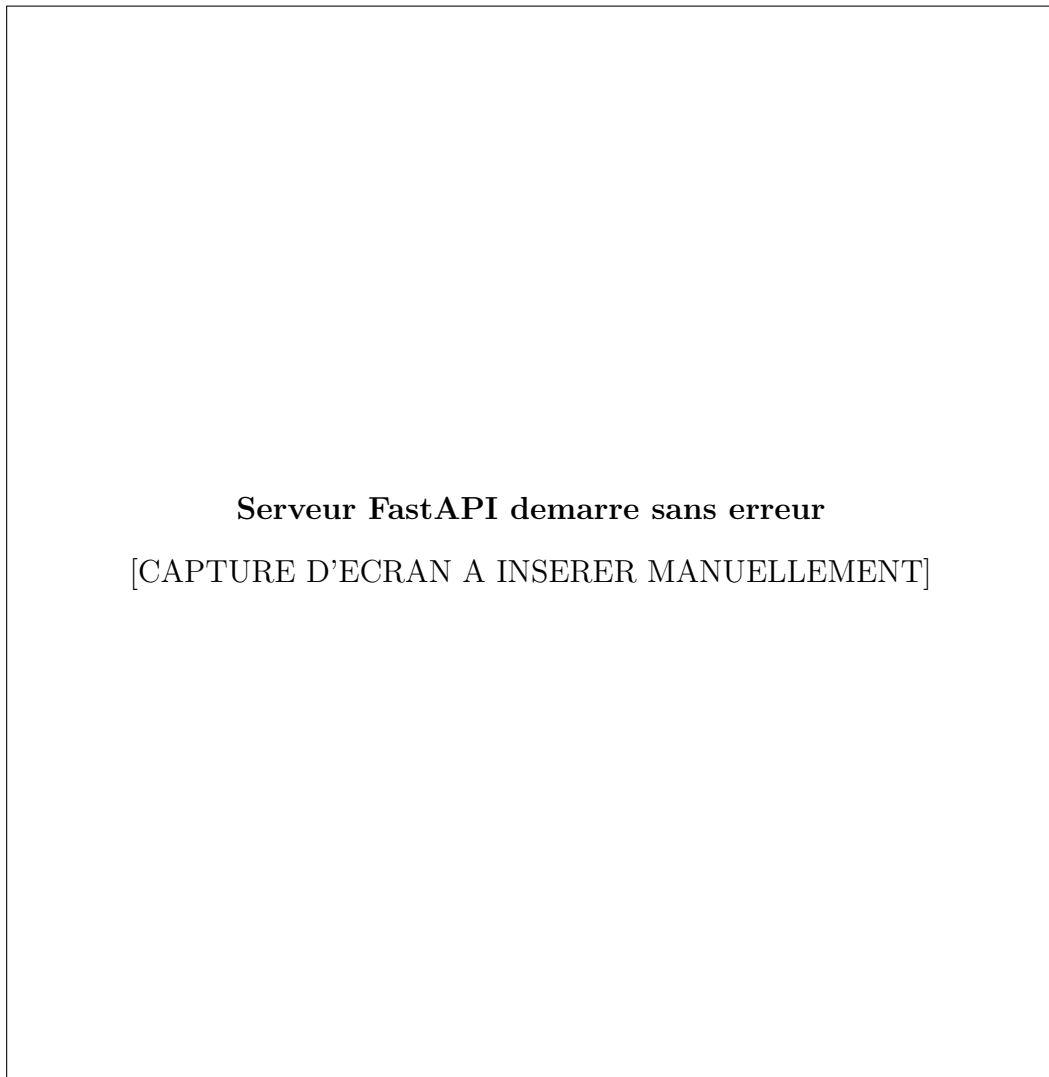


FIGURE 7.1 – Capture d'écran a inserer : Serveur FastAPI démarre sans erreur

7.3 Documentation interactive de l'API

La capture doit présenter la documentation automatique FastAPI. Elle montre les endpoints de création, lecture, mise à jour, heartbeat et WebSocket. Cette capture est utile pour expliquer que l'API n'est pas seulement appelée par l'interface, mais qu'elle constitue un contrat testable indépendamment.

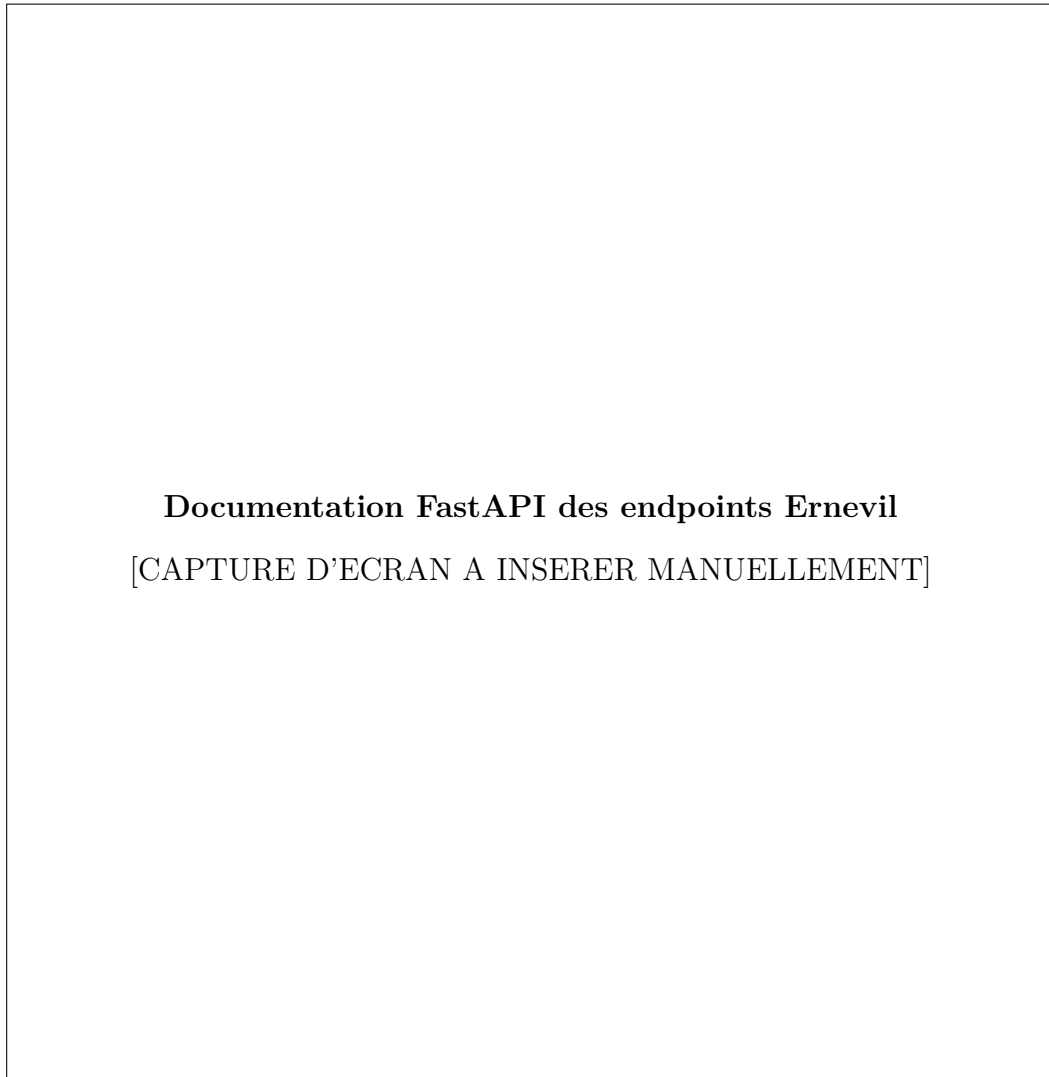


FIGURE 7.2 – Capture d'écran a inserer : Documentation FastAPI des endpoints Ernevil

7.4 Premier lancement de l'agent

La capture doit montrer le lancement d'un agent sur une machine de test. Elle doit etre accompagnee d'une explication sur la lecture de `serverdata.json`, la construction de l'objet `Computer` et l'envoi des donnees initiales. Si des erreurs USB apparaissent selon l'environnement, elles doivent etre expliquees au lieu d'etre masquées.

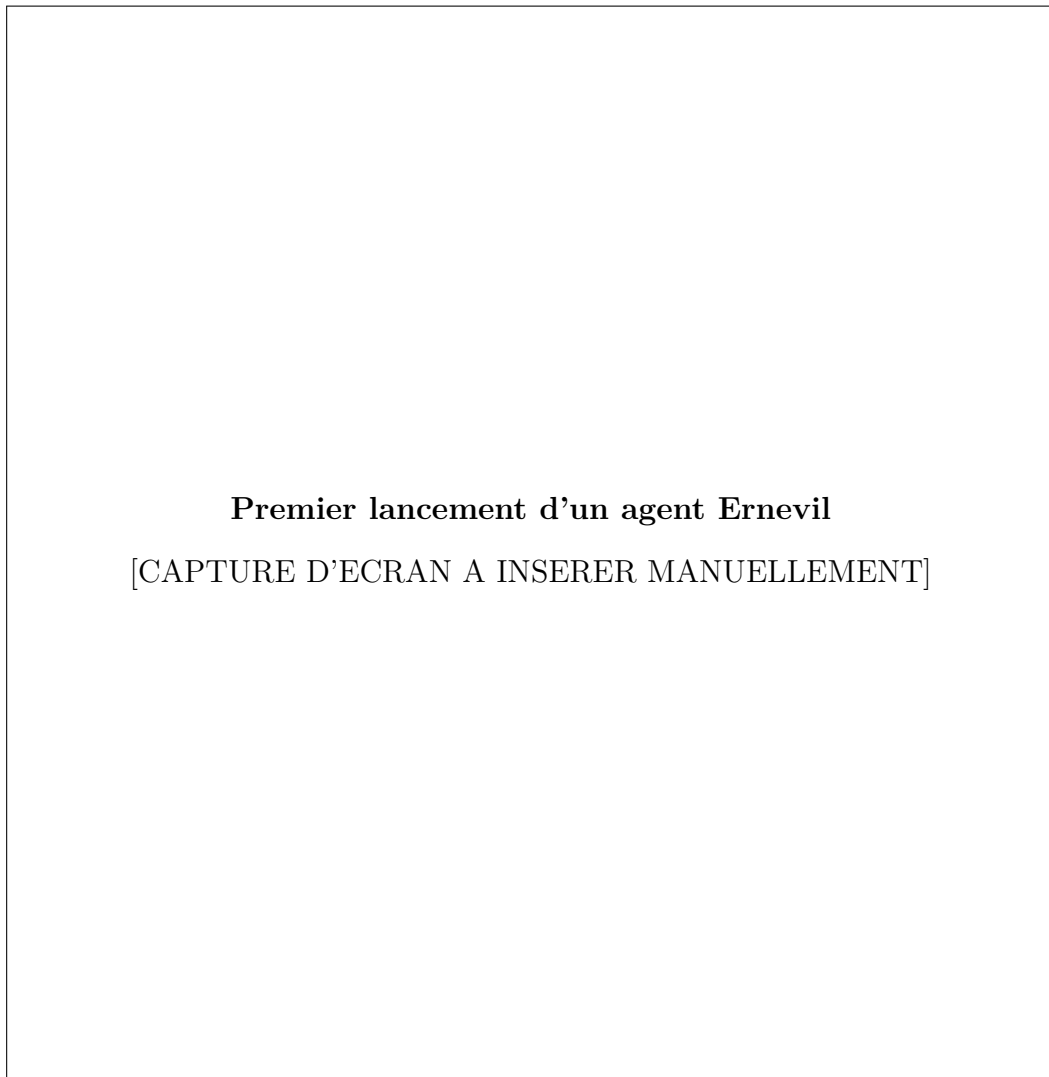


FIGURE 7.3 – Capture d'écran a inserer : Premier lancement d'un agent Ernevil

7.5 Creation de l'identifiant local

La capture doit montrer le fichier `agentdata.json` apres l'enregistrement. Le champ `agentid` doit contenir l'identifiant renvoye par le serveur. Cette preuve visuelle explique pourquoi un agent relance ne cree pas systematiquement une nouvelle machine.



FIGURE 7.4 – Capture d'écran a inserer : Fichier agentdata apres enregistrement

7.6 Insertion en base de donnees

La capture doit montrer la presence d'une ligne dans `computerInfo`. Elle doit etre accompagnee d'un commentaire sur les champs essentiels : `computer_id`, `computername`, `fingerprint`, `last_heartbeat` et `is_alive`. Cette capture lie directement le code SQLAlchemy au resultat stocke.

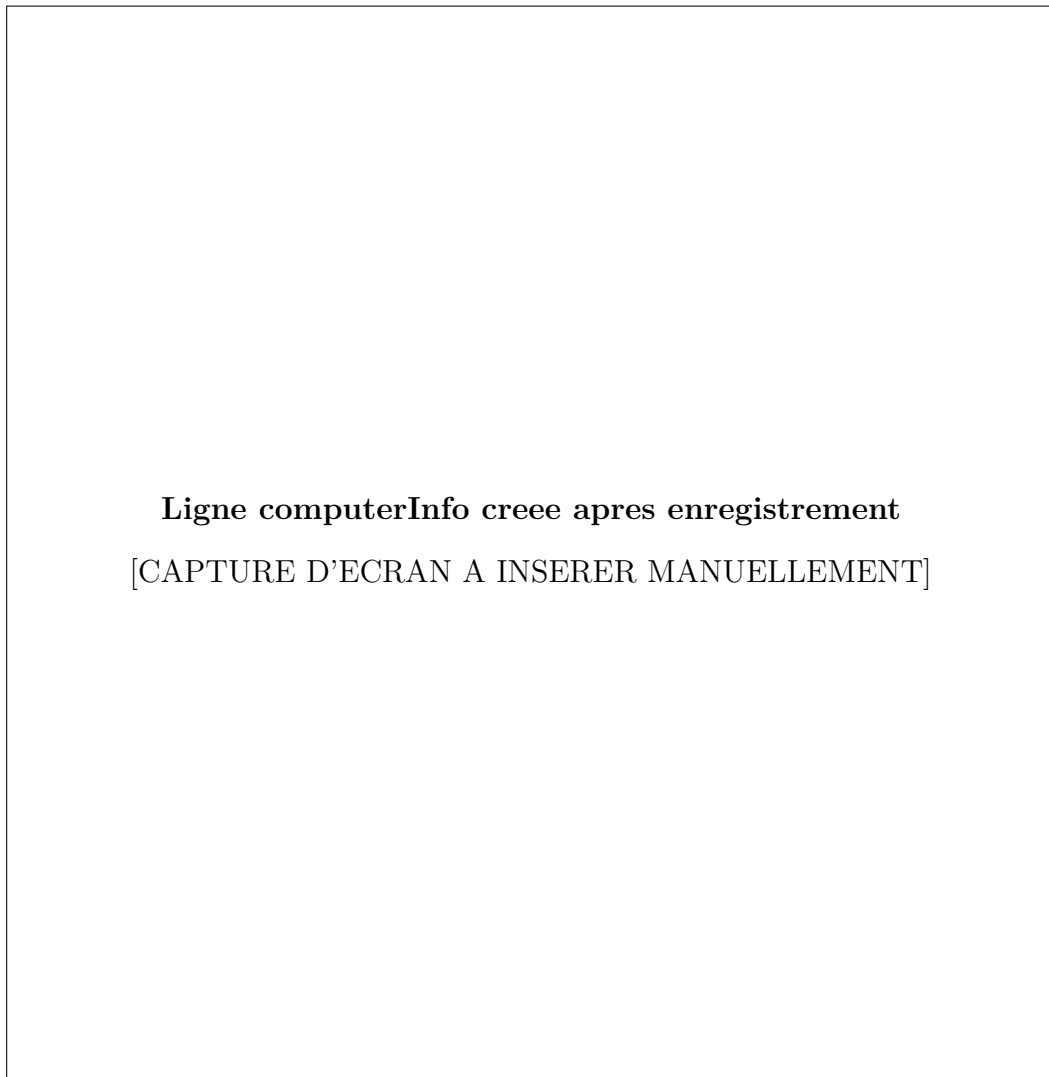


FIGURE 7.5 – Capture d'écran a inserer : Ligne computerInfo creee apres enregistrement

7.7 Donnees memoire

La capture doit montrer la table ou la reponse API de `memoryinfo`. Le commentaire doit expliquer que la memoire est une metrique instantanee et que son interpretation doit etre faite avec prudence. Une valeur elevee ne suffit pas a conclure a une anomalie ; elle doit etre rapprochee des processus.

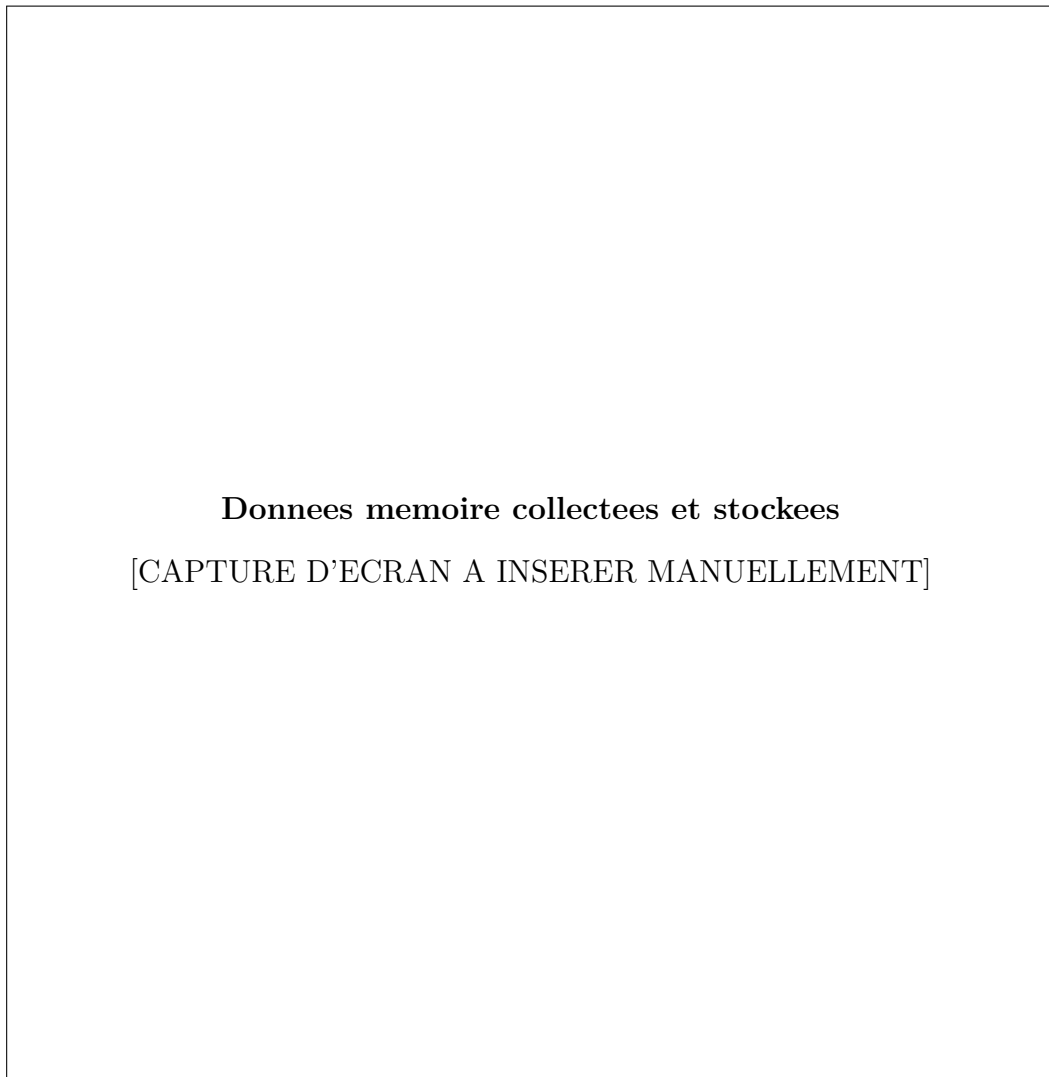


FIGURE 7.6 – Capture d'écran a inserer : Donnees memoire collectees et stockees

7.8 Interfaces reseau

La capture doit presenter les interfaces reseau recuperees par l'API. Le commentaire doit indiquer que seules les adresses IPv4 sont retenues dans la version actuelle. Il faut aussi expliquer la limite liee aux interfaces virtuelles et aux machines possedant plusieurs adresses.

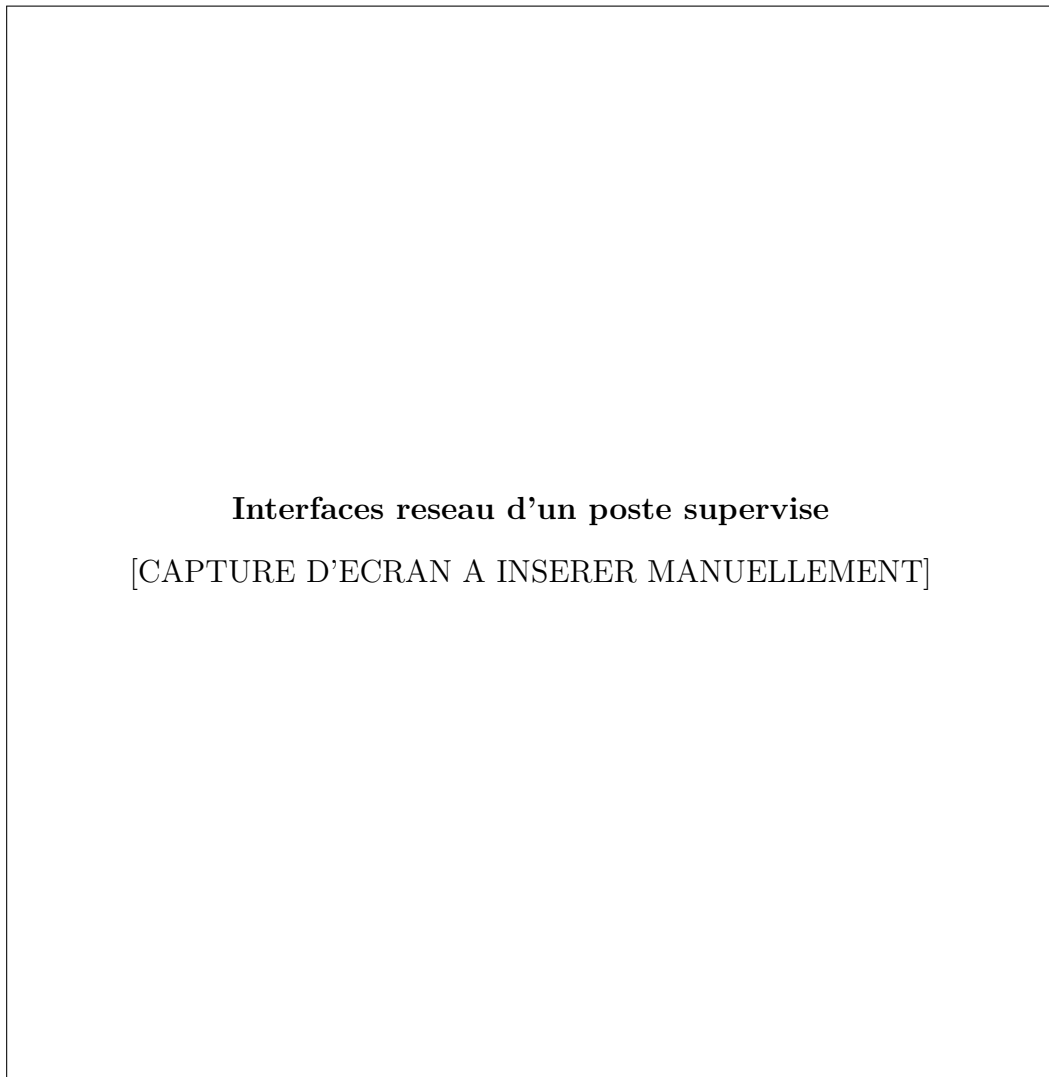


FIGURE 7.7 – Capture d'écran a inserer : Interfaces reseau d'un poste supervise

7.9 Processus collectes

La capture doit montrer le tableau des processus dans la page Monitor. Elle doit mettre en evidence les colonnes PID, nom et utilisateur. Le commentaire doit preciser que cette vue est utile pour l'observation, mais qu'elle ne remplace pas encore une detection avancee basee sur chemin, hash ou arbre de processus.

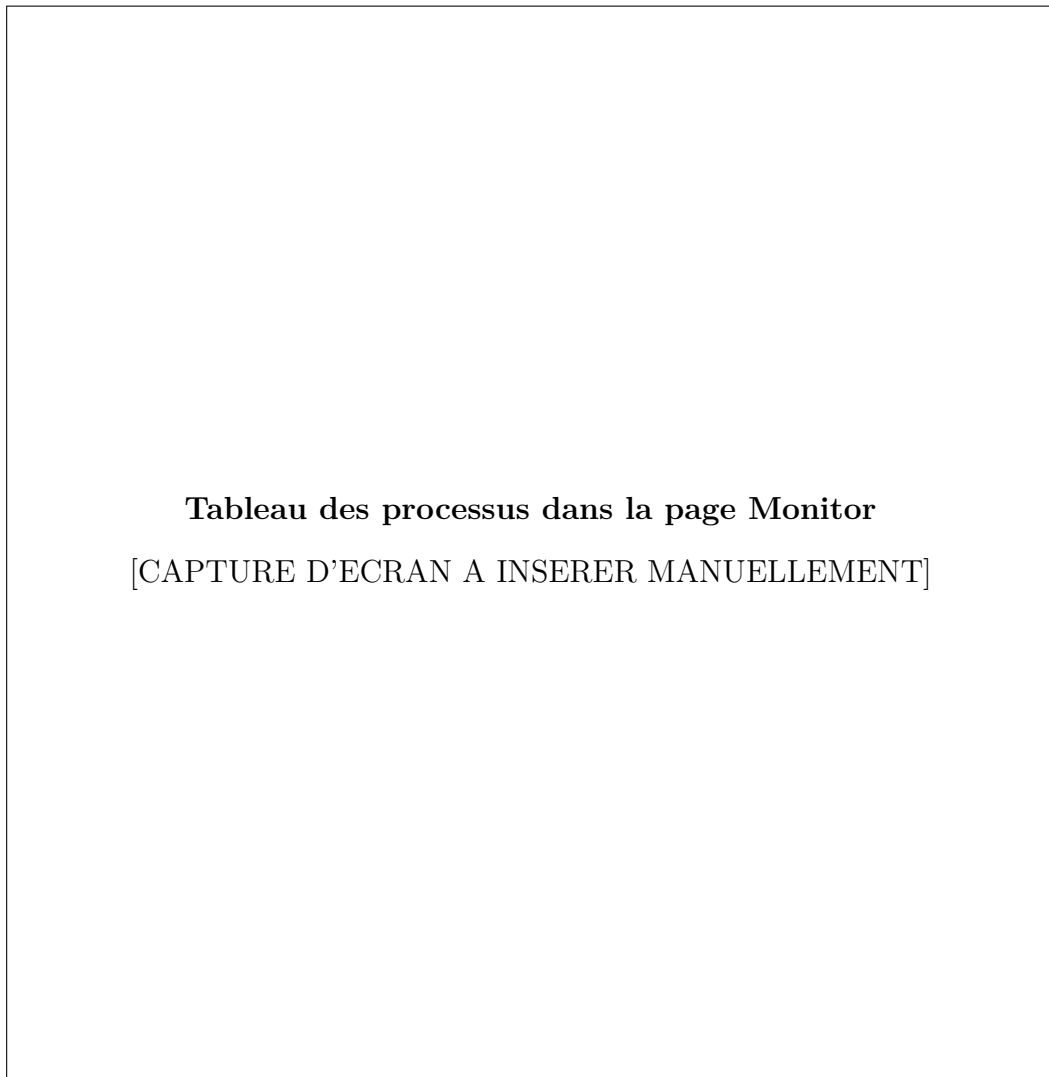


FIGURE 7.8 – Capture d'écran a inserer : Tableau des processus dans la page Monitor

7.10 Partitions et points de montage

La capture doit montrer les partitions et les points de montage. Le commentaire doit rappeler que l'espace utilise n'est pas encore collecte. Cette limite permet d'introduire une amelioration future simple : ajouter `psutil.disk_usage()`.

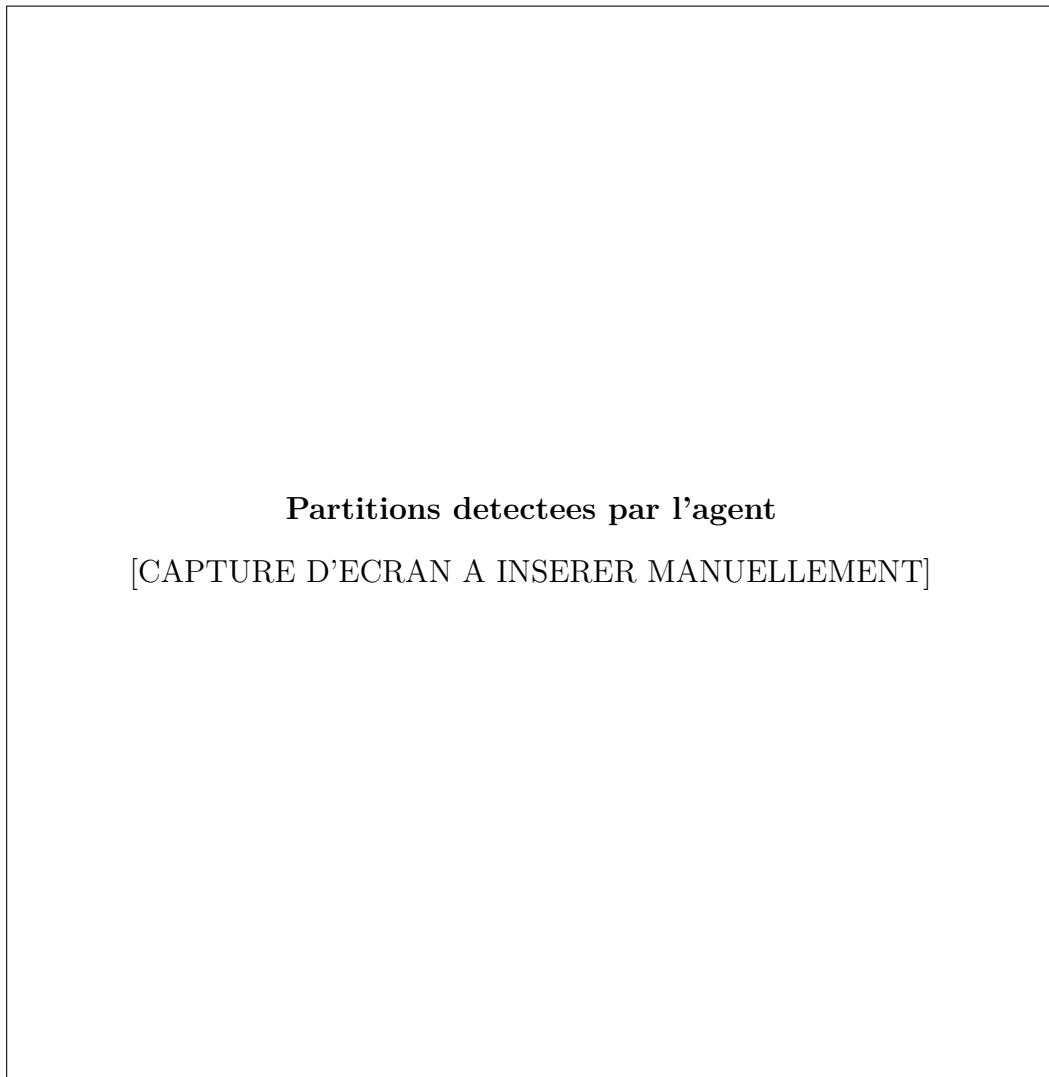


FIGURE 7.9 – Capture d'écran a inserer : Partitions detectees par l'agent

7.11 Utilisateurs

La capture doit montrer la liste ou le nombre d'utilisateurs visibles pour un poste. Le commentaire doit distinguer le comportement Unix et Windows. Sur Unix, la recuperation peut lister les comptes systeme ; sur Windows, elle depend des sessions retournees par `psutil.users()`.

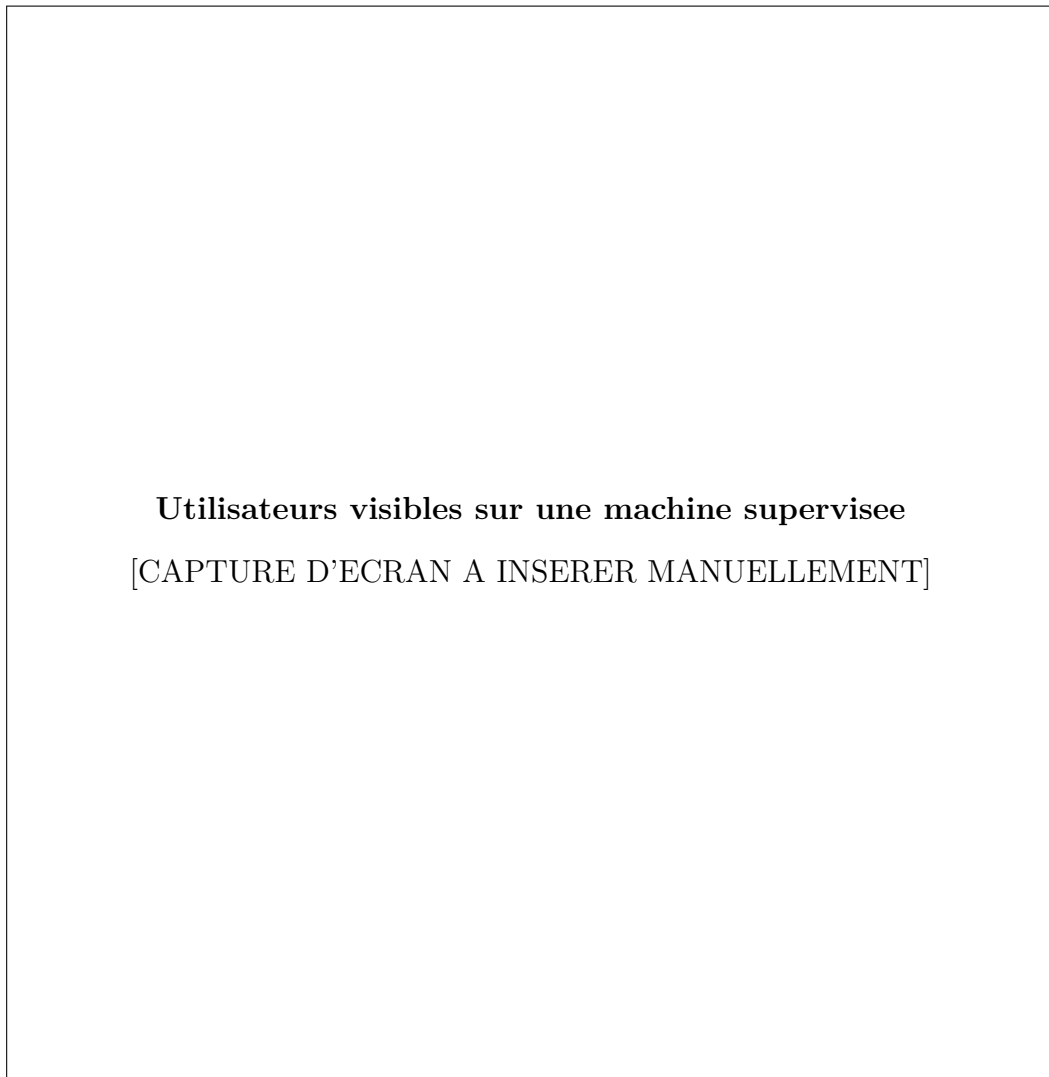


FIGURE 7.10 – Capture d'écran a inserer : Utilisateurs visibles sur une machine supervisee

7.12 Liste des machines disponibles

La capture doit presenter la page Available Devices. Le commentaire doit expliquer le role de cette page : donner une vue rapide du parc connu, de l'OS et du statut en ligne ou hors ligne. Il faut aussi mentionner que la colonne Location reste a finaliser.

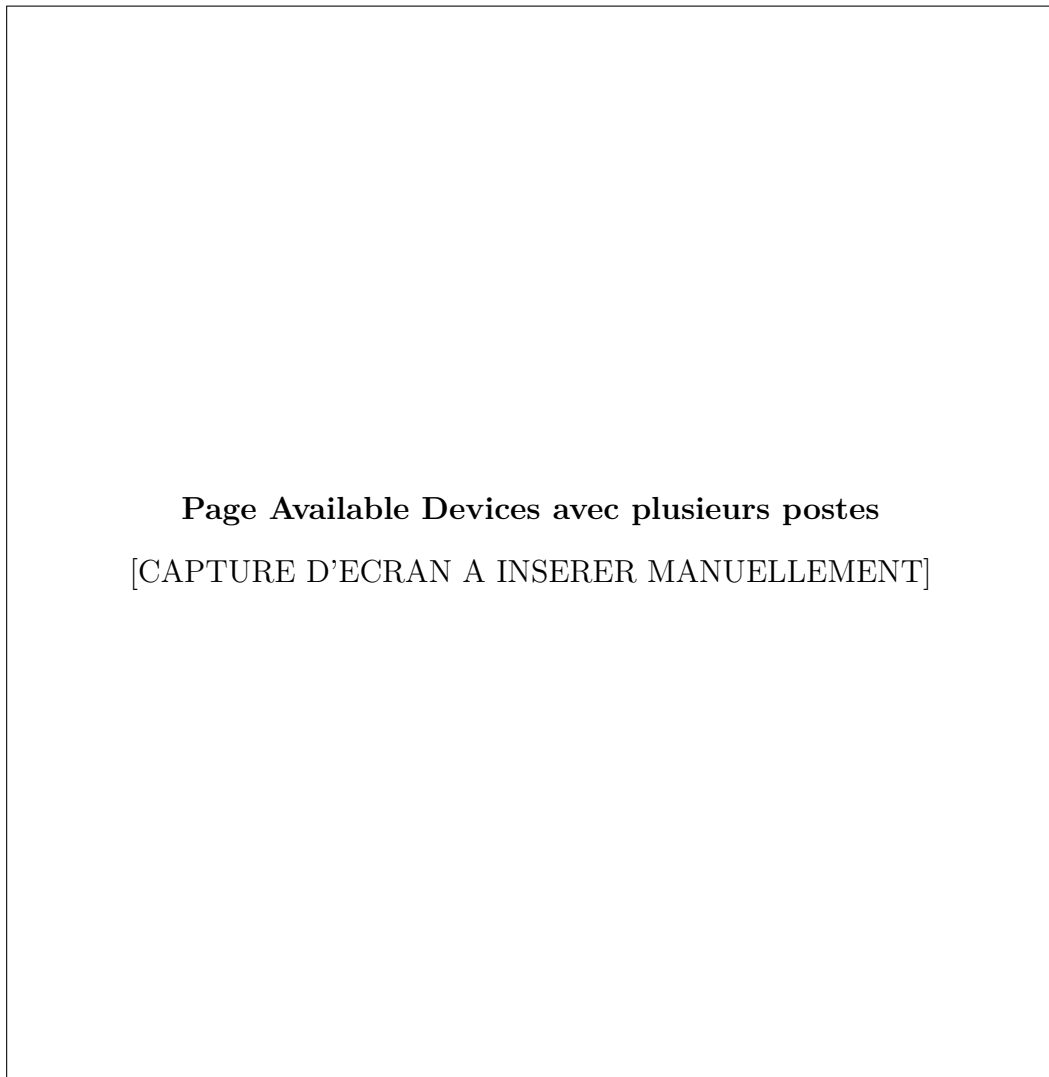


FIGURE 7.11 – Capture d'écran a inserer : Page Available Devices avec plusieurs postes

7.13 Selection d'un poste en ligne

La capture doit montrer la liste deroulante ou le selecteur de machine dans la page Monitor. Elle doit prouver que seuls les postes en ligne sont proposes. Le commentaire doit relier ce comportement a la route `/api/computers/live`.

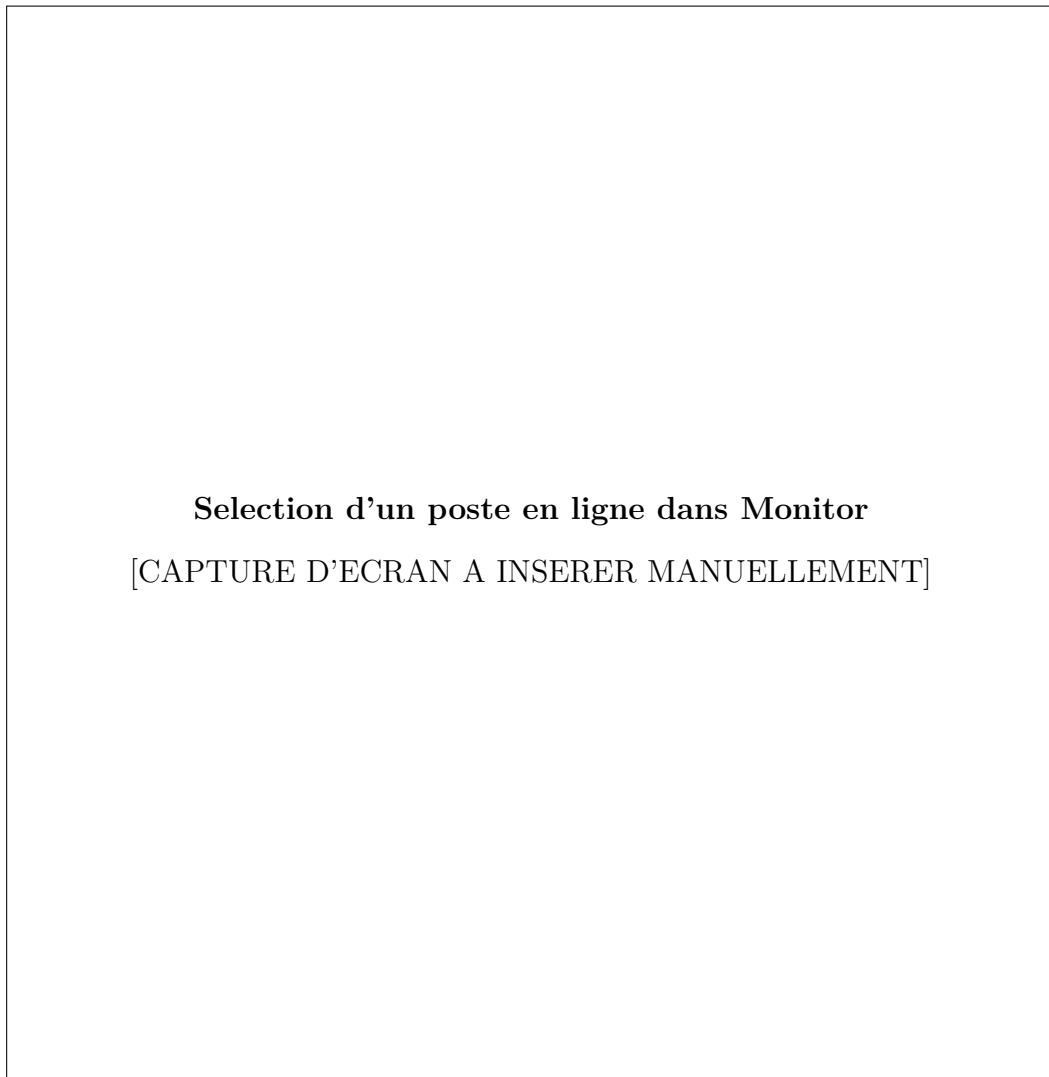


FIGURE 7.12 – Capture d'écran a inserer : Selection d'un poste en ligne dans Monitor

7.14 Metriques principales

La capture doit montrer les cartes CPU, memoire et boot time. Le commentaire doit expliquer que la memoire et le boot time sont relies au backend, tandis que la metrique CPU doit etre finalisee pour devenir dynamique. Cette honnetete technique renforce la credibilite du memoire.

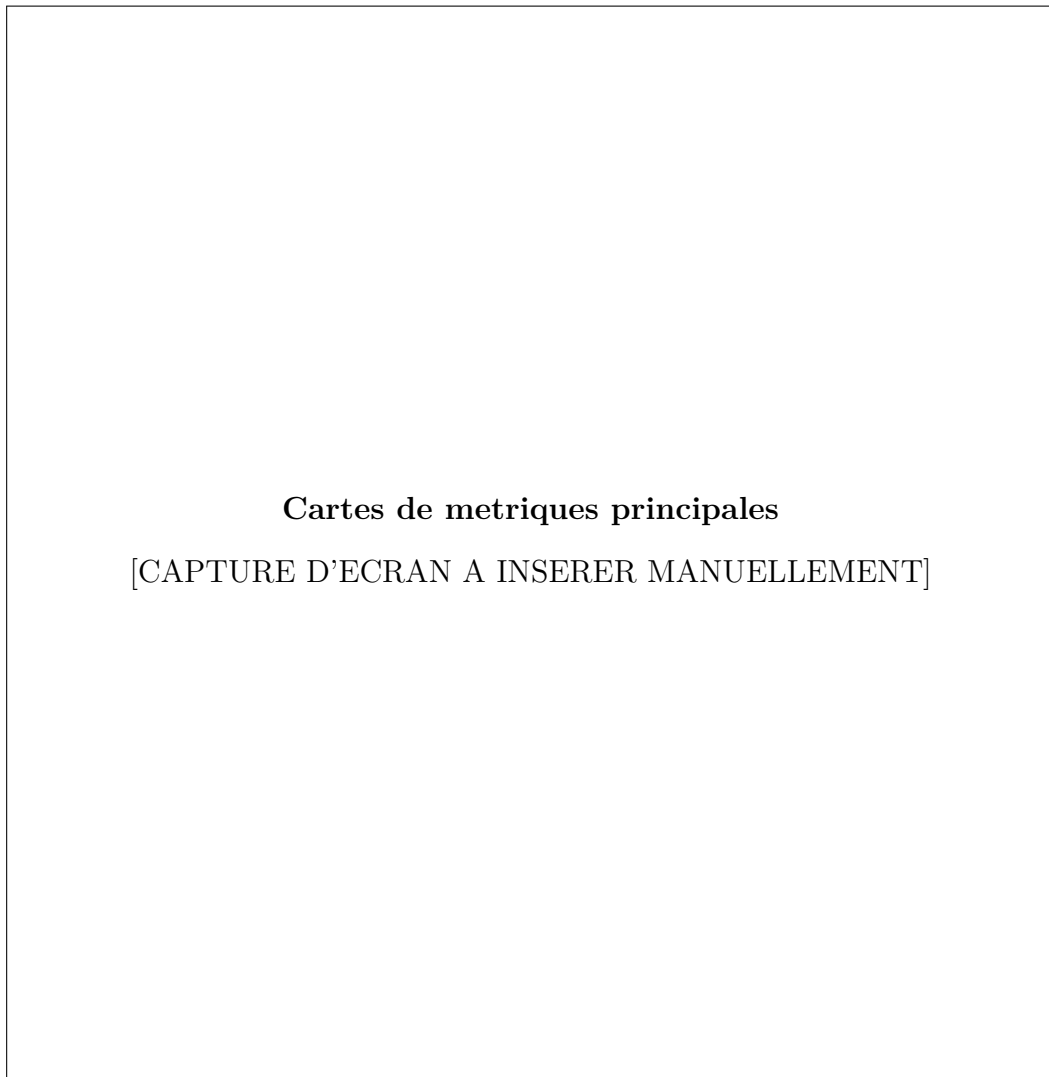


FIGURE 7.13 – Capture d'écran a inserer : Cartes de metriques principales

7.15 Etat hors ligne apres arret de l'agent

La capture doit montrer un poste passant hors ligne apres l'arret de l'agent et l'expiration du seuil. Le commentaire doit expliquer le calcul base sur `last_heartbeat` et le seuil de 150 secondes. Cette capture est essentielle pour valider la supervision de disponibilite.

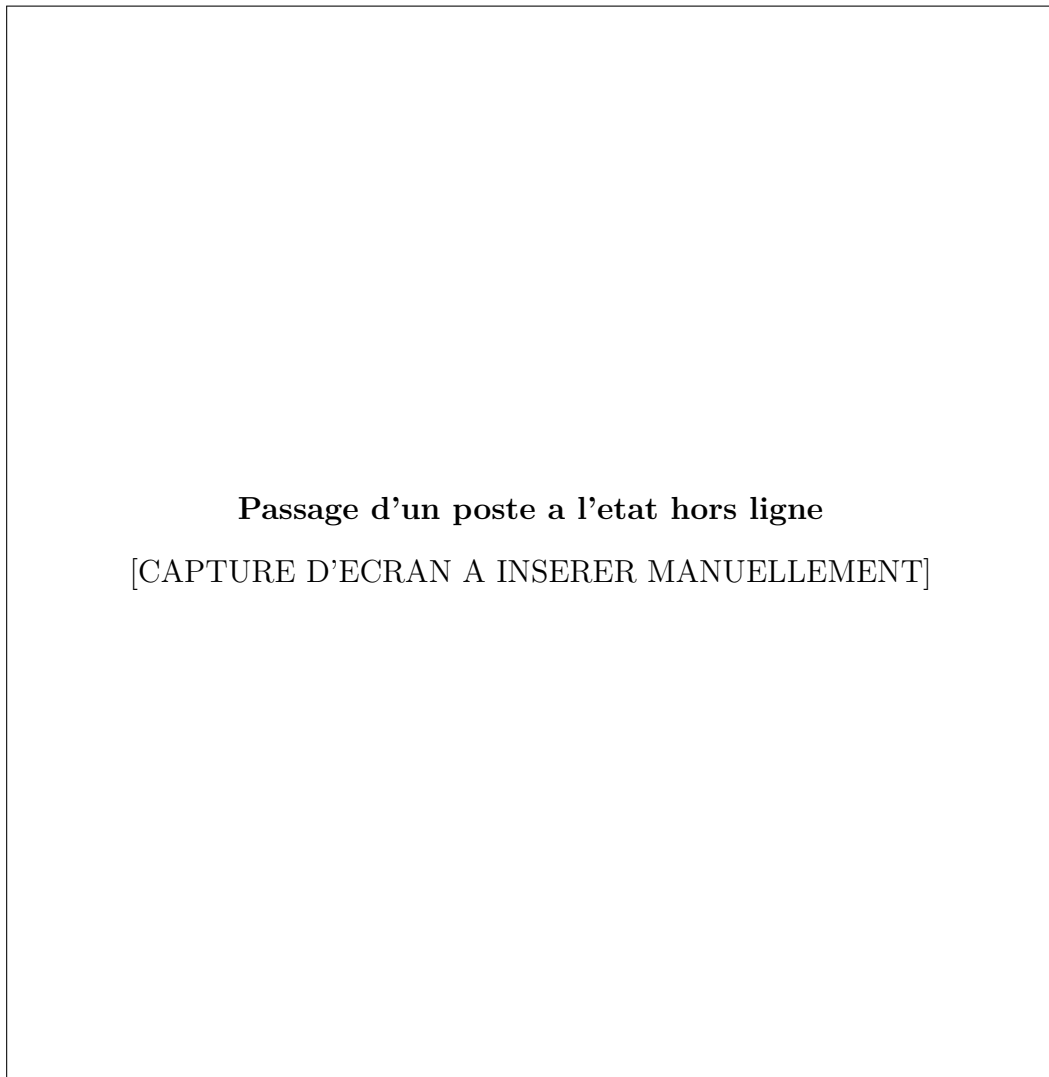


FIGURE 7.14 – Capture d'écran a inserer : Passage d'un poste a l'etat hors ligne

7.16 Retour en ligne apres relance

La capture doit montrer le meme poste redevenant en ligne apres relance de l'agent. Le commentaire doit preciser que le poste conserve son identifiant et que le heartbeat remet `is_alive` a vrai. Cette sequence montre la continuite du cycle de vie.

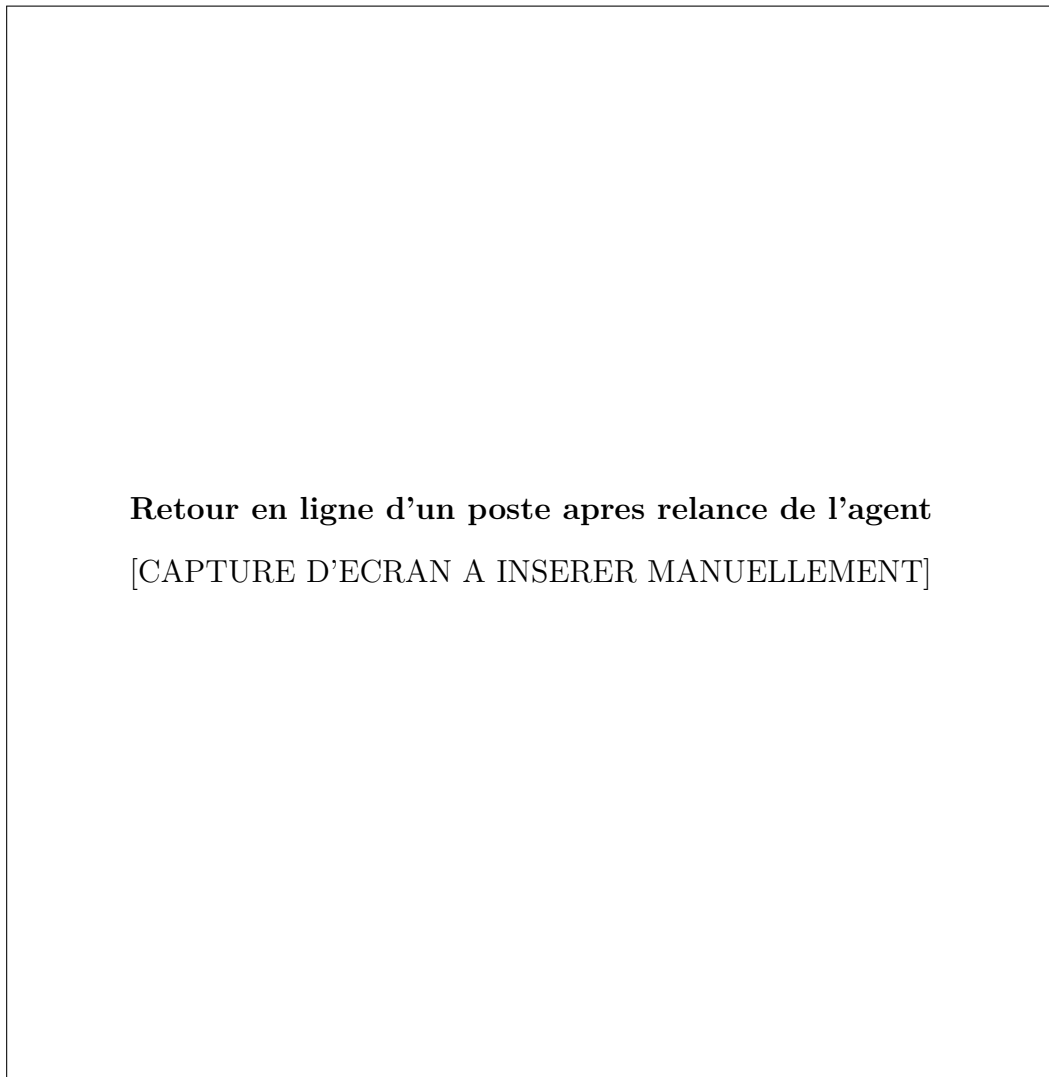
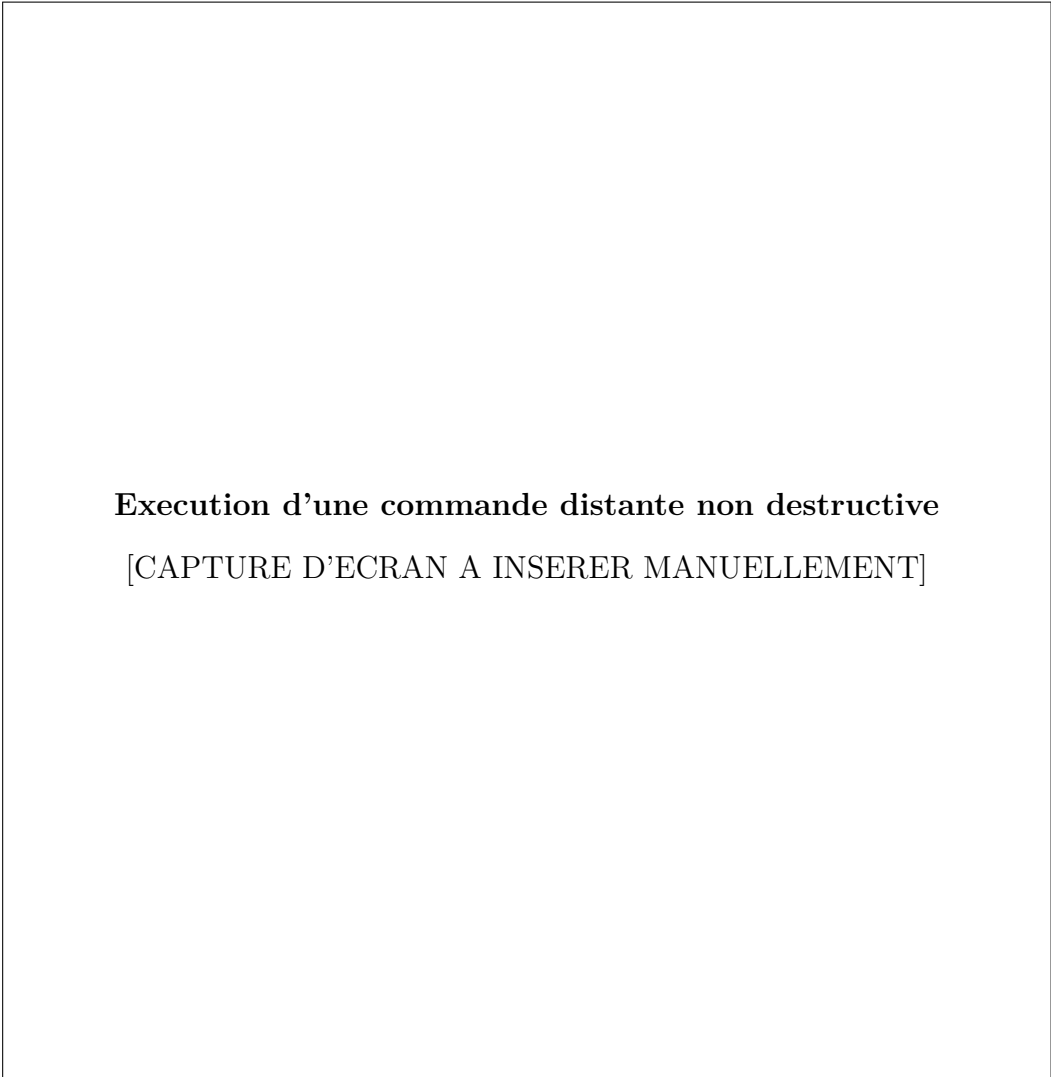


FIGURE 7.15 – Capture d'écran a inserer : Retour en ligne d'un poste apres relance de l'agent

7.17 Commande distante de diagnostic

La capture doit montrer une commande non destructive, par exemple une commande de consultation systeme. Le commentaire doit expliquer le flux : Streamlit envoie la commande au serveur, le serveur la pousse par WebSocket, l'agent execute et renvoie la sortie. Il faut rappeler que cette fonctionnalite doit rester en environnement de test tant que la securite n'est pas renforcee.



Execution d'une commande distante non destructive
[CAPTURE D'ECRAN A INSERER MANUELLEMENT]

FIGURE 7.16 – Capture d'écran a inserer : Execution d'une commande distante non destructive

7.18 Erreur de commande

La capture doit montrer une commande invalide ou refusee. Elle permet d'expliquer la gestion de `stdout`, `stderr` et du code retour. Le commentaire doit proposer une amelioration : afficher un message structure avec statut, sortie standard, erreur standard et horodatage.

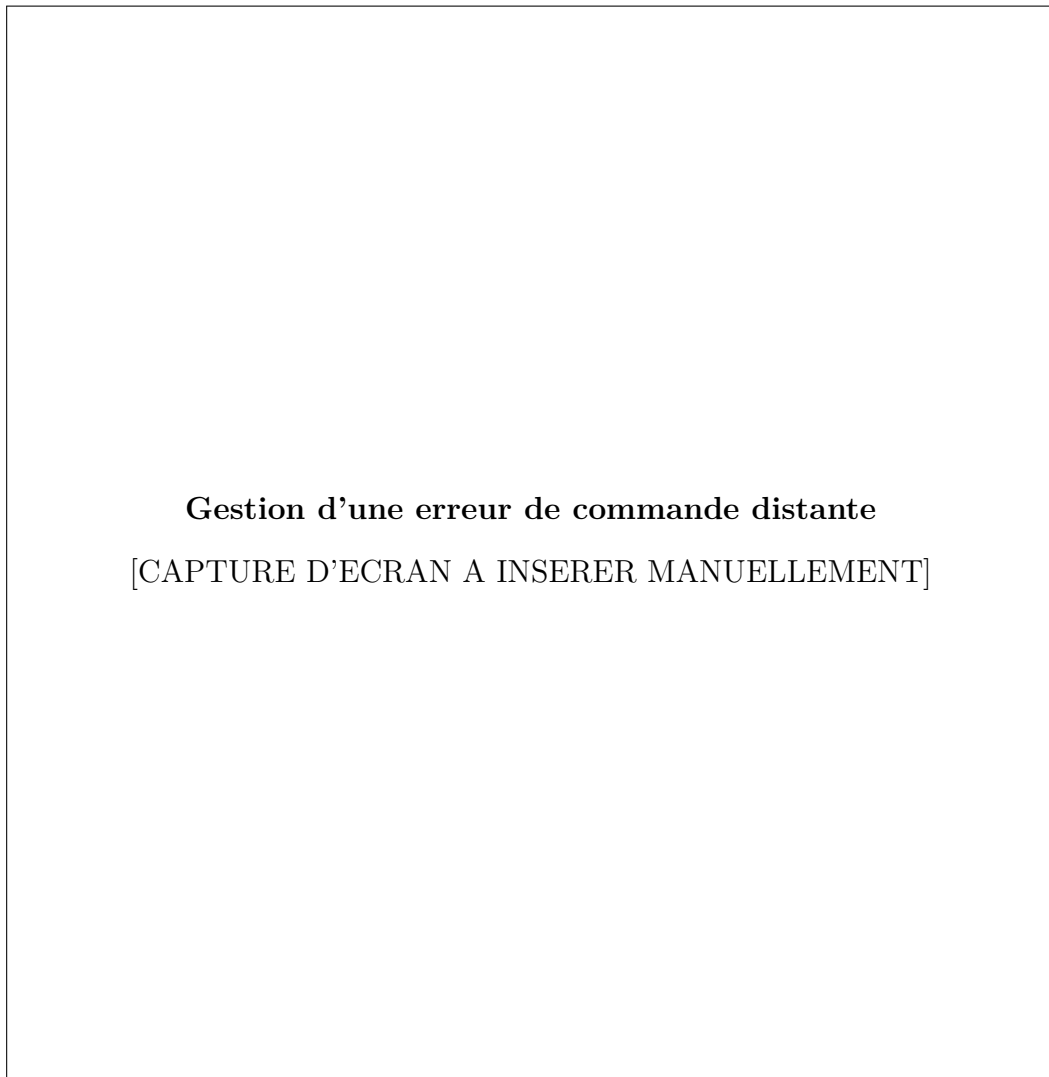


FIGURE 7.17 – Capture d'écran a inserer : Gestion d'une erreur de commande distante

7.19 Arret et redemarrage

La capture doit montrer les boutons Shutdown et Restart sans forcement executer l'action pendant la demonstration. Le commentaire doit insister sur le caractere sensible de ces actions et sur la necessite d'ajouter une confirmation et un role administrateur.

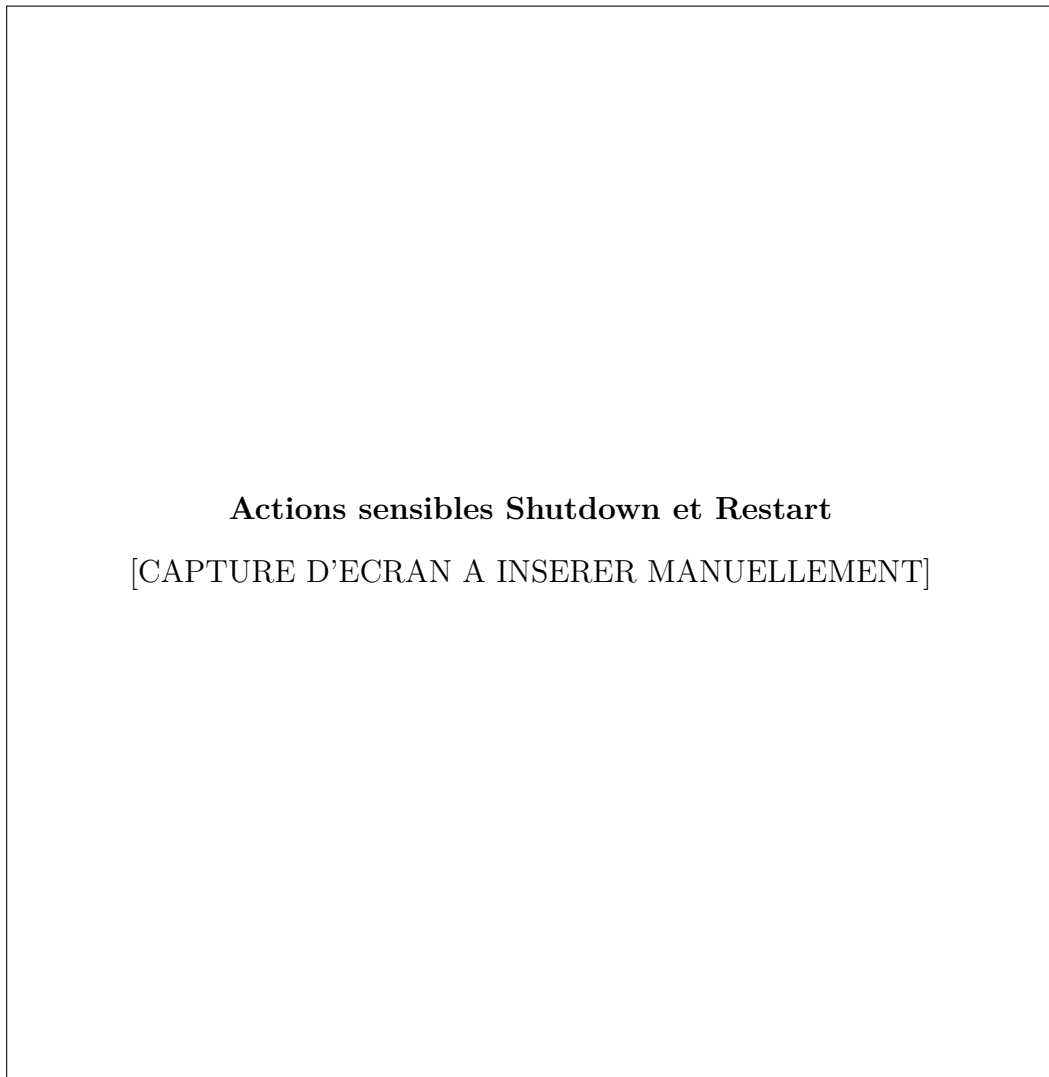


FIGURE 7.18 – Capture d'écran a inserer : Actions sensibles Shutdown et Restart

7.20 Connexion USB

La capture doit montrer la detection d'un evenement USB cote agent ou cote serveur. Le commentaire doit preciser que l'evenement brut ne constitue pas une preuve d'attaque. Il devient utile lorsqu'il est compare a une politique de confiance.

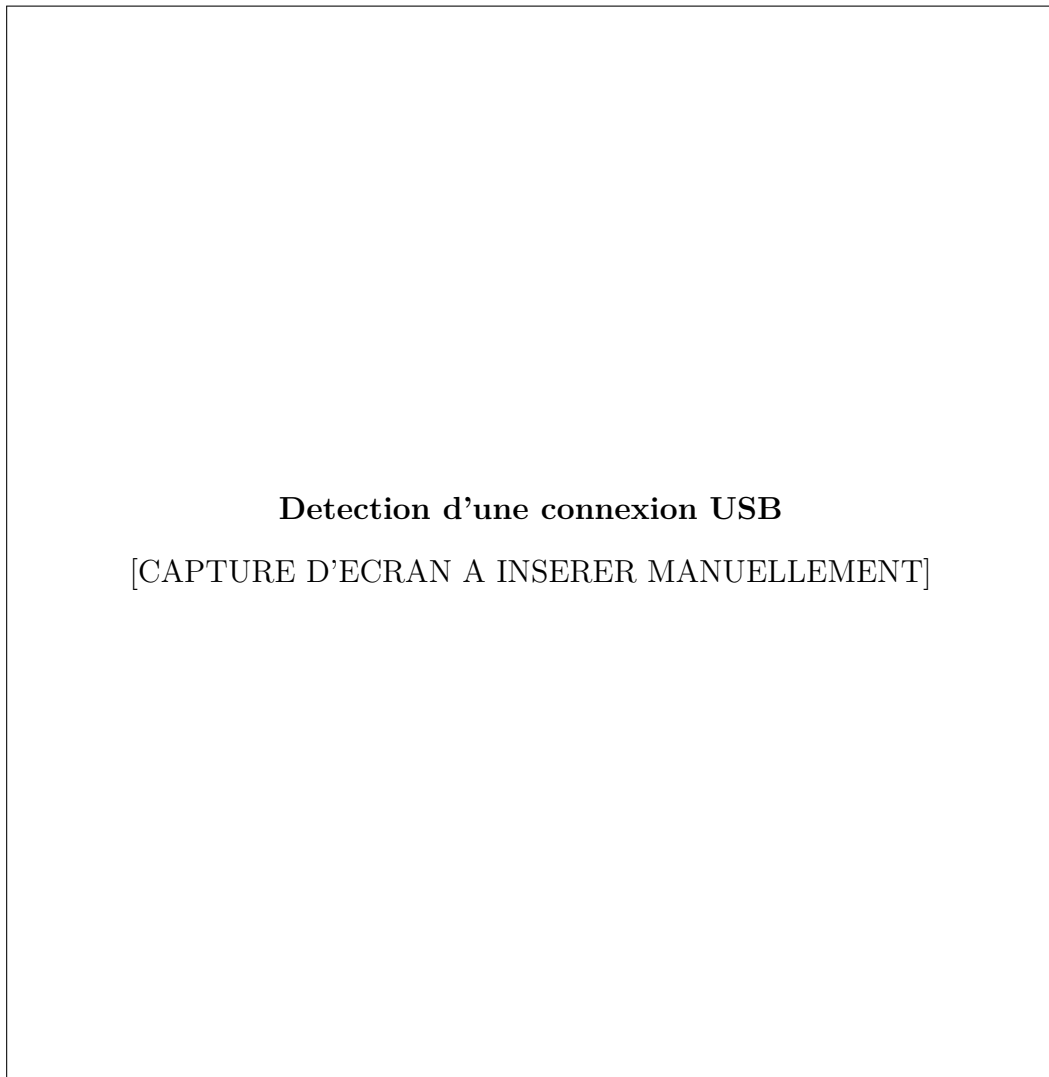


FIGURE 7.19 – Capture d'écran a inserer : Detection d'une connexion USB

7.21 Deconnexion USB

La capture doit montrer une deconnexion USB. Le commentaire doit expliquer l'interet de suivre les deux directions : branchement et retrait. Dans une analyse future, la duree de connexion peut aider a comprendre l'usage d'un peripherique.

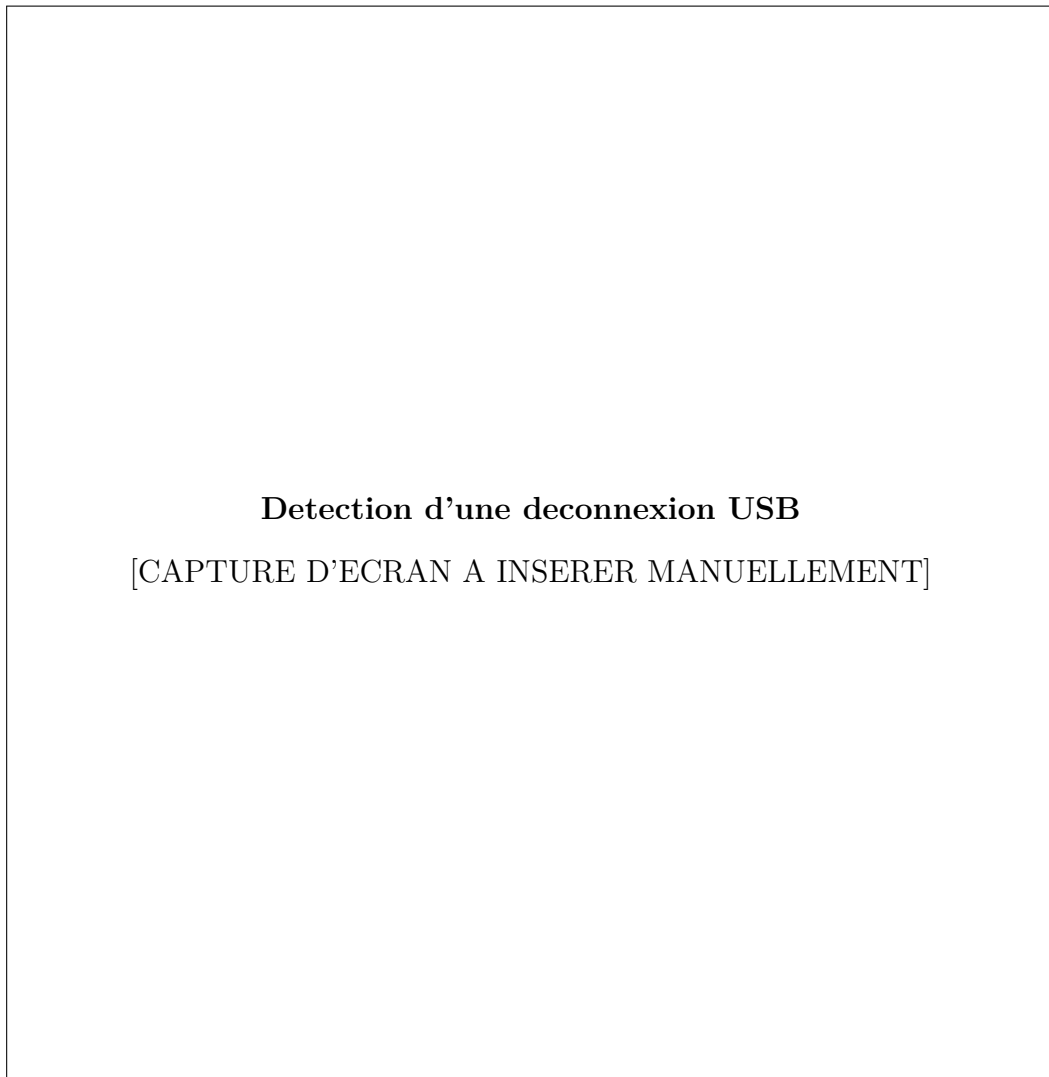


FIGURE 7.20 – Capture d'écran a inserer : Detection d'une deconnexion USB

7.22 Page Network Alerts

La capture doit montrer la page Network Alerts avec une alerte generee par le modele ML. Le commentaire doit expliquer les champs : timestamp, type d'alerte, prediction, score, adresses source/destination, port.

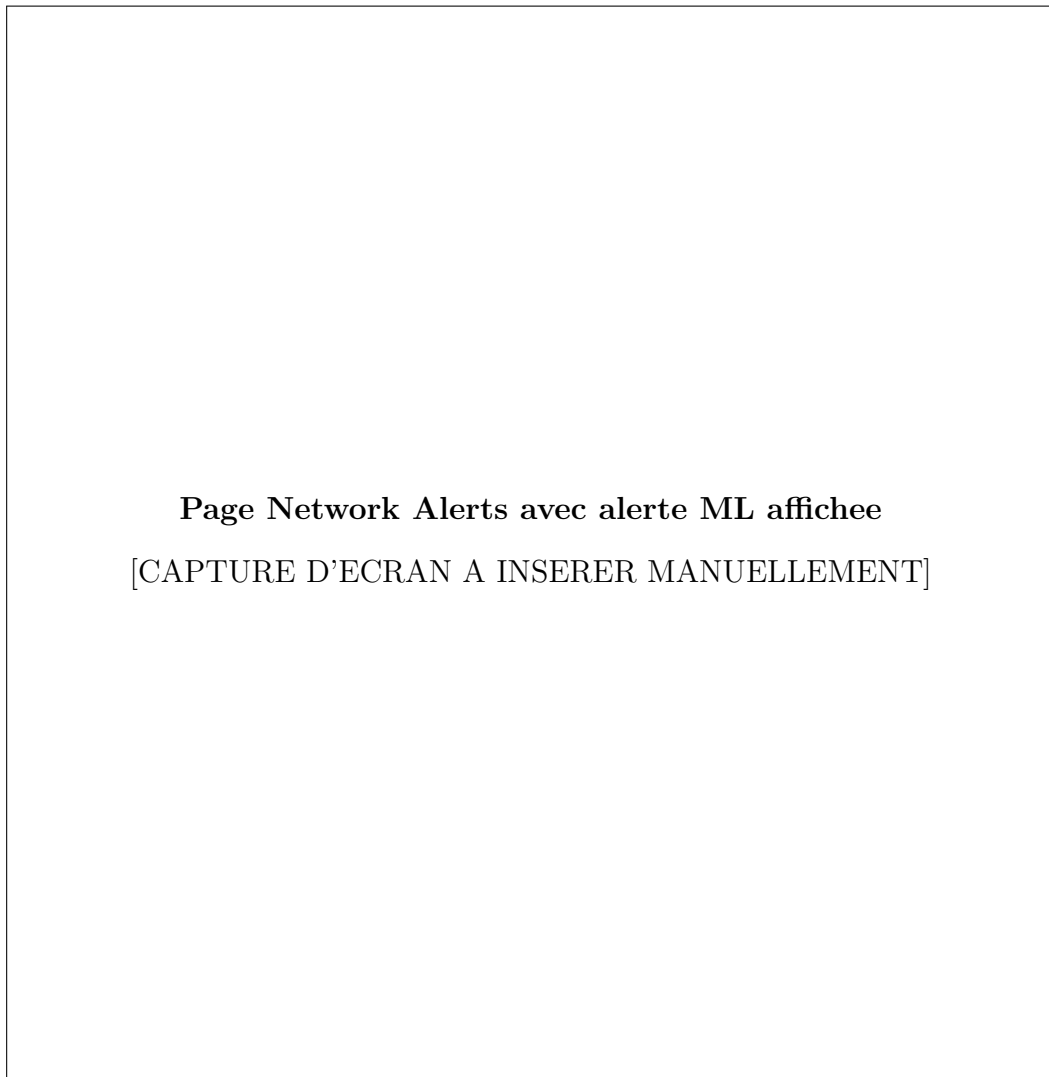


FIGURE 7.21 – Capture d'écran a inserer : Page Network Alerts avec alerte ML affichee

7.23 Page Reports

La capture doit montrer la page Reports. Le commentaire doit indiquer le contenu attendu d'un rapport : identite machine, metriques, processus, reseau, disques, evenements et observations. Elle prepare les perspectives sans pretendre que le generateur est deja operationnel.

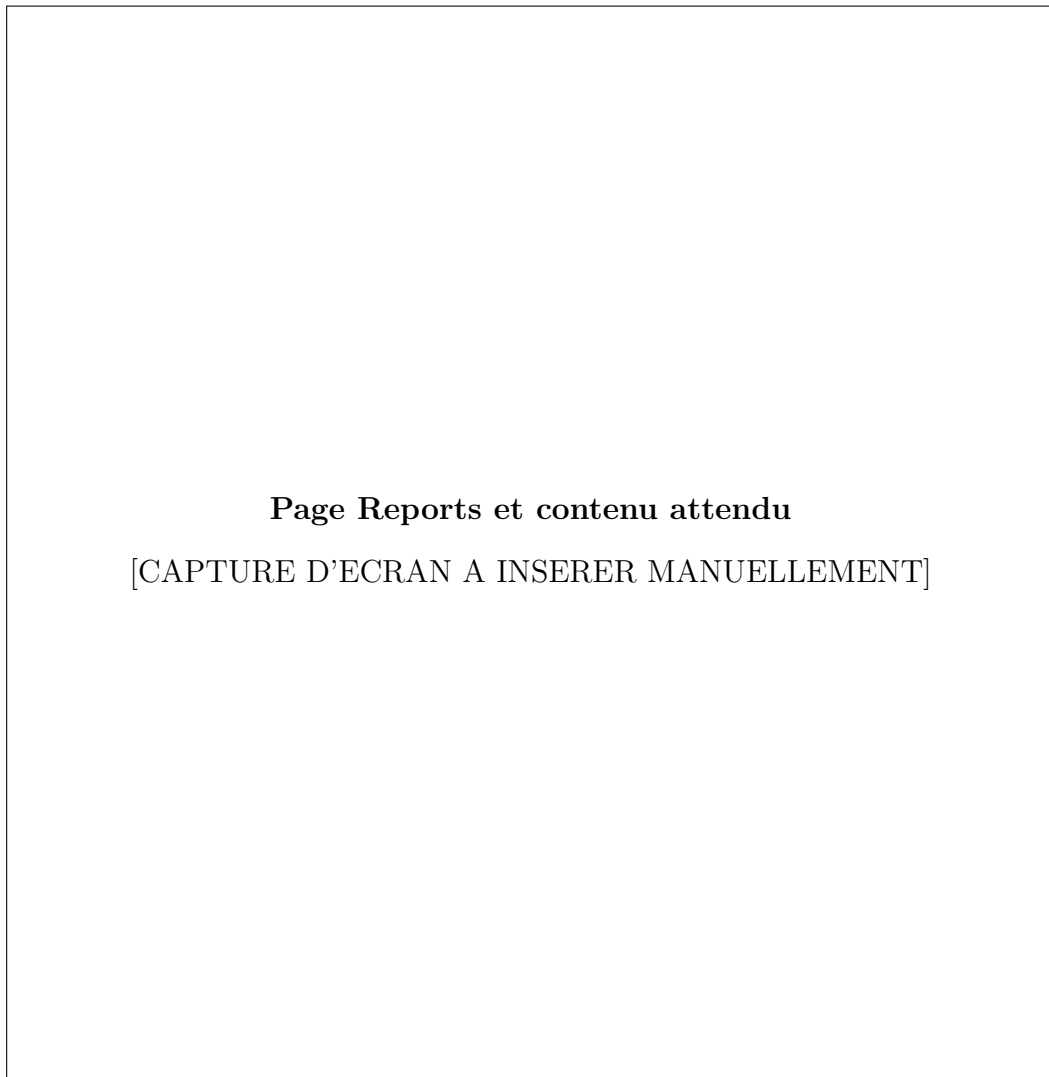


FIGURE 7.22 – Capture d'écran a inserer : Page Reports et contenu attendu

7.24 Page Blacklist

La capture doit montrer la page Blacklist. Le commentaire doit expliquer les choix possibles : blacklist de machines, de processus, de peripheriques USB ou de commandes. Il faut souligner que cette definition doit etre fixee avant implementation pour eviter une fonctionnalite confuse.

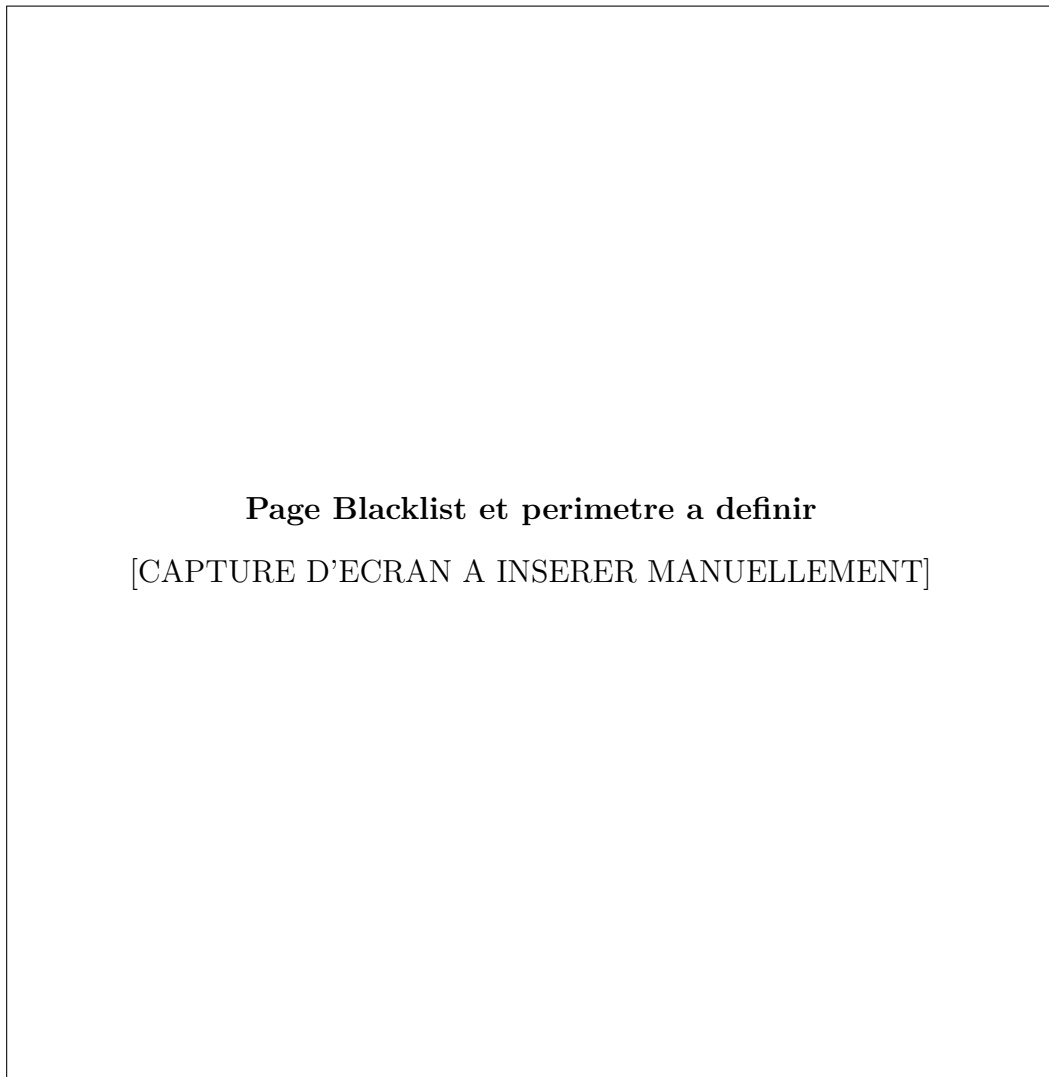


FIGURE 7.23 – Capture d'écran a inserer : Page Blacklist et perimetre a definir

7.25 Pages Locations

La capture doit montrer la page Locations ou Add Locations. Le commentaire doit relier cette fonctionnalite au contexte des salles de l'ISTA NTIC. Il doit aussi mentionner que le routeur retourne encore des donnees de test et que l'association poste-salle doit etre finalisee.

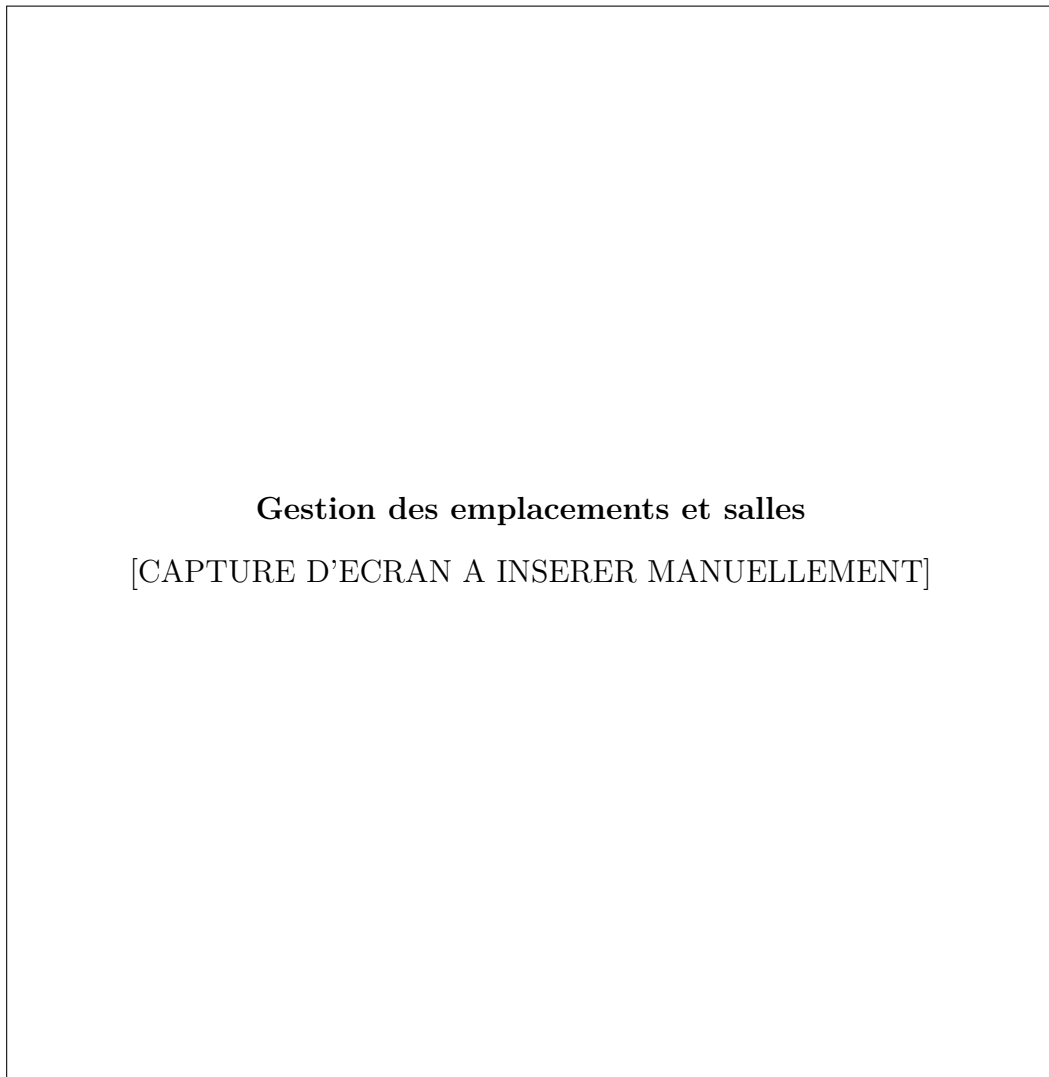


FIGURE 7.24 – Capture d'écran a inserer : Gestion des emplacements et salles

7.26 Scenario multi-postes

La capture doit presenter plusieurs agents connectes au meme serveur. Cette capture est importante pour montrer que la plateforme ne se limite pas a une seule machine. Le commentaire doit discuter les enjeux de concurrence, de lisibilite et de filtrage lorsque le nombre de postes augmente.

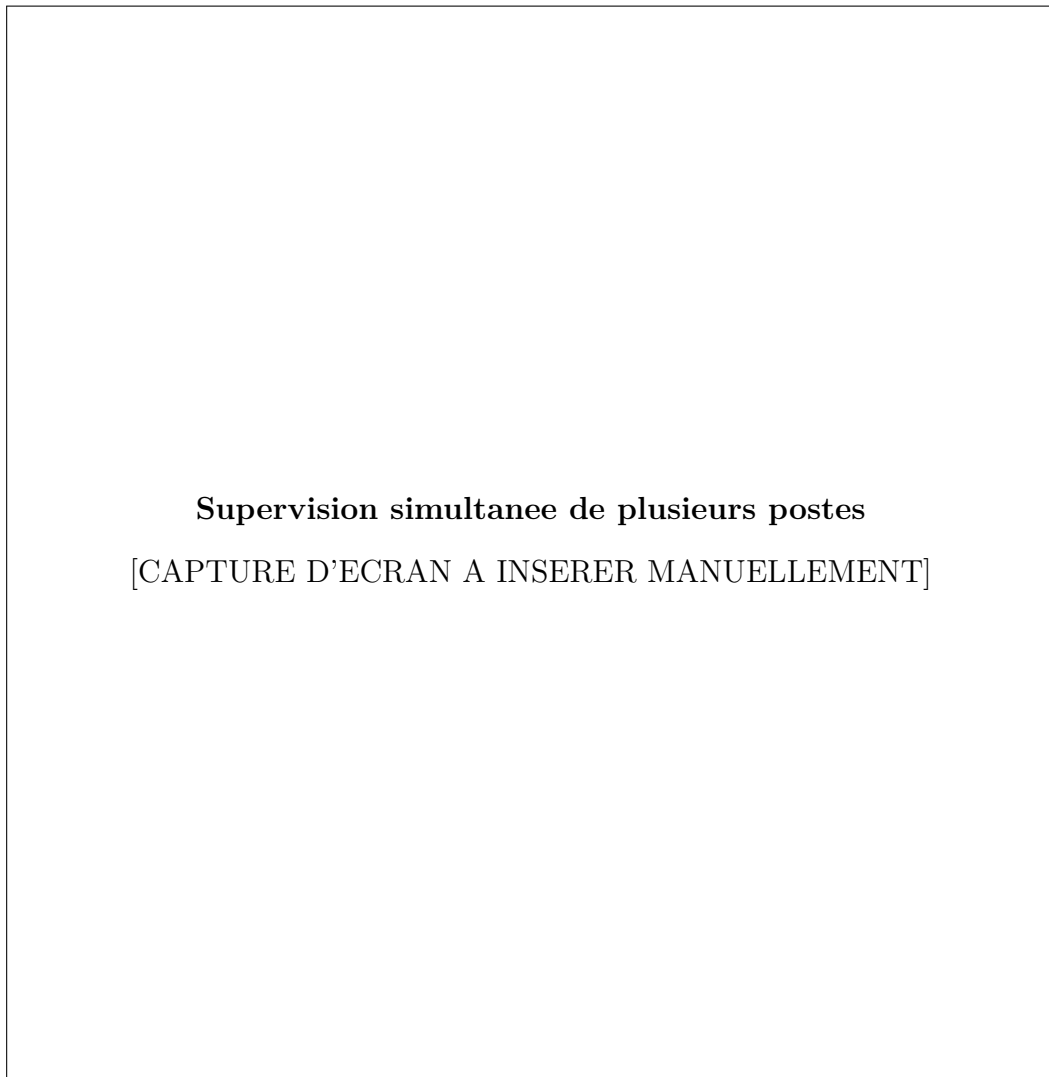


FIGURE 7.25 – Capture d'écran a inserer : Supervision simultanee de plusieurs postes

7.27 Scenario de changement d'adresse IP

La capture doit montrer une interface dont l'adresse change apres une modification reseau. Le commentaire doit expliquer la comparaison effectuee par `updateNetInfo`. Il faut aussi indiquer que le test depend de l'environnement et peut etre simule sur une interface de laboratoire.

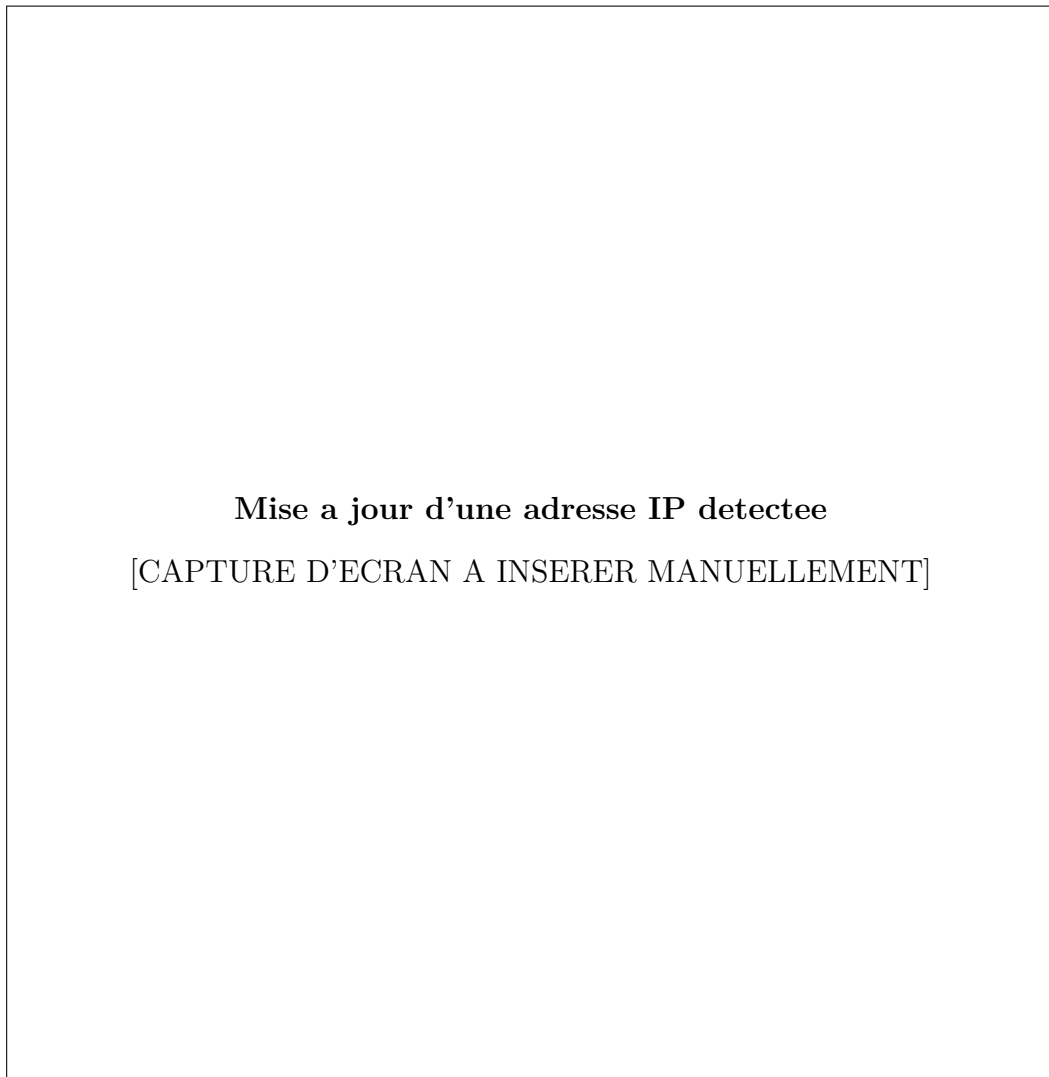


FIGURE 7.26 – Capture d'écran a inserer : Mise a jour d'une adresse IP detectee

7.28 Scenario de changement de partitions

La capture doit montrer l'apparition d'une nouvelle partition ou d'un support de stockage. Le commentaire doit expliquer la difference entre disque interne, partition montee et support amovible. Cette capture peut etre associee a la detection USB si le support est une cle USB.

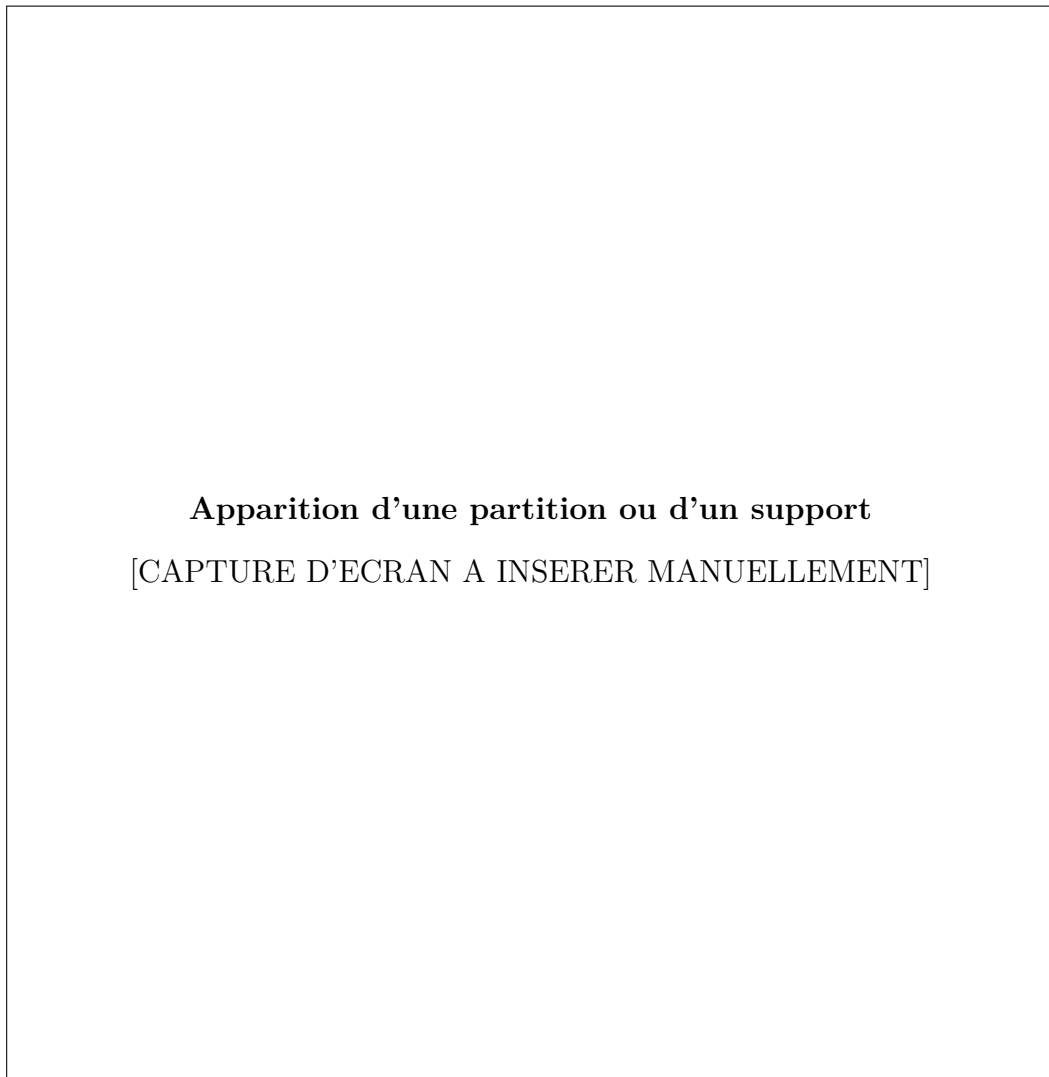


FIGURE 7.27 – Capture d'écran a inserer : Apparition d'une partition ou d'un support

7.29 Controle de coherence API-interface

La capture doit montrer cote a cote une reponse API et l'affichage Streamlit correspondant. Le commentaire doit expliquer que cette verification evite de confondre une erreur backend et une erreur d'affichage. Elle est particulierement utile pour les metriques memoire et les listes de processus.

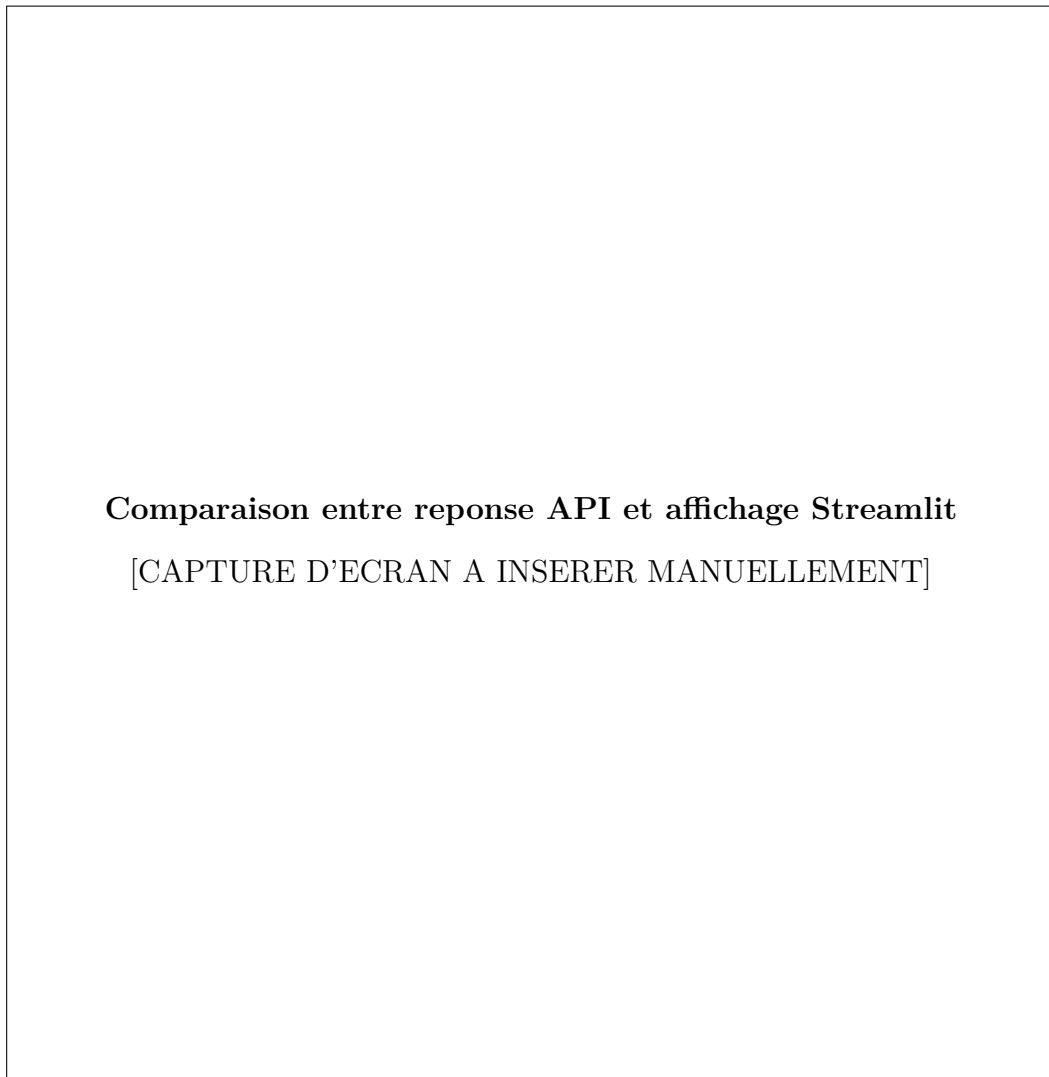


FIGURE 7.28 – Capture d'écran a inserer : Comparaison entre reponse API et affichage Streamlit

7.30 Trace d'une erreur d'authentification agent

La capture doit montrer une tentative de mise a jour avec une mauvaise empreinte ou un identifiant invalide. Le commentaire doit indiquer que le serveur refuse la mise a jour. Il faut ensuite rappeler que ce mecanisme est minimal et doit etre remplace par des secrets d'agent robustes.

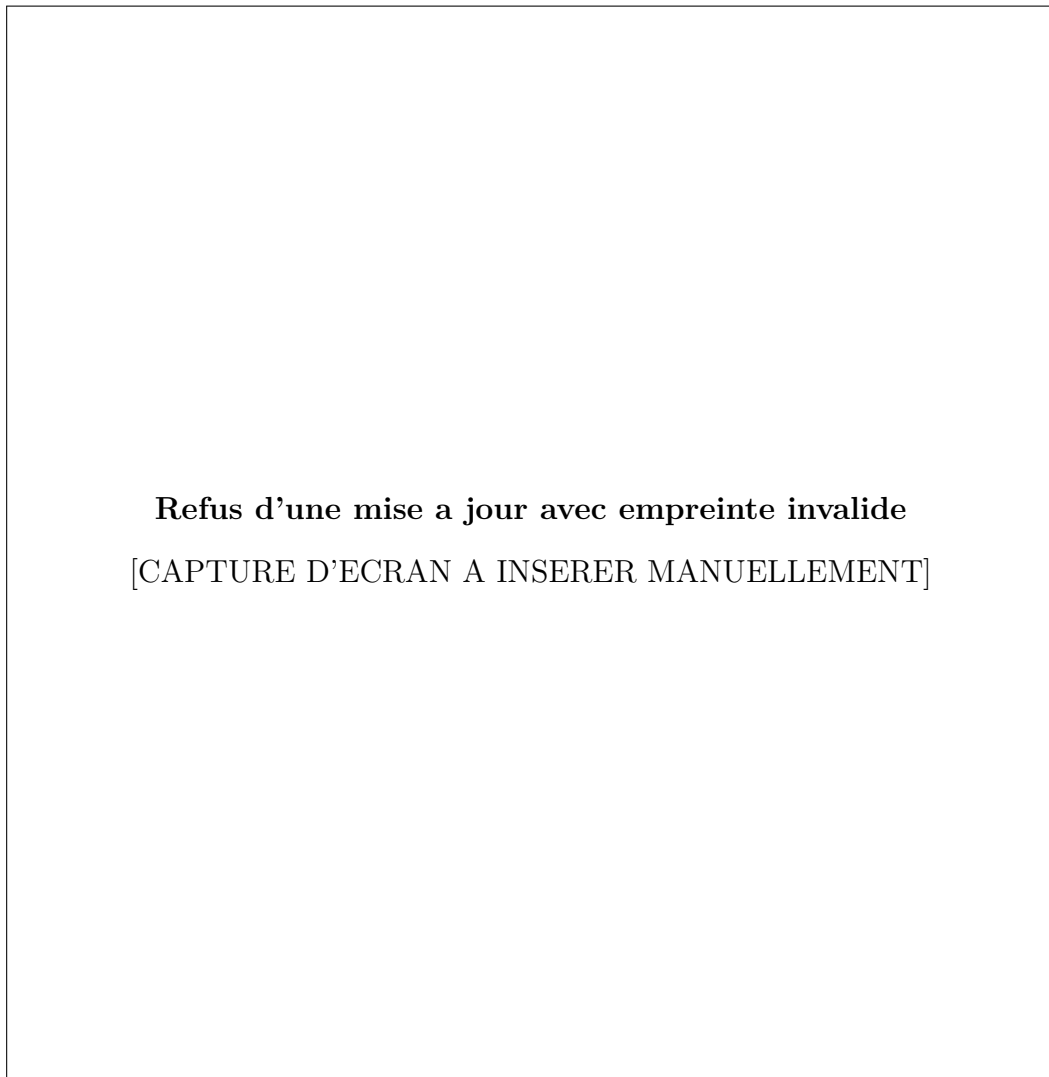


FIGURE 7.29 – Capture d'écran a inserer : Refus d'une mise a jour avec empreinte invalide

7.31 Synthèse visuelle pour la soutenance

La dernière capture doit servir de synthèse : tableau de bord, machine sélectionnée, métriques et tableaux visibles. Elle peut être utilisée pendant la soutenance pour expliquer rapidement l'apport global d'Ernevil avant de passer aux détails de code et de sécurité.

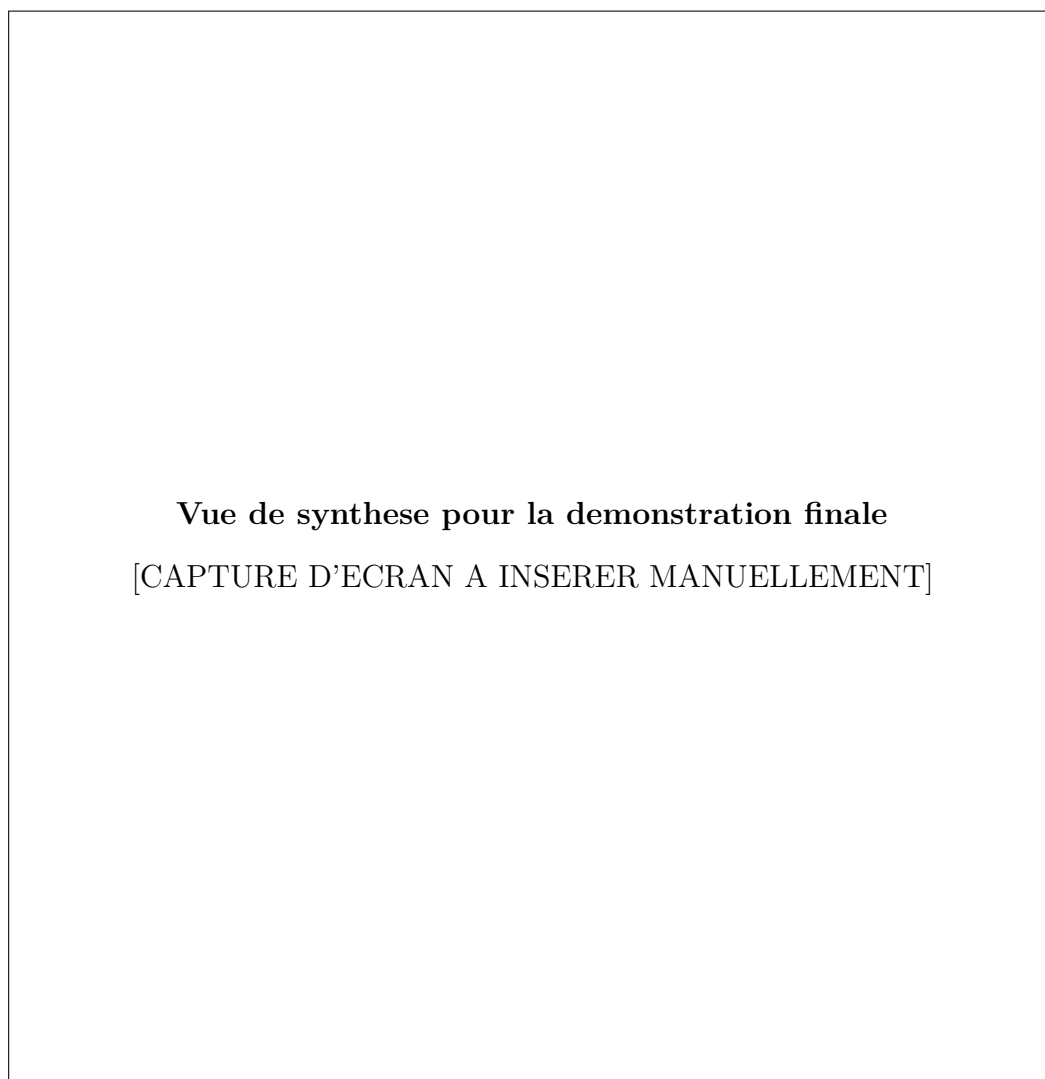


FIGURE 7.30 – Capture d'écran a inserer : Vue de synthese pour la demonstration finale

Chapitre 8

Intelligence artificielle et detection

8.1 Perimetre

L'intelligence artificielle dans Ernevil est deployee sous la forme d'un module de detection d'intrusion (IDS) par apprentissage supervise. Il repose sur un `HistGradientBoostingClassifier` (scikit-learn), entraine hors ligne sur le jeu de donnees public **CIC-IDS-2017** (100 000 lignes etiquetees, 18 caracteristiques numeriques extraites des flux reseau). Apres optimisation par `GridSearchCV` (validation croisee 5 plis, score F1-macro), le modele final atteint environ **95% de precision** avec un taux de faux positifs inferieur a 3%.

8.2 Inference temps reel

Cote agent, `netsniffer.py` capture les paquets via `scapy` et les regroupe en flux par conversation unique. `alert_handler.py` charge `bestmodel.pkl` via `joblib` et, pour chaque flux expire, extrait les 18 caracteristiques et les presente au modele. Si la prediction est `malignant`, une alerte (`flow_id`, `prediction`, `score`, `adresses`, `ports`) est envoyee au serveur par WebSocket (`/api/ws/alert`), stockee dans `notifications.json` et affichee dans l'interface `netalerts.py`.

8.3 Mecanismes deterministes

En parallele du module ML, des mecanismes deterministes restent actifs : detection d'indisponibilite par absence de heartbeat, detection de changement d'interface reseau par comparaison `ifaddr`, detection de changement de partitions par `psutil.disk_partitions()`, detection d'evenement USB par callback `usb-monitor` et observation des processus par `psutil.process_iter()`.

8.4 Limites

Le modele est entraine sur un jeu public qui ne reflète pas forcément le trafic d'un etablissement de formation. La capture reseau par scapy requiert des privileges root, limitant le deploiement sur certains postes. Les donnees exposees (IP, ports, horodatage) doivent etre protegees. Dans un etablissement de formation, la transparence et la proportionnalite sont essentielles.

Chapitre 9

Tests et validation

9.1 Strategie de test

La validation d’Ernevil doit couvrir les trois couches : agent, serveur et interface. Les tests fonctionnels verifient que les donnees sont collectees et visibles. Les tests de supervision verifient que le heartbeat et l’etat en ligne/hors ligne fonctionnent. Les tests de securite identifient les faiblesses avant deploiement. Les tests d’ergonomie verifient que l’administrateur comprend rapidement l’etat du parc.

9.2 Scenarios fonctionnels

TABLE 9.1 – Scenarios de validation fonctionnelle

ID	Scenario	Resultat attendu	Observation
T01	Lancer un agent sans identifiant local.	Creation d’un poste en base.	Valide si l’API renvoie un <code>computer_id</code> .
T02	Relancer un agent deja enregistre.	Reutilisation de l’identifiant local.	Depend de <code>agentdata.json</code> .
T03	Arreter un agent.	Passage hors ligne apres le seuil.	Valide avec attente superieure a 150 s.
T04	Modifier l’adresse IP d’une interface.	Mise a jour de <code>netinfo</code> .	A verifier selon nom d’interface.
T05	Brancher un peripherique USB.	Evenement genere cote agent.	Persistence serveur a finaliser.
T06	Consulter la page Monitor.	Affichage memoire, processus, disques, reseau.	Valide si API disponible.

T07	Envoyer une commande distante.	Retour stdout/stderr af-fiche.	A limiter a environ-nement de test.
-----	--------------------------------	--------------------------------	-------------------------------------

9.3 Validation de l'enregistrement

Le test d'enregistrement consiste a supprimer l'identifiant local de l'agent, lancer le serveur puis lancer l'agent. Le serveur doit recevoir une requete `POST /api/computers`, creer la machine et renvoyer l'identifiant. Le fichier `agentdata.json` doit ensuite conserver cet identifiant pour eviter un nouvel enregistrement a chaque lancement.

9.4 Validation du heartbeat

Le test du heartbeat consiste a lancer un agent puis observer `last_heartbeat`. Tant que l'agent fonctionne, la valeur doit etre actualisee et `is_alive` doit rester vrai. Lorsque l'agent est arrete, la tache serveur doit marquer la machine hors ligne apres le seuil. Ce test valide la distinction entre "machine connue" et "machine actuellement disponible".

9.5 Validation de la supervision

La supervision est validee en comparant les donnees affichees par Streamlit avec les donnees locales du poste. La memoire affichee doit correspondre approximativement a `psutil.virtual_memory().percent`. Les partitions doivent correspondre aux partitions montees. Les interfaces doivent contenir les adresses IPv4 actives. La liste des processus doit contenir les processus visibles par l'utilisateur qui lance l'agent.

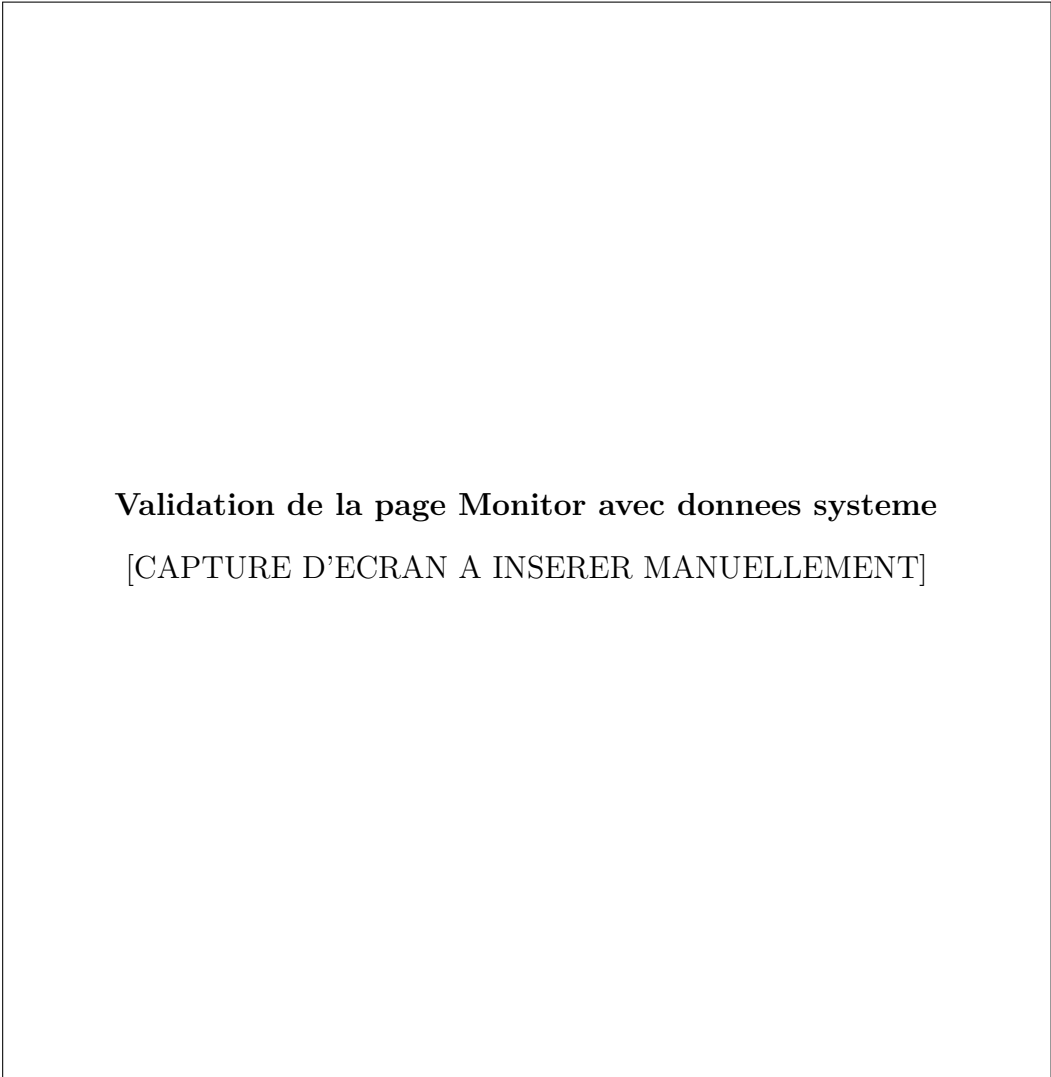


FIGURE 9.1 – Capture d’écran a inserer : Validation de la page Monitor avec donnees systeme

9.6 Validation de la securite

La validation de securite n’a pas pour objectif de prouver que la plateforme est prete pour une exposition publique. Elle sert a identifier les points a renforcer. Les tests doivent verifier qu’une mise a jour avec une mauvaise empreinte est refusee, que les routes sensibles ne sont pas accessibles sans controle dans une version cible, que les commandes distantes sont journalisees et que les requetes SQL sont remplacees par des requetes parametrees.

TABLE 9.2 – Points de controle securite

Controle	Etat actuel	Action recommandee
----------	-------------	--------------------

Authentification agent	Empreinte + identifiant.	Secret d'enrolement et rotation.
Authentification admin	Table preparee, routes absentes.	JWT/session, hachage fort, roles.
SQL injection	Requetes textuelles presentes.	ORM ou parametres lies.
Transport chiffre	Non configure dans le depot.	HTTPS/WSS avec certificats.
Commandes distantes	Executees via WebSocket.	Liste blanche, audit, confirmation.
Journalisation	Peu presente.	Logs serveur et logs d'action.

9.7 Resultats observes

Les resultats attendus restent modestes et coherents avec le stade du projet. Ernevil permet de construire un inventaire vivant des postes, d'afficher des donnees systeme et de detecter une indisponibilite par heartbeat. Les modules d'alertes et de commandes montrent la faisabilite d'une interaction temps reel. Les parties inachevees, notamment la gestion des utilisateurs, les emplacements, les rapports et la persistance des alertes, constituent des axes d'amelioration clairement identifies.

9.8 Limites identifiees

Les limites les plus importantes sont la securite des commandes distantes, l'absence d'authentification administrateur complete, l'absence d'historique, la base SQLite pour un usage local, les pages encore vides, l'absence de tests automatises dans le depot, le modele entraine sur un jeu public et non sur des donnees locales, et la necessite de privileges root pour la capture reseau. Ces limites doivent etre presentees devant le jury comme des constats techniques serieux et non comme des defaults caches.

9.9 Captures de validation complementaires

Les validations suivantes complètent les scenarios fonctionnels. Elles doivent etre produites pendant la phase finale de preparation du memoire, lorsque l'environnement de demonstration est stabilise. Chaque capture doit etre accompagnee d'une phrase d'observation dans la version finale imprimee.

9.9.1 Validation de la route de liste complete

Cette capture doit montrer l'appel a `/api/computers` et le tableau correspondant dans l'interface. Elle permet de verifier que la liste affichee n'est pas construite manuellement, mais bien consommee depuis l'API.

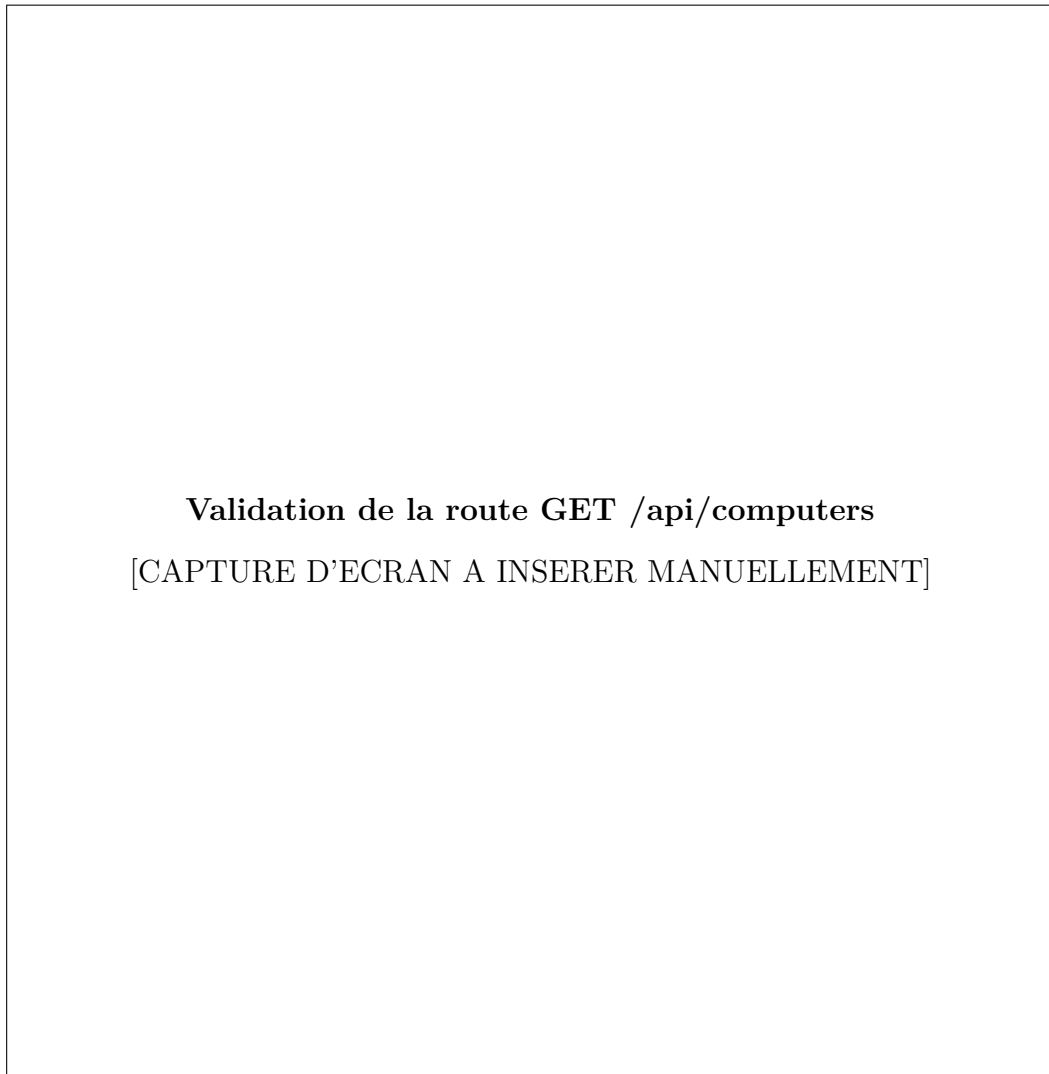


FIGURE 9.2 – Capture d'écran a inserer : Validation de la route GET `/api/computers`

9.9.2 Validation de la route des postes en ligne

Cette capture doit montrer la difference entre la liste complete et la liste des machines en ligne. Elle confirme que `/api/computers/live` filtre selon `is_alive`. Le commentaire doit rappeler que le filtre depend du heartbeat.

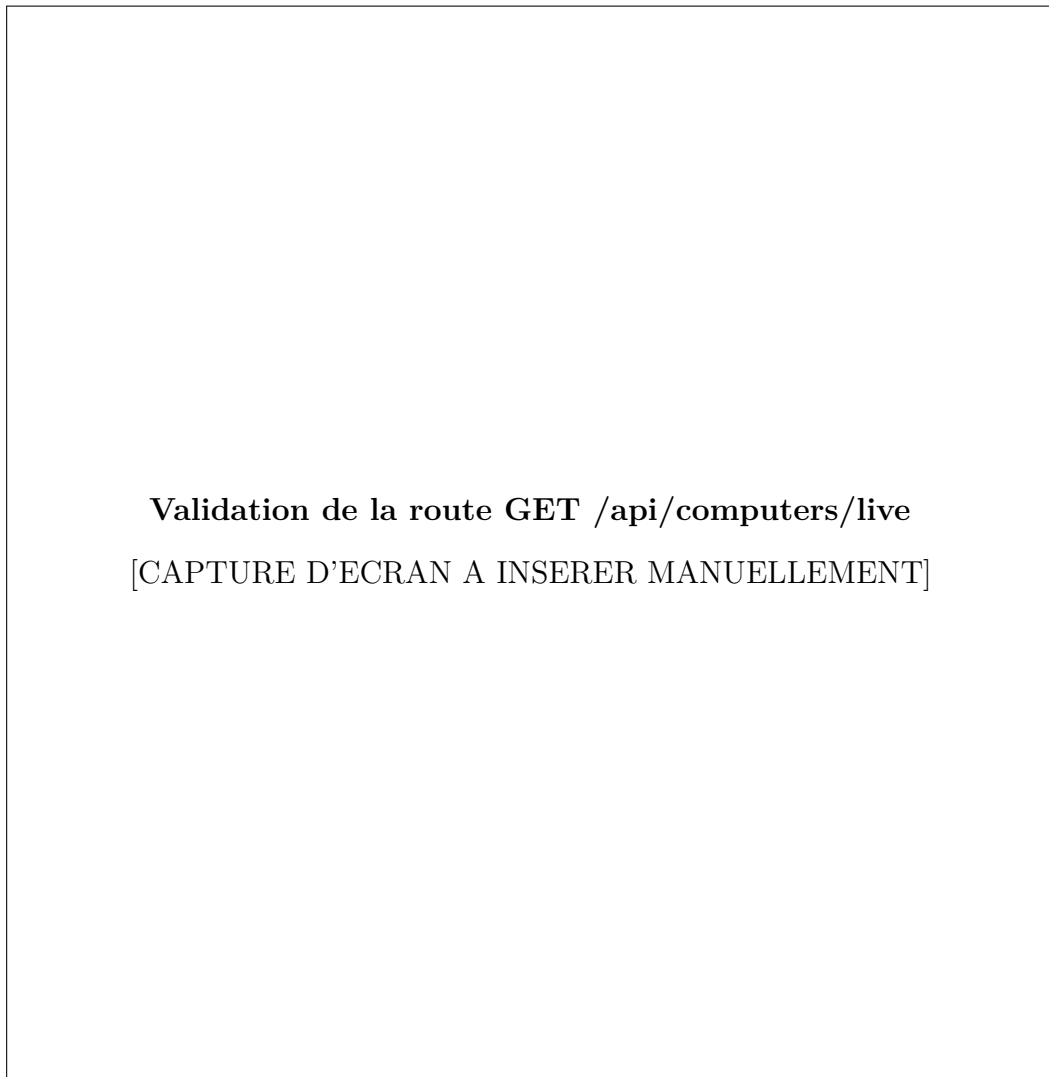


FIGURE 9.3 – Capture d'écran à insérer : Validation de la route GET /api/computers/live

9.9.3 Validation d'une mise à jour mémoire

Cette capture doit montrer une variation de la mémoire entre deux lectures. L'objectif n'est pas d'obtenir une valeur spectaculaire, mais de prouver que la mise à jour est dynamique et que la base ne reste pas figée après l'enregistrement initial.

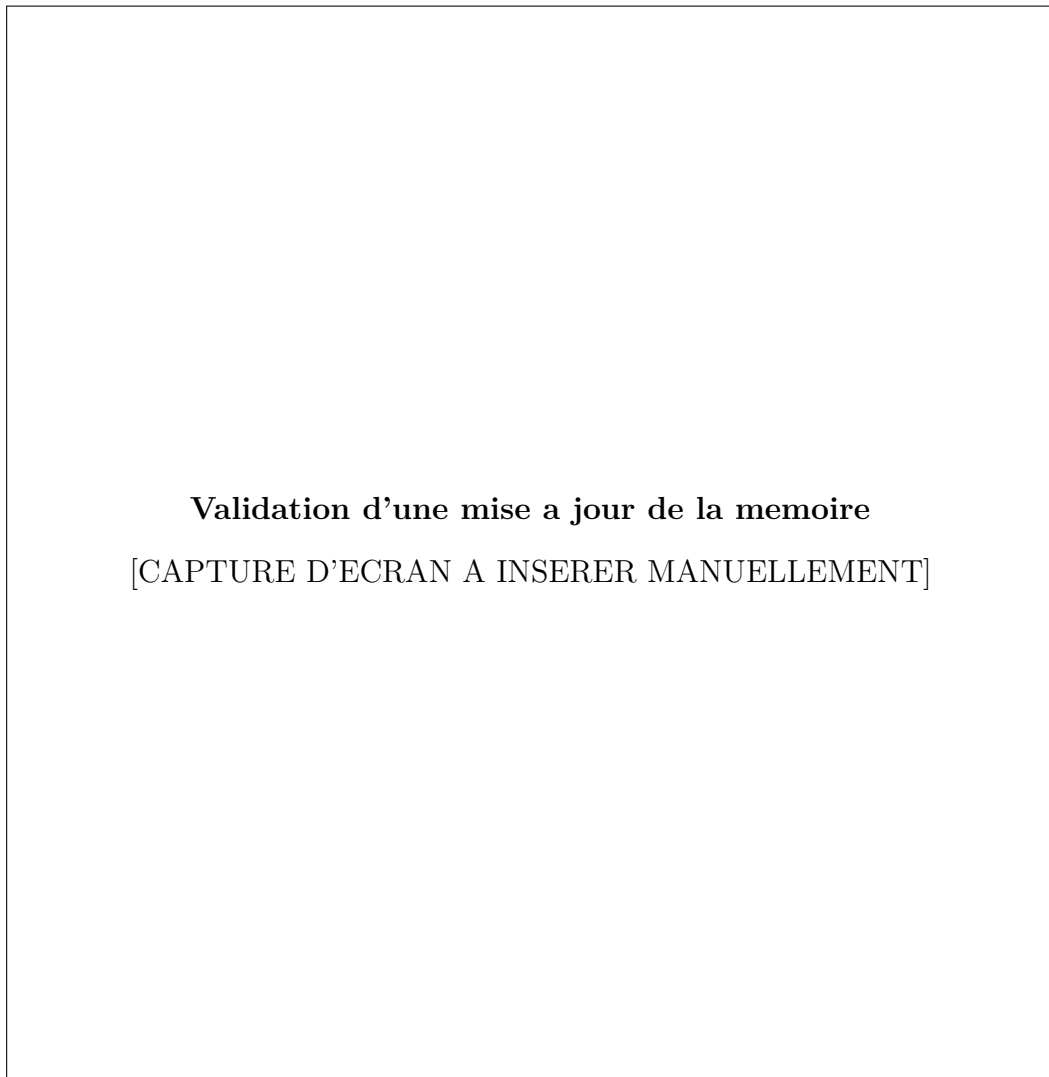


FIGURE 9.4 – Capture d'écran a inserer : Validation d'une mise a jour de la memoire

9.9.4 Validation d'une mise a jour reseau

Cette capture doit montrer une modification d'interface ou une relecture de la table reseau apres mise a jour. Elle doit etre commente selon l'environnement de test : changement reel d'adresse, activation d'interface ou simulation controlee.

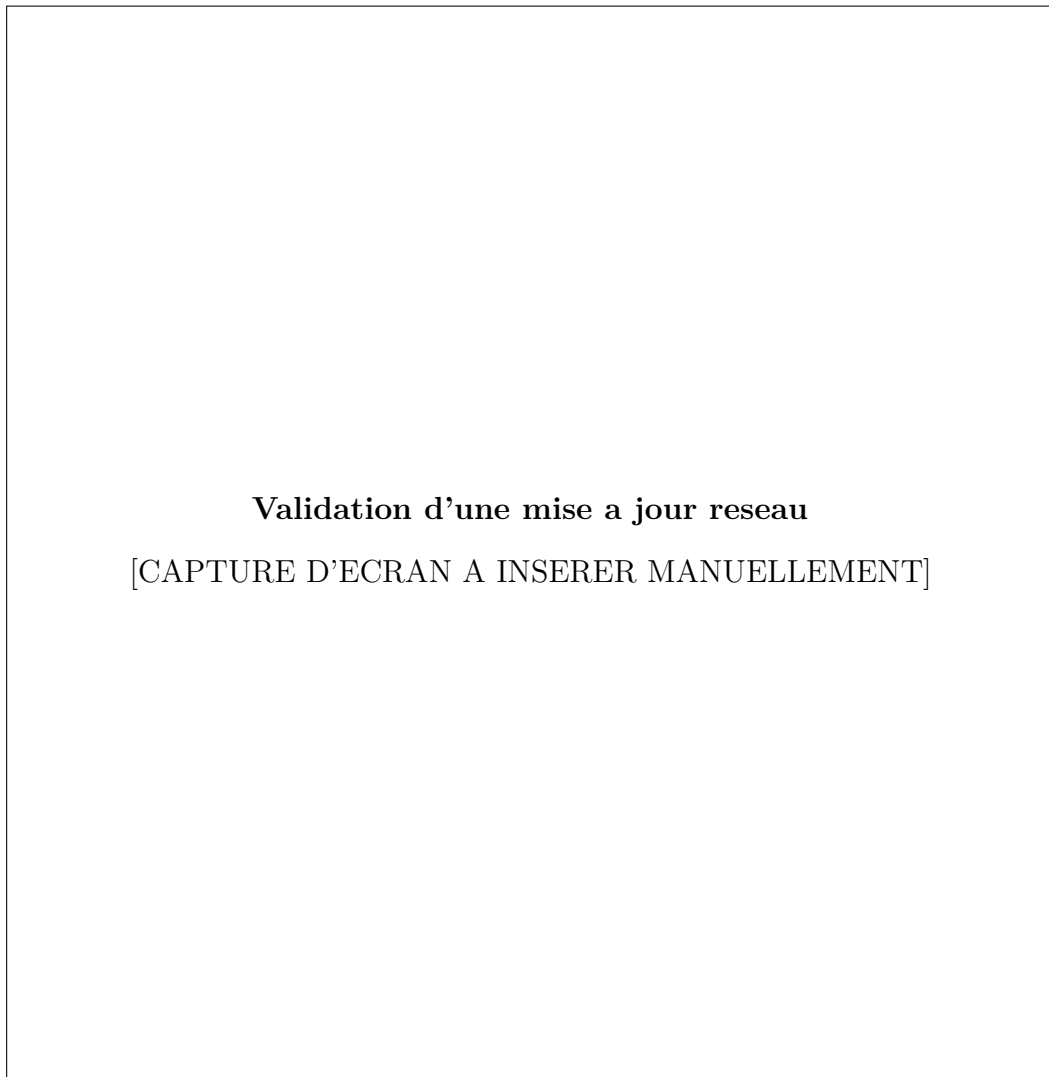


FIGURE 9.5 – Capture d'écran a inserer : Validation d'une mise a jour reseau

9.9.5 Validation du remplacement des processus

Cette capture doit montrer que la liste des processus est remplacée et non simplement ajoutée en doublon. Le commentaire doit expliquer le choix de supprimer les anciennes lignes avant d'ajouter l'image courante.

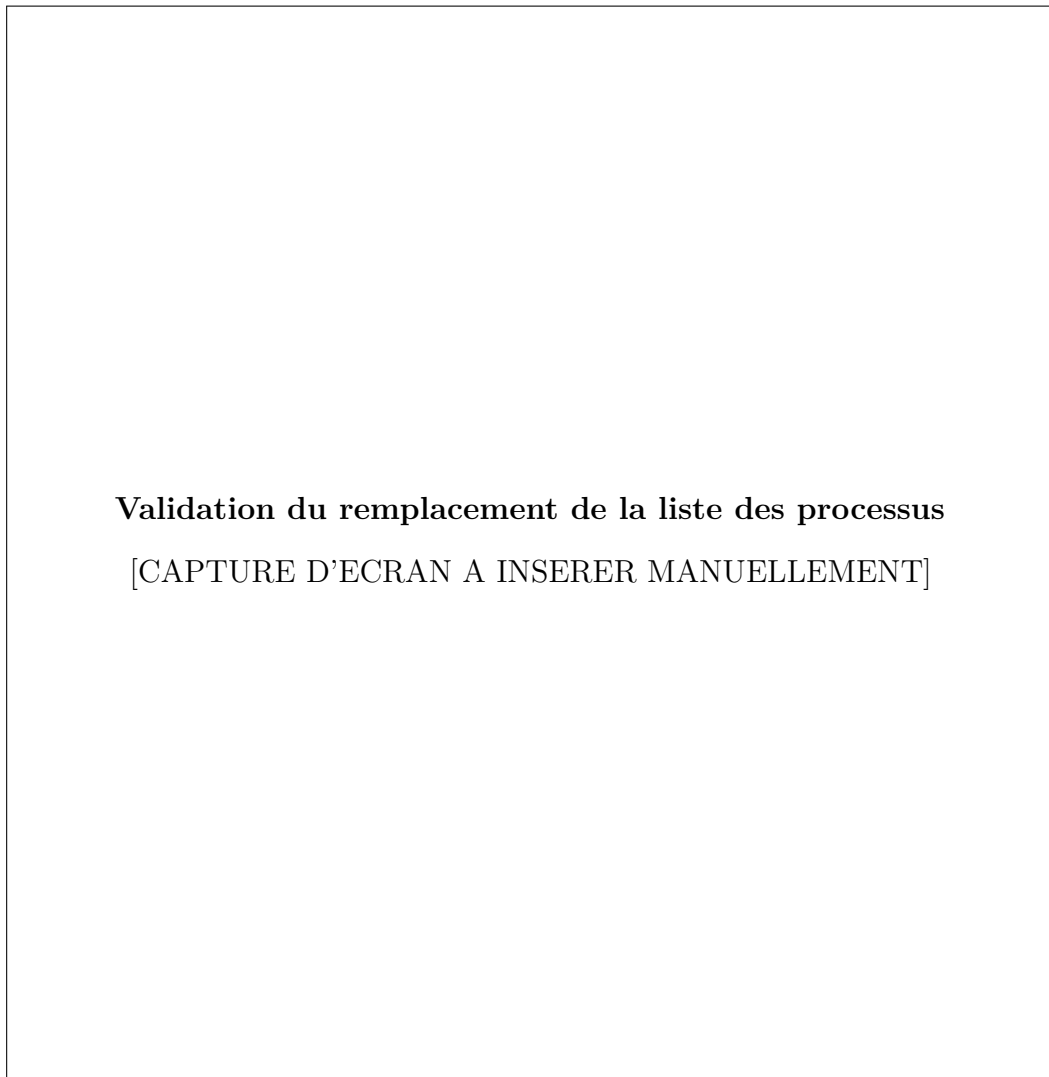


FIGURE 9.6 – Capture d'écran a inserer : Validation du remplacement de la liste des processus

9.9.6 Validation d'un refus d'agent non authentifié

Cette capture doit montrer une requete de mise a jour rejetee lorsque l'empreinte ne correspond pas. Elle permet de montrer que le serveur ne prend pas toutes les donnees entrantes sans verification. Le commentaire doit rappeler que ce controle reste minimal.

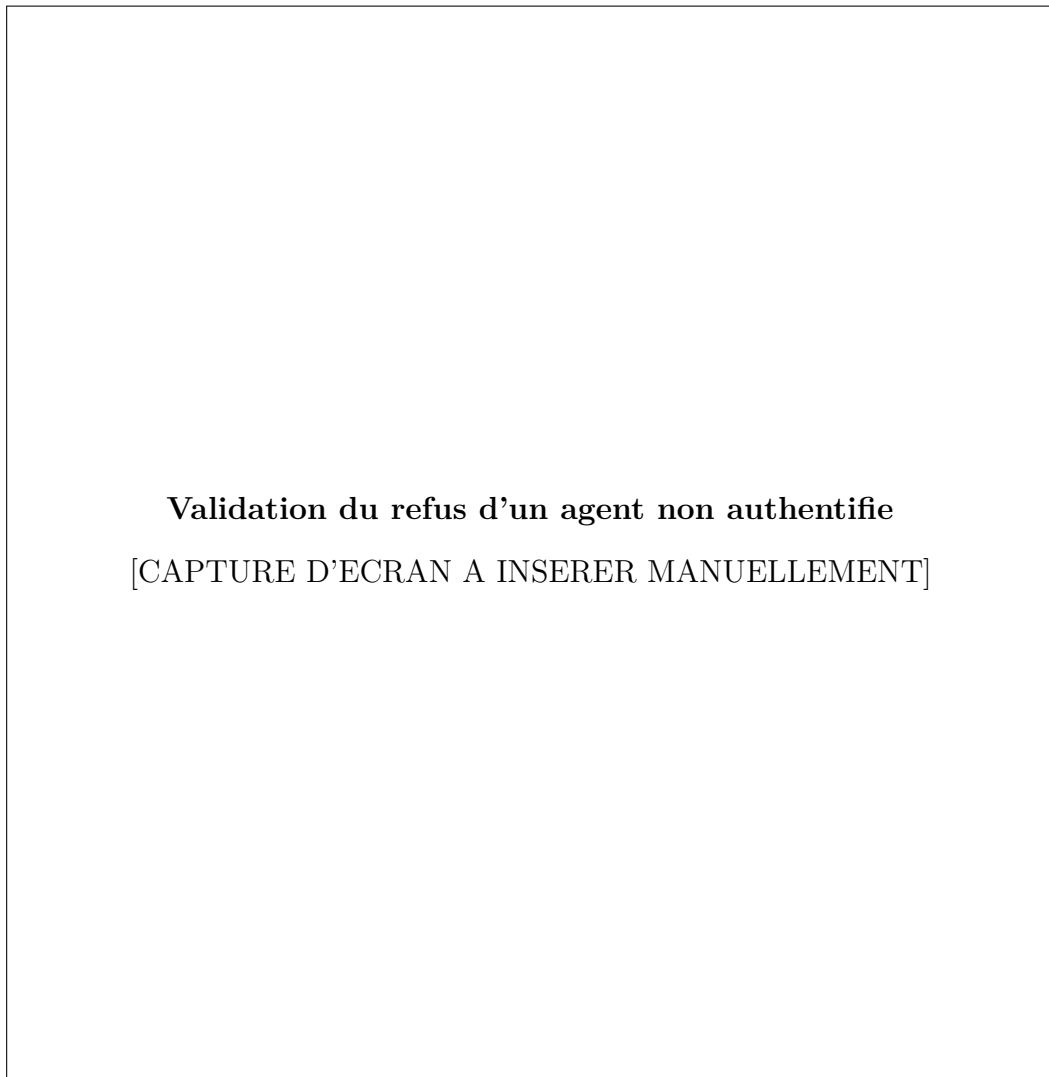


FIGURE 9.7 – Capture d'écran a inserer : Validation du refus d'un agent non authentifie

9.9.7 Validation d'un WebSocket absent

Cette capture doit montrer le cas ou une commande est envoyee vers un poste sans WebSocket actif. Le serveur doit retourner une erreur indiquant que la connexion n'est pas trouvee. Ce test est utile pour eviter une interface qui attendrait indefiniment une reponse.

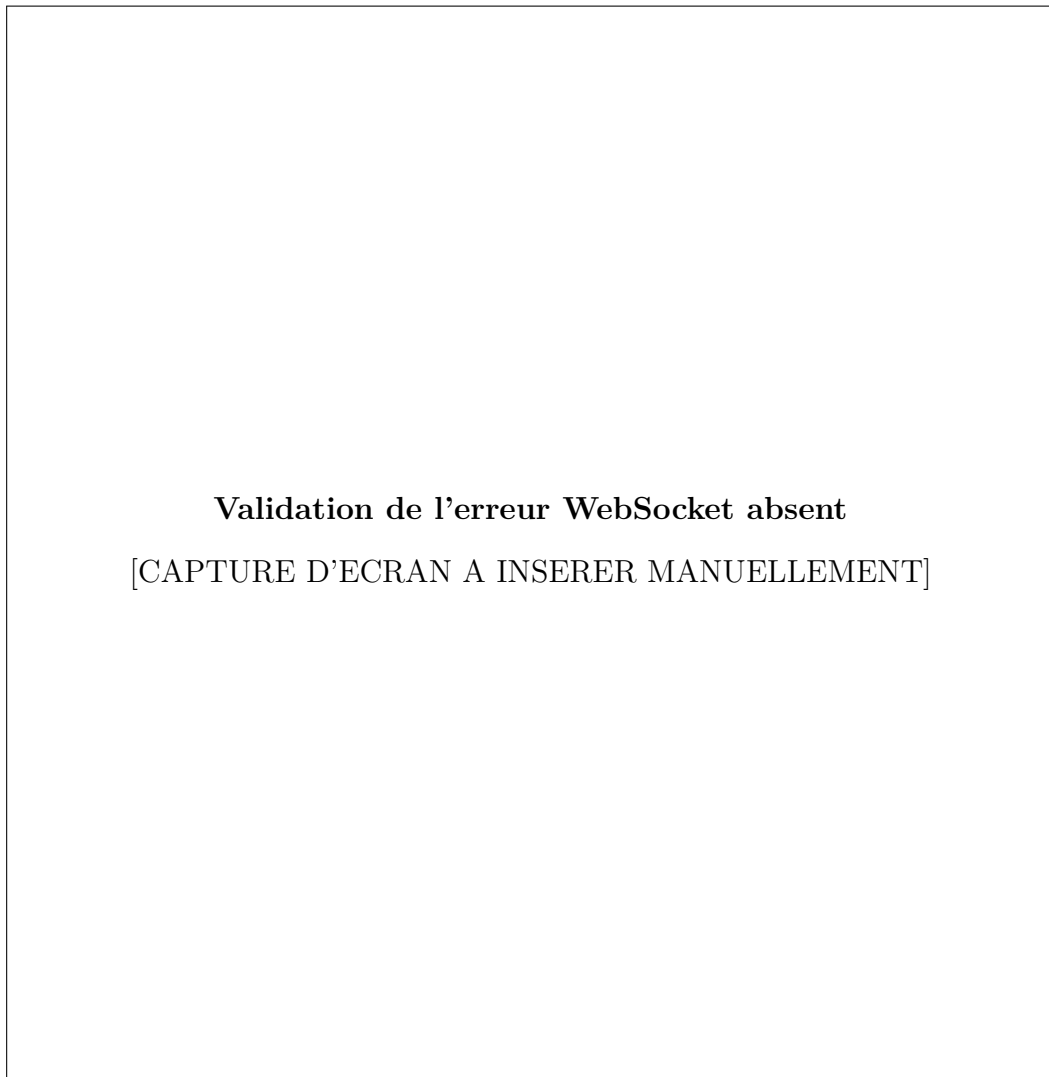


FIGURE 9.8 – Capture d'écran a inserer : Validation de l'erreur WebSocket absent

9.9.8 Validation d'une alerte USB brute

Cette capture doit montrer le message d'alerte USB transmis sous forme JSON ou affiche cote serveur. Le commentaire doit expliquer les champs disponibles et l'absence actuelle de persistance definitive.

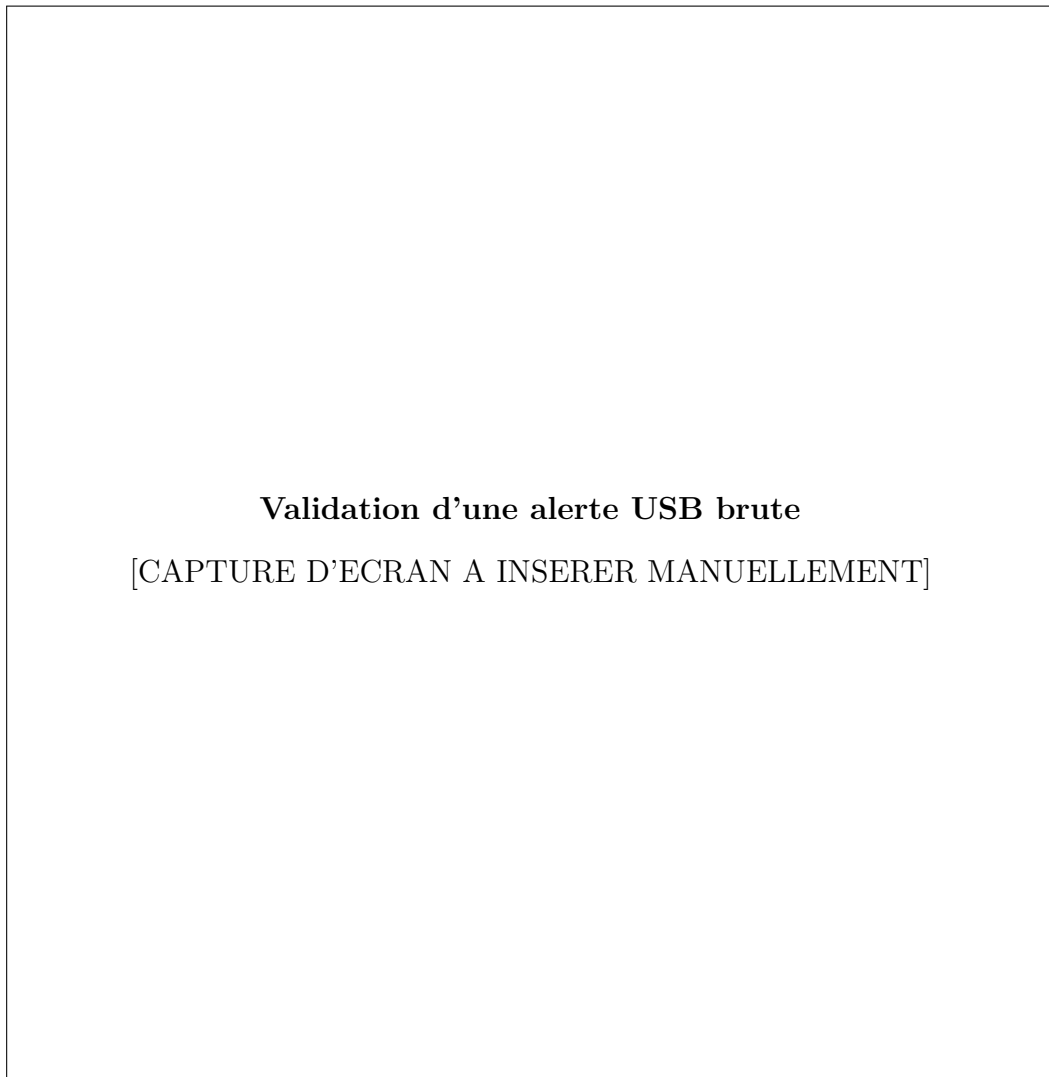
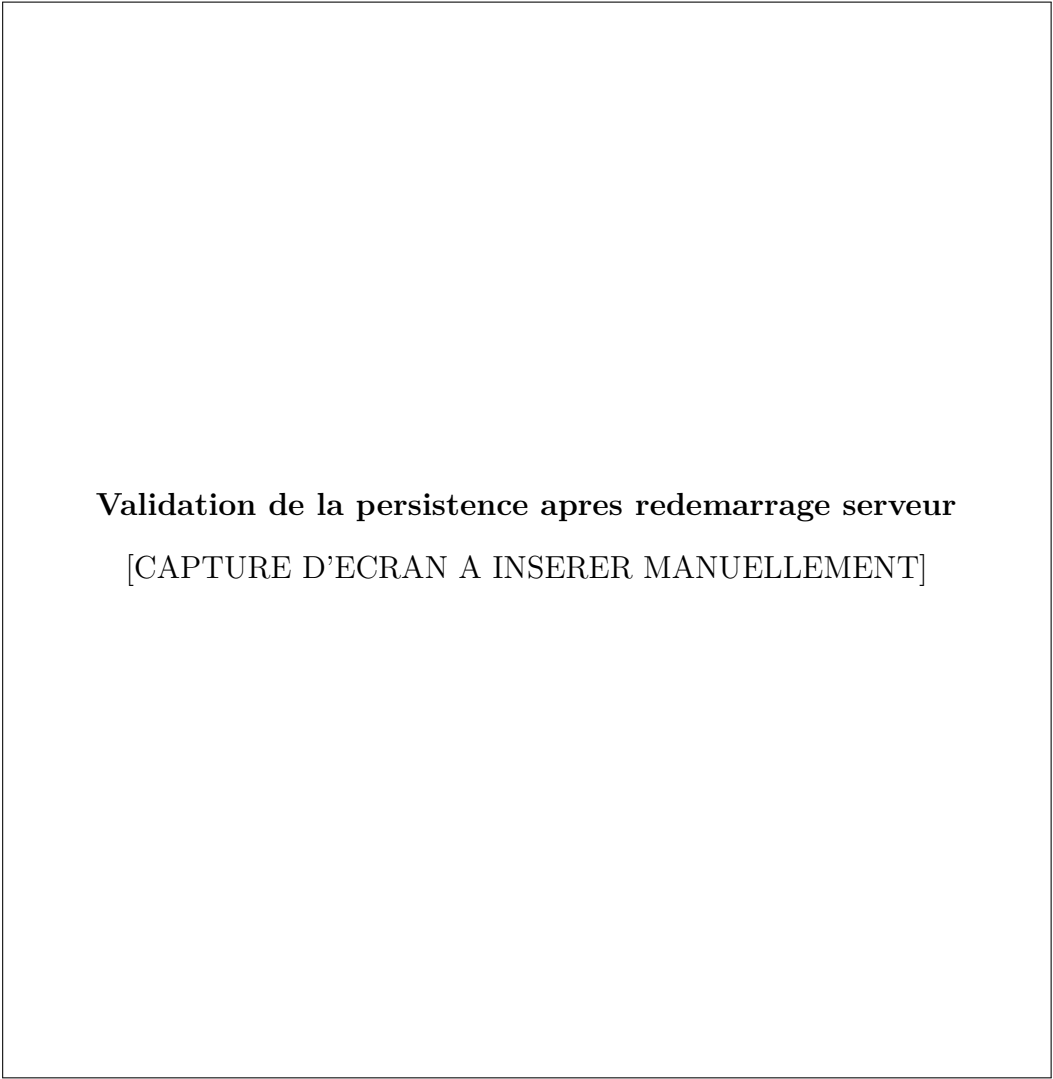


FIGURE 9.9 – Capture d'écran a inserer : Validation d'une alerte USB brute

9.9.9 Validation de la coherence apres redemarrage serveur

Cette capture doit montrer que les donnees persistees restent disponibles apres redemarrage du serveur, tant que la base SQLite est conservee. Elle permet de distinguer les donnees en memoire, comme les WebSockets actifs, des donnees persistantes, comme les postes enregistres.



Validation de la persistance apres redemarrage serveur
[CAPTURE D'ECRAN A INSERER MANUELLEMENT]

FIGURE 9.10 – Capture d'écran a inserer : Validation de la persistance apres redemarrage serveur

Chapitre 10

Conclusion generale et perspectives

10.1 Bilan du projet

Ernevil a permis de transformer un besoin concret de supervision en une architecture logicielle complete. Le projet couvre l'agent, le serveur, la base de donnees, l'interface, la communication REST, les WebSockets, le heartbeat, la collecte systeme et les premiers mecanismes d'alertes. Il montre qu'une plateforme de supervision locale peut etre construite progressivement avec des technologies accessibles et maitrisables.

10.2 Objectifs atteints

Les objectifs atteints sont l'enregistrement automatique des postes, la collecte des informations systeme, la persistance relationnelle, l'affichage des machines, la supervision detaillee d'un poste, le suivi de disponibilite par heartbeat, la mise en place d'un canal WebSocket pour les commandes et alertes et un module de detection d'intrusion par apprentissage supervise (HistGradientBoostingClassifier entraine sur CIC-IDS-2017, integre dans l'agent avec inference temps reel). Ces objectifs constituent un socle coherent pour un PFE de cybersecurite oriente supervision.

10.3 Difficultes et apprentissages

Les principales difficultes ont concerne la delimitation du perimetre, la portabilite entre systemes, la gestion de la fraicheur des donnees et la securite des fonctions sensibles. Le projet a aussi montre l'importance de documenter les limites. Dans un domaine comme la cybersecurite, pretendre qu'un prototype est complet peut etre plus dangereux que reconnaitre clairement ce qui doit etre renforce.

10.4 Perspectives

Les perspectives prioritaires sont l'authentification administrateur, le hachage des mots de passe, les roles, la securisation des WebSockets, la migration vers PostgreSQL, l'historisation des metriques, la persistance des alertes, la finalisation des pages vides, les rapports exportables, la gestion complete des emplacements et un moteur de regles de detection. Concernant le module ML, les evolutions possibles incluent le reentrainement du modele sur des donnees locales pour ameliorer la pertinence, l'extension de l'analyse a d'autres sources (processus, USB, heartbeat), l'ajout d'un apprentissage incremental et la mise en place d'un pipeline de retrain automatique.

10.5 Conclusion finale

Le projet Ernevil represente une base solide pour une plateforme de supervision locale. Il n'est pas encore une solution de securite finalisee, mais il montre une comprehension concrete des mecanismes necessaires : identifier, collecter, transmettre, stocker, afficher, alerter et agir. L'ajout d'un module de detection par apprentissage supervise temoigne d'une volonte d'explorer les approches ML tout en conservant une architecture explicable et testable. Le travail realise pendant le stage a permis d'acquérir une experience technique et critique, directement liee aux besoins d'un environnement de formation et aux exigences d'un projet de fin d'etudes en cybersecurite.

Annexe A

Annexes

A.1 Annexe A : endpoints principaux

TABLE A.1 – Endpoints REST et WebSocket

Endpoint	Methode	Role
/api/computers	POST	Ajouter un poste.
/api/computers	GET	Lister les postes.
/api/computers/live	GET	Lister les postes en ligne.
/api/computers/mem	GET/PUT	Lire ou mettre a jour la memoire.
/api/computers/net	GET/PATCH	Lire ou mettre a jour le reseau.
/api/computers/ps	GET/PUT	Lire ou remplacer les processus.
/api/computers/heartbeat	POST	Actualiser la presence.
/api/ws	WebSocket	Canal de commandes.
/api/ws/alert	WebSocket	Canal d'alertes.

A.2 Annexe B : captures a inserer

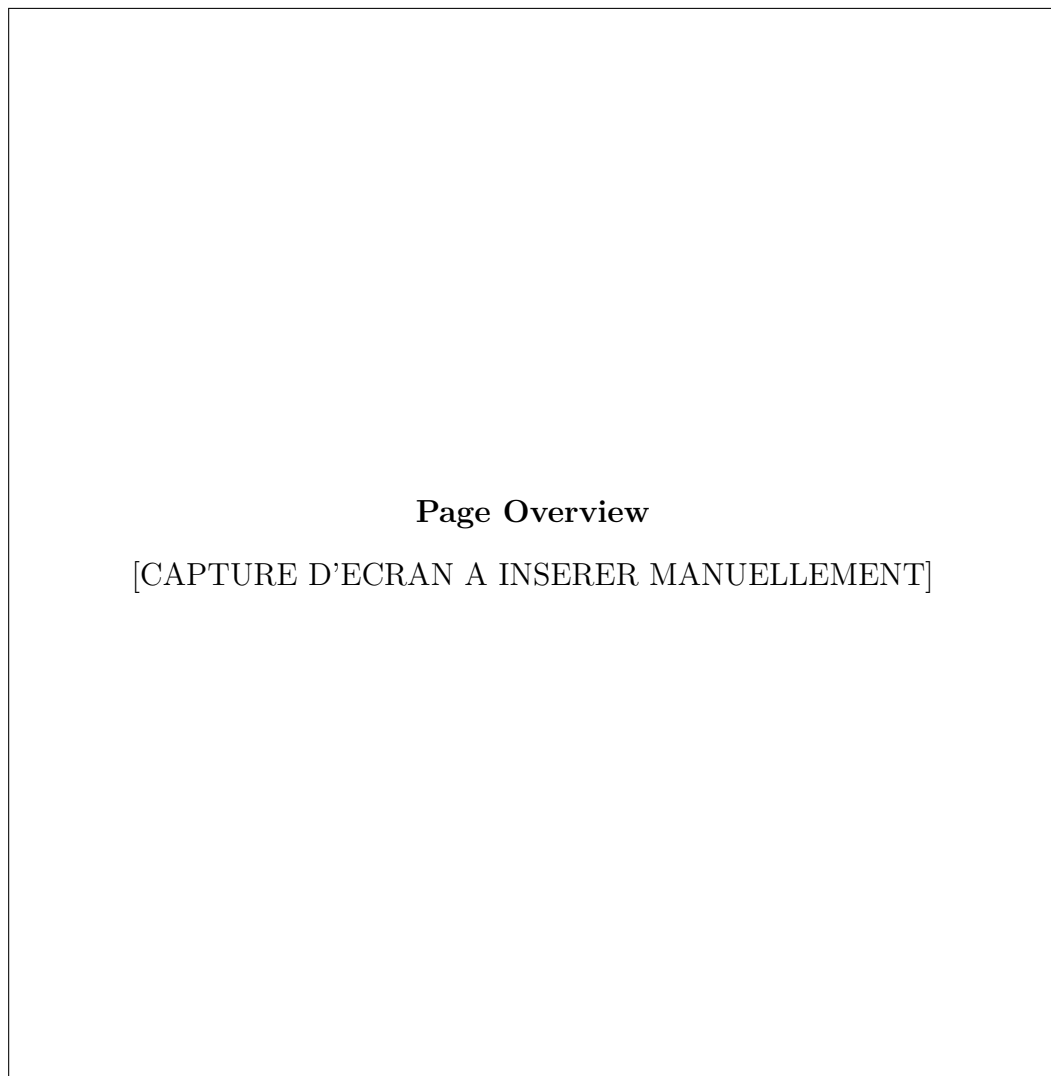


FIGURE A.1 – Capture d'écran a inserer : Page Overview

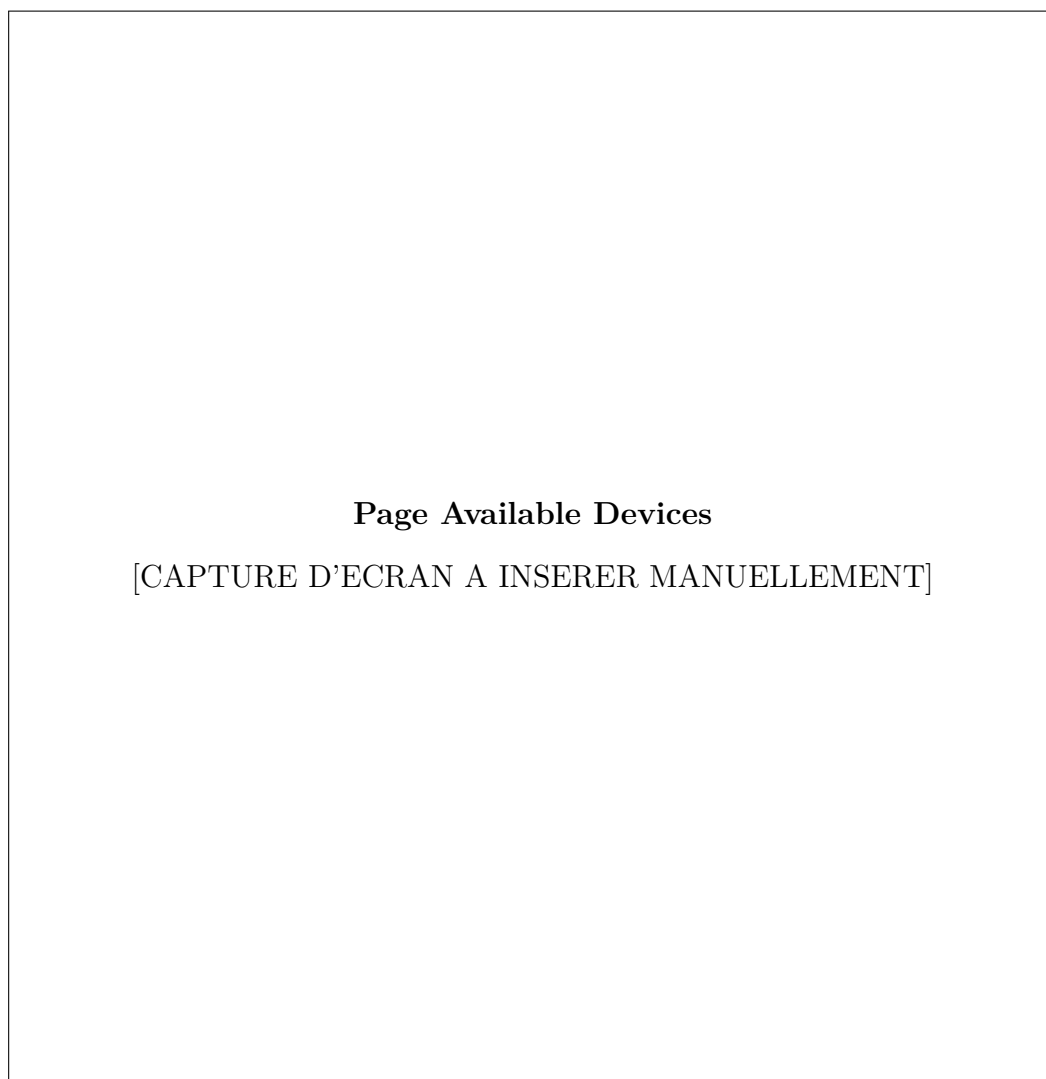


FIGURE A.2 – Capture d'écran a inserer : Page Available Devices

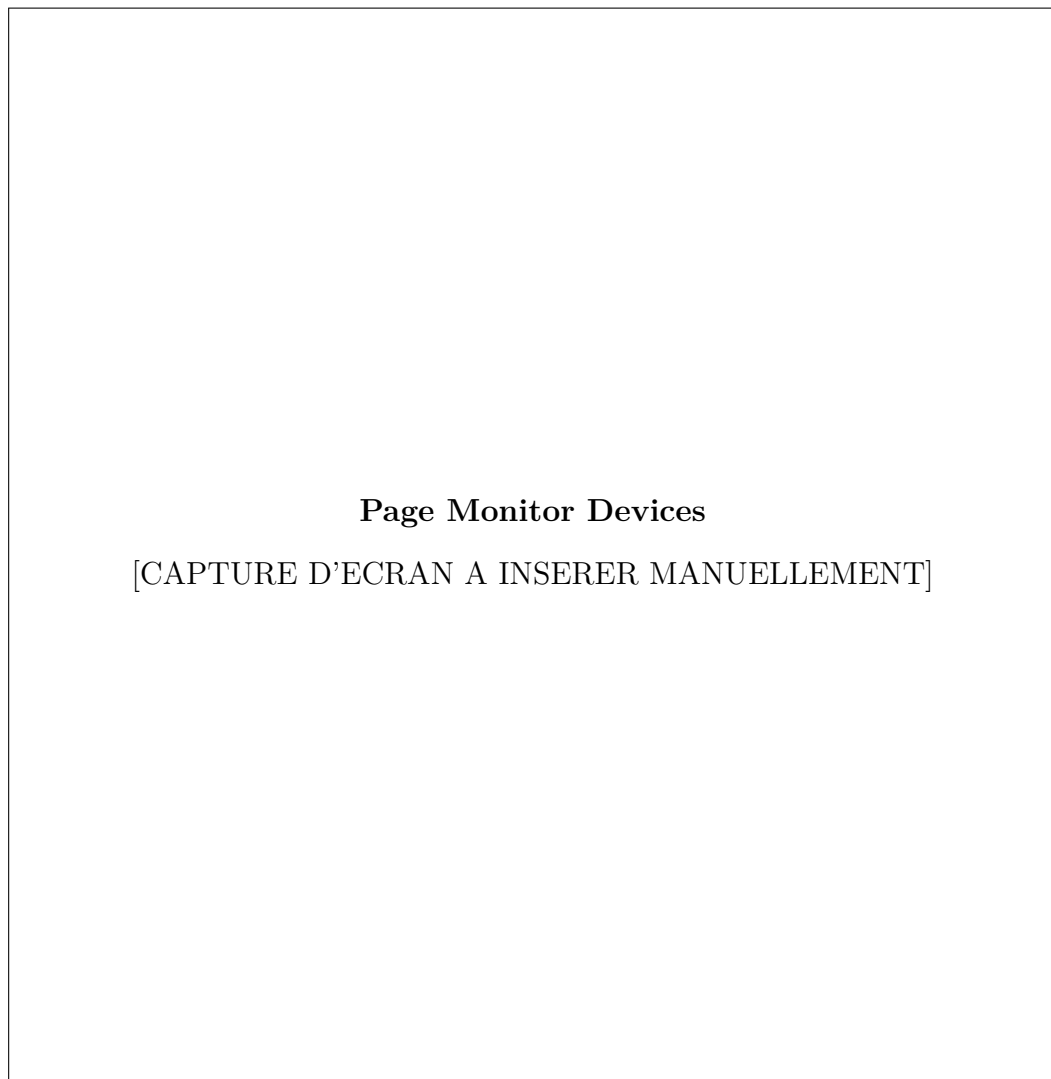


FIGURE A.3 – Capture d'écran a inserer : Page Monitor Devices



FIGURE A.4 – Capture d'écran a inserer : Execution d'une commande de test

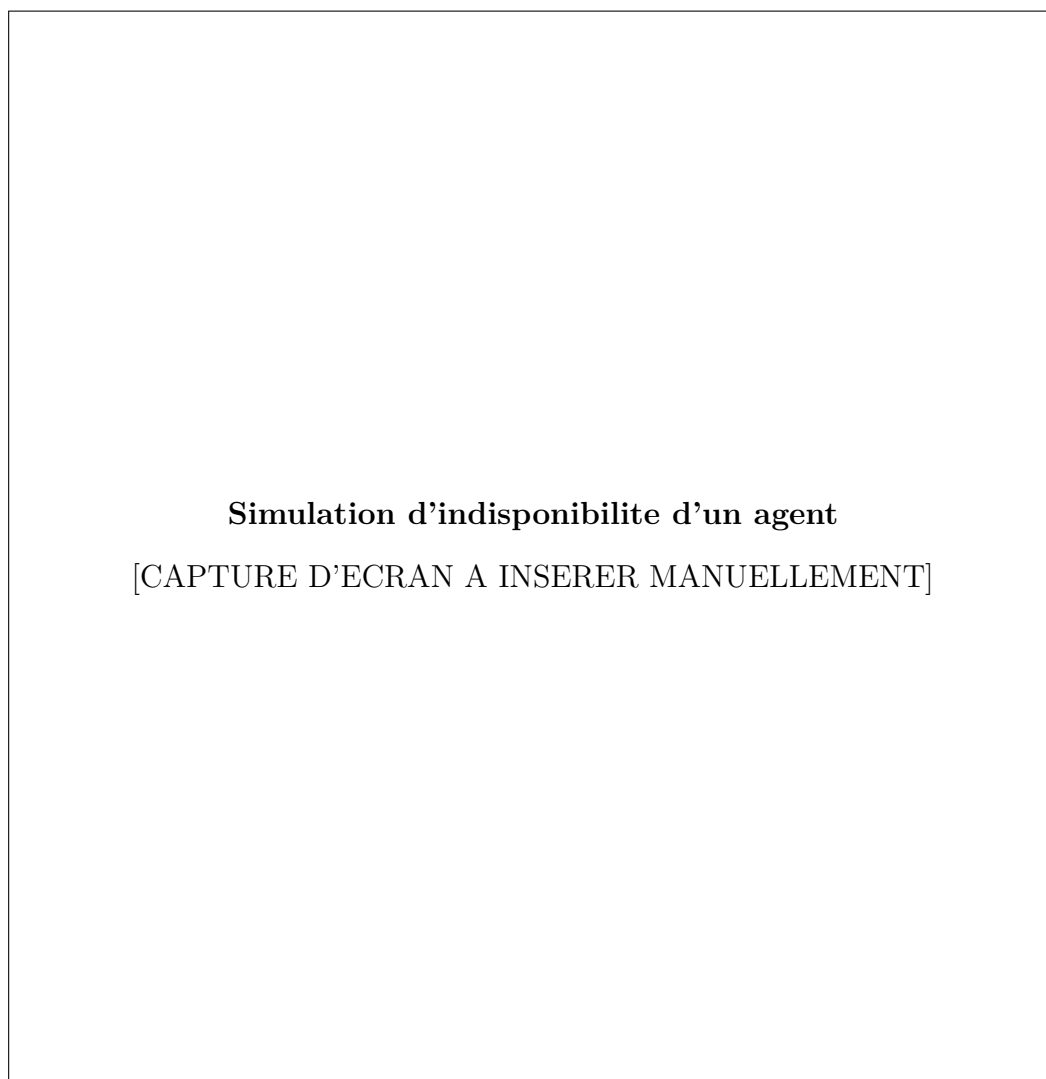


FIGURE A.5 – Capture d'écran a inserer : Simulation d'indisponibilite d'un agent

Annexe B

Bibliographie indicative

Bibliographie

- [1] Documentation officielle FastAPI, consultation pour la conception des routes REST et WebSocket.
- [2] Documentation officielle SQLAlchemy, consultation pour la modelisation ORM et les sessions.
- [3] Documentation officielle Streamlit, consultation pour la creation de l'interface.
- [4] Documentation officielle psutil, consultation pour la collecte des informations systeme.
- [5] Documentation officielle websockets, consultation pour la communication agent-serveur.
- [6] Pedregosa et al., Scikit-learn : Machine Learning in Python, JMLR 12, 2011, consultation pour l'implementation du HistGradientBoostingClassifier et de GridSearchCV.
- [7] Sharafaldin, Lashkari, Ghorbani, Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization, ICISSP 2018, consultation pour le jeu de donnees CIC-IDS-2017.
- [8] Documentation officielle Scapy, consultation pour la capture et l'analyse des paquets reseau.